Android remote

Android TV responsiveness

دليل الاتصال والاستخدام وتعديل التطبيقين والموقع

أجهزة Android TV المتوافقة، والربط الاختياري مع تطبيق الرسيفر، وبنية المشروع وواجهاته وطريقة بنائه ونشره.
يشمل شرح الربط المباشر مع

الإصدار 1.1.0 | مشروع Kotlin وJetpack Compose

رمز Downloader لتطبيق الرسيفر: 9187062

ملخص سريع

Android TV. الميزات الإضافية - مثل نقل الملفات وإدارة التطبيقات ومعاينة الشاشة - تحتاج التطبيق المرافق على الرسيفر أو ADB.
للاتصال. الريموت الأساسي يكتشف خدمة Android TV Remote على الشبكة ويقترن مباشرة بالتلفاز المتوافق، دون تثبيت responsiveness
يعالج هذا الإصدار مسارين منفصلين

الميزات	الرمز	ما الذي يحتاجه	المسار
أزرار الريموت الأساسية	6 خانات تظهر على شاشة التلفاز	Android TV Remote، والهاتف على Wi-Fi نفسها تلفاز يدعم	Android TV مباشر
وملفات وتطبيقات وJSON ولقطات شاشة ريموت	أرقام يعرضها التطبيق المرافق أو QR 7	Android TV وتشغيل خادم الاتصال تثبيت responsiveness	التطبيق المرافق
أوامر shell وبعض الإدارة المتقدمة	عنوان IP ومنفذ ADB	تفعيل تصحيح USB/الشبكة وموافقة RSA	ADB

الرمز ذو 6 خانات ليس رمز Downloader، والرمز ذو 7 أرقام ليس رمز Android TV. استخدم شاشة الاتصال المناسبة في التطبيق.

الخطوة الأولى: الاتصال المباشر بدون تثبيت التطبيق الثاني

- اتصل بالهاتف والتلفاز على شبكة Wi-Fi المنزلية نفسها، وتجنب شبكة الضيوف التي تعزل الأجهزة.
- افتح Android remote، ثم الأجهزة، ثم بحث Wi-Fi. اختر جهاز Android TV الذي يظهر ضمن Android TV مباشر.
- اضغط ابدأ الاقتران على التلفاز، ووافق على طلب الاتصال إذا ظهر تنبيه على الشاشة.
- أدخل رمز الاقتران ذي 6 خانات الذي يعرضه التلفاز، ثم اضغط تأكيد رمز التلفاز.
- بعد نجاح الاقتران، استخدم أزرار الريموت. يحفظ التطبيق شهادة الجهاز محليا لإعادة الاتصال لاحقا.

IP ومنفذها. لا يعني ذلك أن كل جهاز يظهر في الشبكة قابل للتحكم: يجب أن يعلن الجهاز خدمة Android TV Remote ويدعم بروتوكول الاقتران.
الاكتشاف يبحث عن خدمة DNS-SD اسمها _androidtvremote2._tcp. يستخدم Android NsdManager لحل اسم الخدمة إلى عنوان

الخطوة الثانية: الاتصال بالتطبيق المرافق

- ثبت Android TV responsiveness على الرسيفر أو Android TV وافتحه.
- شغل خادم الاتصال من شاشة الرسيفر. يعرض التطبيق كودا من 7 أرقام ويمكنه عرض QR.
- في الهاتف، افتح الأجهزة واختر تطبيق الرسيفر أو استخدم مسح QR. عند الإدخال اليدوي، اكتب IP والكود ذي 7 أرقام.
- تأكد أن الهاتف والرسيفر على شبكة LAN نفسها وأن الراوتر يسمح باتصالات الأجهزة المحلية.

عبر TCP على المنفذ 39001. يرسل التطبيق المرافق تأكيدا أو رسالة رفض، ويعرض تطبيق الهاتف الرسالة بدل ترك المستخدم أمام اتصال معلق.
يقوم تطبيق الرسيفر بالإعلان عبر UDP على المنفذ 39000، ويستقبل أوامر JSON مؤطرة

إذا فشل الاتصال أو رفض التلفاز الرمز

- تأكد أن الجهازين متصلان بنفس شبكة Wi-Fi وليسا على شبكة ضيوف أو شبكتين معزولتين.
- في المسار المباشر، ابحث عن Android TV Remote وليس عن التطبيق المرافق. استخدم الرمز الذي يظهر على التلفاز بعد بدء اقتران جديد.

إذا انتهت مهلة الرمز، ابدأ الاقتران مجددا، ثم أدخل الرمز الجديد الظاهر على التلفاز. •

فقد لا يدعم خدمة Android TV Remote أو قد تكون الخدمة متوقفة. الاتصال اليدوي بعنوان IP لا يجعل جهازا غير متوافق يدعم البروتوكول. • إن لم يظهر التلفاز،

في مسار التطبيق المرافق، تأكد أن خادم الرسيفر يعمل، وأن الكود الحالي من 7 أرقام صحيح. أعد تشغيل الخدمة إذا تغير عنوان IP. •

أوقف VPN مؤقتا على الهاتف، وتحقق من إعدادات عزل العملاء في الراوتر والجدار الناري. •

مميزات كل تطبيق

Android remote - تطبيق الهاتف

- لوحة تحكم مستوحاة من Android TV Remote تشمل D-Pad ولوحة لمس ولوحة أرقام وأزرار الصوت والقنوات.
- اكتشاف أجهزة Android TV المتوافقة وخادم التطبيق المرافق على Wi-Fi.
- الاقتران المباشر مع Android TV Remote برمز التلفاز، والاقتران بالتطبيق المرافق عبر QR أو كود من 7 أرقام.
- قائمة تطبيقات الرسيفر وفتح التطبيقات أو حذفها عبر التطبيق المرافق أو ADB، بحسب صلاحيات Android.
- إرسال الصور والفيديو والملفات، وإرسال JSON مخصص إلى التطبيق المرافق.
- معاينة شاشة على شكل لقطات متتابعة ونافذة عائمة Picture-in-Picture عند دعم الجهاز.
- واجهة عربية وإنجليزية، ثيم بنفسجي رمادي أو أبيض، أوامر ADB مخفية في الإعدادات وعشرة أوامر مفضلة قابلة للتحرير.

Android TV responsiveness - التطبيق المرافق

- خادم محلي يعمل على الرسيفر ويعلن عن نفسه على الشبكة عند تشغيل الخدمة.
- يعرض حالة الخادم ورمز اقتران من 7 أرقام وQR، ويستقبل أوامر الهاتف.
- يدعم واجهة الملفات والوسائط في المشروع، واستقبال ملفات الجوال، وإدارة أو فتح التطبيقات حسب إمكانات النظام.
- يعرض لقطات شاشة عبر خدمة Accessibility التي يفعلها المستخدم، مع مشغل صور وفيديو.
- تفضيلات اللغة والثيم محفوظة محليا.

حدود الميزات والمسارات

ADB	التطبيق المرافق	Android TV مباشر	الميزة
نعم	نعم وفق صلاحيات Accessibility	نعم للأجهزة المتوافقة	أزرار الريموت
إدخال IP يدوي	UDP لخدمة المشروع	لخدمة Android TV Remote DNS-SD	اكتشاف Wi-Fi
نعم غالبا	نعم عند السماح	لا	إدارة التطبيقات
نعم	نعم	لا	نقل الملفات
حسب الأمر/الأداة	لقطات دورية عند تفعيل Accessibility	لا	معاينة الشاشة
أوامر shell	نعم عبر بروتوكول المشروع	لا	JSON مخصص

كذلك قد يمنع Android تنفيذ بعض الأوامر من الخلفية أو يعرض مربع تأكيد قبل حذف تطبيق. التطبيقات التي تحمي الشاشة قد تمنع التقاط صورها.

المرافق TCP محلي ومخصص لهذا المشروع؛ لا يعادل تشفير TLS في بروتوكول Android TV Remote. استخدمه على شبكة منزلية موثوقة.

الاتصال

المنافذ والرموز

ملاحظة	المنفذ	النقل	الاستخدام
Android NsdManager	حسب الخدمة	DNS-SD / mDNS	اكتشاف Android TV Remote
رمز 6 خانات على التلفاز	6467	TLS/TCP	اقتران Android TV Remote
بعد الاقتران	6466	TLS/TCP	جلسة الريموت Android TV
يعمل عند تشغيل خادم الرسيفر	39000	UDP broadcast	اكتشاف التطبيق المرافق
JSON مؤطر بطول 4 بايت	39001	TCP	أوامر التطبيق المرافق
يطلب موافقة RSA	عادة 5555	ADB/TCP	ADB الشبكي

كيف تعدل التطبيقين

المتطلبات

- Android Studio حديث، JDK 17، وAndroid SDK Platform 37 حسب إعداد المشروع.
- افتح مجلد ReceiverControlAndroid من Android Studio ثم نفذ Gradle Sync.
- ابن التطبيقين معا من نافذة Gradle أو استخدم الأمر في Terminal:

```
.\gradlew.bat :controller:assembleDebug :receiver:assembleDebug
```

وAPK الرسيفر في receiver/build/outputs/apk/debug/receiver-debug.apk.
بعد البناء تجد APK الجوال في controller/build/outputs/apk/debug/controller-debug.apk،

أين تعدل كل شيء؟

المسار النسبي داخل المشروع	الغرض
controller/src/main/java/com/monib/receivercontrol/controller/ControllerUi.kt	واجهة الهاتف والريموت
controller/src/main/java/com/monib/receivercontrol/controller/MainActivity.kt	سلوك واتصال تطبيق الهاتف
controller/src/main/java/com/monib/receivercontrol/controller/AndroidTvDiscovery.kt	اكتشاف Android TV
controller/src/main/java/com/monib/receivercontrol/controller/AndroidTvRemoteClient.kt	اقتران Android TV Remote
controller/src/main/java/com/monib/receivercontrol/controller/LanClient.kt	عميل TCP وUDP للتطبيق المرافق
receiver/src/main/java/com/monib/receivercontrol/receiver/LocalReceiverService.kt	خادم الرسيفر
receiver/src/main/java/com/monib/receivercontrol/receiver/ReceiverDashboard.kt	واجهة الرسيفر
receiver/src/main/java/com/monib/receivercontrol/receiver/MainActivity.kt وPlayerActivity.kt	السلوك المحلي والوسائط
website/index.html	الموقع

تعديل الواجهة في Android Studio

- افتح ControllerUi.kt للواجهة الرئيسية. ابحث عن RemotePage أو DevicesPage أو SettingsPage لتعديل الشاشة المطلوبة.
- ونصوص العربية والإنجليزية تظهر غالبا في loc(ar, النص العربي, النص الإنجليزي). أضف النص باللغتين حتى لا تبقى ترجمة ناقصة.
الألوان والثيم محددان في ControllerUi.kt،
- افتح ReceiverDashboard.kt لتعديل شاشة الرسيفر. أضف مكونات Compose داخل Composable المناسب.
- في Android Studio عند توفره، ثم شغل التطبيق على محاكي أو جهاز حقيقي. اختبر ميزات Wi-Fi على شبكة فعلية لأن المحاكي قد لا يرى التلفاز.
استخدم Preview

تعديل المشروع في VS Code

- في VS Code. استخدم إضافة Kotlin للتلوين والتنقل، لكن Android Studio هو الأسهل لمزامنة Gradle وإدارة SDK ومعاينة Compose.
افتح مجلد ReceiverControlAndroid
- وشغل gradlew.bat مع اسم المهمة. استخدم :controller:assembleDebug أو :receiver:assembleDebug لبناء تطبيق منفرد.
افتح Terminal في جذر المشروع

- بعد التعديل، راجع أخطاء Kotlin وGradle في نافذة Problems أو خرج Terminal. ثبت APK الناتج على الجهاز المطلوب قبل نشره.

تعديل الموقع والنشر على GitHub Pages

- في المشروع. صفحة GitHub Pages المنشورة داخل AndroidRemote-Site1-Deploy/index.html في المستودع. مصدر الموقع داخل مجلد website/index.html
- في data-ar وdata-en معا. روابط APK وملفات الدليل والمصدر يجب أن تطابق أسماء الملفات الموجودة بجوار index.html في مجلد النشر. عدل النصوص
- كود Downloader الحالي للرسيفر هو 9187062. رابط AFTVnews موجود بصفحة الموقع بصيغة http://aftv.news/9187062.
- PDF/ZIP إلى AndroidRemote-Site1-Deploy ثم اعمل commit وpush إلى main. سير العمل pages.yml ينشر هذا المجلد تلقائيا. للنشر، انسخ index.html وAPK وملفات
- بعد كل دفع، افتح Actions وتأكد أن مهمة Deploy Android Remote site انتهت بنجاح، ثم حدث الموقع في المتصفح.

ADB من الإعدادات

بعض الأجهزة تحتاج تفعيل Network debugging أولا. وضع Wireless Debugging الذي يقترن على منفذ مؤقت ليس مطبقا في هذا المشروع. (غالبا 5555) واضغط اتصال؛ وافق على RSA من التلفاز إذا طلب ذلك. يمكنك تشغيل أمر shell مباشرة أو تحرير أسماء وأوامر المفضلات العشرة. تظهر أدوات ADB تحت الإعدادات المتقدمة. أدخل IP ومنفذ ADB

يستطيع تغيير إعدادات النظام أو حذف بيانات. عند عدم توفر ADB، استخدم مسار Android TV Remote أو التطبيق المرافق بحسب الميزة المطلوبة. لا تضع أوامر لا تعرف تأثيرها؛ shell

المراجع والقرارات الفنية

على Android NsdManager ودليل Android الرسمي: https://developer.android.com/develop/connectivity/wifi/use-nsd اعتمد تنفيذ اكتشاف الشبكة

. ملفات المكتبة المرفقة مرخصة وفق MIT، ونسخة الترخيص محفوظة في THIRD_PARTY_LICENSES.
بروتوكول Android TV Remote v2 ومكتبة Java المرجعية: https://github.com/outofcontextapps/androidtvremote

حتى يميز بين اكتشاف Android TV المباشر وخادم التطبيق المرافق؛ فالبحث العام عن خدمات الشبكة لا يوفر بروتوكول تحكم موحدا لكل الأجهزة. تمت مراجعة الملفين اللذين أرفقهما المستخدم كمرجع لأمثلة Kotlin. عدل المشروع الأمثلة

ملحق: ملفات Kotlin وواجهات المشروع

الأصلية بحروفها العربية وقابلة للبناء. ملفات بروتوكول Android TV المأخوذة من المكتبة المرجعية موجودة كاملة في حزمة المصدر مع ترخيصها. قراءة الأسطر من اليسار إلى اليمين، تظهر الحروف غير الإنجليزية في الشيفرة بصيغة Unicode مثل \u0627؛ حزمة المصدر ZIP تحتوي الملفات تضم الصفحات التالية ملفات Kotlin التي يملكها التطبيق. لسهولة

داخل Android Studio بدل نسخ النص من PDF. يوجد مصدر صفحة الموقع website/index.html وإعدادات Gradle في حزمة ZIP كذلك. عند تعديل المشروع، استخدم ملف المصدر

controller/src/main/java/com/monib/receivercontrol/controller/AdbShellClient.kt

```
package com.monib.receivercontrol.controller

import android.security.keystore.KeyGenParameterSpec
import android.security.keystore.KeyProperties
import android.util.Base64
import java.io.ByteArrayOutputStream
import java.io.DataInputStream
import java.io.DataOutputStream
import java.io.EOFException
import java.math.BigInteger
import java.net.InetSocketAddress
import java.net.Socket
import java.security.KeyPair
import java.security.KeyPairGenerator
import java.security.KeyStore
import java.security.Signature
import java.security.interfaces.RSAPublicKey
import java.util.concurrent.TimeUnit

/** Minimal ADB transport client for legacy ADB-over-TCP receivers (normally port 5555). */
class AdbShellClient(private val keyStoreAlias: String = "receiver-control-adb-v1") {
    fun shell(host: String, port: Int, command: String): String =
        String(openService(host, port, "shell:$command\u0000"), Charsets.UTF_8).trimEnd('\u0000', '\r', '\
n')

    fun shellBytes(host: String, port: Int, command: String): ByteArray =
        openService(host, port, "shell:$command\u0000")

    fun push(host: String, port: Int, remoteName: String, input: java.io.InputStream, size: Long) {
        require(remoteName.matches(Regex("[\\p{L}\\p{N}._ -]{1,160}"))) { "\u0627\u0633\u0645 \u0627\u0644\
u0645\u0644\u0641 \u063A\u064A\u0631 \u0635\u0627\u0644\u062D" }
        val (socket, inputStream, outputStream) = connected(host, port)
        socket.use {
            inputStream.use { adbIn ->
                outputStream.use { adbOut ->
                    val remoteId = open(adbIn, adbOut, "sync:\u0000")
                    val safePath = "/sdcard/Download/$remoteName,0644".toByteArray(Charsets.UTF_8)
                    sendSyncRecord(adbIn, adbOut, remoteId, "SEND", safePath)

                    val chunk = ByteArray(3500)
                    var remaining = size
                    while (remaining > 0) {
                        val count = input.read(chunk, 0, minOf(chunk.size.toLong(), remaining).toInt())
                        if (count < 0) throw EOFException("\u0627\u0646\u062A\u0647\u0649 \u0627\u0644\
u0645\u0644\u0641 \u0642\u0628\u0644 \u0627\u0643\u062A\u0645\u0627\u0644 \u0627\u0644\u0625\u0631\u0633\
u0627\u0644")
                        val record = ByteArrayOutputStream(count + 8)
                        writeAscii(record, "DATA")
                        writeLeInt(record, count)
                        record.write(chunk, 0, count)
                        sendServiceData(adbIn, adbOut, remoteId, record.toByteArray())
                        remaining -= count
                    }
                    val done = ByteArrayOutputStream(8)
                    writeAscii(done, "DONE")
                    writeLeInt(done, (System.currentTimeMillis() / 1000L).toInt())
                    sendServiceData(adbIn, adbOut, remoteId, done.toByteArray())
                    val response = readServiceResponse(adbIn, adbOut, maxBytes = 4096)
                    val tag = response.take(4).toByteArray().toString(Charsets.US_ASCII)
                    if (tag == "FAIL") {
                        val length = response.copyOfRange(4, 8).toLeInt()
                        throw IllegalStateException(response.copyOfRange(8, minOf(8 + length, response.
size)).toString(Charsets.UTF_8))
                    }
                    if (tag != "OKAY") throw IllegalStateException("\u0631\u062F ADB \u063A\u064A\u0631 \
u0645\u062A\u0648\u0642\u0639 \u0623\u062B\u0646\u0627\u0621 \u0646\u0642\u0644 \u0627\u0644\u0645\u0644\
u0641")
                    sendPacket(adbOut, "CLSE", LOCAL_ID, remoteId, byteArrayOf())
                }
            }
        }
    }

    private fun openService(host: String, port: Int, service: String): ByteArray {
        val (socket, inputStream, outputStream) = connected(host, port)
        socket.use {
            inputStream.use { adbIn ->
                outputStream.use { adbOut ->
                    open(adbIn, adbOut, service)
                    return readServiceResponse(adbIn, adbOut, maxBytes = 32 * 1024 * 1024)
                }
            }
        }
    }

    private fun connected(host: String, port: Int): Triple<Socket, DataInputStream, DataOutputStream> {
        val socket = Socket()
        socket.tcpNoDelay = true
        socket.connect(InetSocketAddress(host, port), 4500)
```

```
        socket.soTimeout = 45_000
        val input = DataInputStream(socket.getInputStream())
        val output = DataOutputStream(socket.getOutputStream())
        try {
            authenticate(input, output)
        } catch (e: Exception) {
            socket.close()
            throw e
        }
        return Triple(socket, input, output)
    }

    private fun authenticate(input: DataInputStream, output: DataOutputStream) {
        val banner = "host::\u0000".toByteArray(Charsets.UTF_8)
        sendPacket(output, "CNXN", ADB_VERSION, 4096, banner)
        var authStep = 0
        val deadline = System.nanoTime() + TimeUnit.SECONDS.toNanos(40)
        while (System.nanoTime() < deadline) {
            val packet = readPacket(input)
            when (packet.command) {
                cmd("CNXN") -> return
                cmd("AUTH") -> {
                    if (packet.arg0 != AUTH_TYPE_TOKEN || packet.data.size != 20) {
                        throw IllegalStateException("\u062A\u0641\u0627\u0648\u0636 ADB \u063A\u064A\u0631 \
u0645\u062F\u0639\u0648\u0645 \u0639\u0644\u0649 \u0647\u0630\u0627 \u0627\u0644\u0645\u0646\u0641\u0630")
                    }
                    val keys = getOrCreateKeyPair()
                    when (authStep++) {
                        0 -> {
                            val signer = Signature.getInstance("SHA1withRSA")
                            signer.initSign(keys.private)
                            signer.update(packet.data)
                            sendPacket(output, "AUTH", AUTH_TYPE_SIGNATURE, 0, signer.sign())
                        }
                        1 -> sendPacket(output, "AUTH", AUTH_TYPE_RSAPUBLICKEY, 0, adbPublicKey(keys.public
as RSAPublicKey))
                        else -> Thread.sleep(150)
                    }
                }
                cmd("CLSE") -> throw IllegalStateException("\u0631\u0641\u0636 \u0627\u0644\u0631\u0633\
u064A\u0641\u0631 \u0627\u062A\u0635\u0627\u0644 ADB")
            }
        }
        throw java.net.SocketTimeoutException("\u0627\u0646\u062A\u0647\u062A \u0645\u0647\u0644\u0629 \
u0645\u0635\u0627\u062F\u0642\u0629 ADB. \u0648\u0627\u0641\u0642 \u0639\u0644\u0649 \u0628\u0635\u0645\
u0629 \u0627\u0644\u0645\u0641\u062A\u0627\u062D \u0641\u064A \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \
u062B\u0645 \u0623\u0639\u062F \u0627\u0644\u0645\u062D\u0627\u0648\u0644\u0629.")
    }

    private fun open(input: DataInputStream, output: DataOutputStream, service: String): Int {
        sendPacket(output, "OPEN", LOCAL_ID, 0, service.toByteArray(Charsets.UTF_8))
        while (true) {
            val packet = readPacket(input)
            when (packet.command) {
                cmd("OKAY") -> return packet.arg0
                cmd("CLSE") -> throw IllegalStateException(packet.data.toString(Charsets.UTF_8).ifBlank { "\
u0631\u0641\u0636 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u062E\u062F\u0645\u0629 ADB" })
                cmd("WRTE") -> sendPacket(output, "OKAY", LOCAL_ID, packet.arg0, byteArrayOf())
            }
        }
    }

    private fun readServiceResponse(input: DataInputStream, output: DataOutputStream, maxBytes: Int):
ByteArray {
        val bytes = ByteArrayOutputStream()
        while (bytes.size() < maxBytes) {
            val packet = readPacket(input)
            when (packet.command) {
                cmd("WRTE") -> {
                    bytes.write(packet.data)
                    sendPacket(output, "OKAY", LOCAL_ID, packet.arg0, byteArrayOf())
                }
                cmd("CLSE") -> {
                    sendPacket(output, "CLSE", LOCAL_ID, packet.arg0, byteArrayOf())
                    return bytes.toByteArray()
                }
                cmd("OKAY") -> Unit
                else -> throw IllegalStateException("\u0631\u062F ADB \u063A\u064A\u0631 \u0645\u062A\u0648\
u0642\u0639")
            }
        }
        throw IllegalStateException("\u062A\u062C\u0627\u0648\u0632 \u062E\u0631\u062C \u0627\u0644\u0623\
u0645\u0631 \u0627\u0644\u062D\u062F \u0627\u0644\u0645\u0633\u0645\u0648\u062D")
    }

    private fun sendSyncRecord(input: DataInputStream, output: DataOutputStream, remoteId: Int, tag: String,
 data: ByteArray) {
        val record = ByteArrayOutputStream(data.size + 8)
        writeAscii(record, tag)
        writeLeInt(record, data.size)
        record.write(data)
        sendServiceData(input, output, remoteId, record.toByteArray())
    }

    private fun sendServiceData(input: DataInputStream, output: DataOutputStream, remoteId: Int, data:
```

```
ByteArray) {
        sendPacket(output, "WRTE", LOCAL_ID, remoteId, data)
        while (true) {
            val packet = readPacket(input)
            when (packet.command) {
                cmd("OKAY") -> return
                cmd("WRTE") -> sendPacket(output, "OKAY", LOCAL_ID, packet.arg0, byteArrayOf())
                cmd("CLSE") -> throw IllegalStateException("\u0627\u0646\u0642\u0637\u0639 \u0646\u0642\
u0644 \u0627\u0644\u0645\u0644\u0641 \u0628\u0648\u0627\u0633\u0637\u0629 \u0627\u0644\u0631\u0633\u064A\
u0641\u0631")
            }
        }
    }

    private fun getOrCreateKeyPair(): KeyPair {
        val store = KeyStore.getInstance("AndroidKeyStore").apply { load(null) }
        val publicKey = store.getCertificate(keyStoreAlias)?.publicKey
        val privateKey = store.getKey(keyStoreAlias, null) as? java.security.PrivateKey
        if (publicKey != null && privateKey != null) return KeyPair(publicKey, privateKey)

        val generator = KeyPairGenerator.getInstance(KeyProperties.KEY_ALGORITHM_RSA, "AndroidKeyStore")
        generator.initialize(
            KeyGenParameterSpec.Builder(keyStoreAlias, KeyProperties.PURPOSE_SIGN or KeyProperties.
PURPOSE_VERIFY)
                .setKeySize(2048)
                .setDigests(KeyProperties.DIGEST_SHA1)
                .setSignaturePaddings(KeyProperties.SIGNATURE_PADDING_RSA_PKCS1)
                .build()
        )
        return generator.generateKeyPair()
    }

    private fun adbPublicKey(key: RSAPublicKey): ByteArray {
        val modulus = key.modulus
        val r = BigInteger.ONE.shiftLeft(2048)
        val n0 = wordAtLittleEndian(modulus, 0).toLong() and 0xffff_ffffL
        val inverse = BigInteger.valueOf(n0).modInverse(BigInteger.ONE.shiftLeft(32))
        val n0inv = BigInteger.ONE.shiftLeft(32).subtract(inverse).and(BigInteger.ONE.shiftLeft(32).
subtract(BigInteger.ONE)).toLong().toInt()
        val encoded = ByteArrayOutputStream(524)
        writeLeInt(encoded, 64)
        writeLeInt(encoded, n0inv)
        writeLittleWords(encoded, modulus)
        writeLittleWords(encoded, r.multiply(r).mod(modulus))
        writeLeInt(encoded, key.publicExponent.toInt())
        return (Base64.encodeToString(encoded.toByteArray(), Base64.NO_WRAP) + " ReceiverControl@android\
u0000").toByteArray(Charsets.US_ASCII)
    }

    private fun writeLittleWords(out: ByteArrayOutputStream, value: BigInteger) {
        for (i in 0 until 64) writeLeInt(out, wordAtLittleEndian(value, i))
    }

    private fun wordAtLittleEndian(value: BigInteger, index: Int): Int {
        val raw = value.toByteArray()
        val littleIndex = index * 4
        var word = 0
        for (i in 0..3) {
            val fromEnd = littleIndex + i
            val byte = if (fromEnd < raw.size) raw[raw.lastIndex - fromEnd].toInt() and 0xff else 0
            word = word or (byte shl (i * 8))
        }
        return word
    }

    private data class Packet(val command: Int, val arg0: Int, val arg1: Int, val data: ByteArray)

    private fun readPacket(input: DataInputStream): Packet {
        val command = readLeInt(input)
        val arg0 = readLeInt(input)
        val arg1 = readLeInt(input)
        val length = readLeInt(input)
        val checksum = readLeInt(input)
        val magic = readLeInt(input)
        if (length !in 0..MAX_PACKET_SIZE || magic != (command xor -1)) throw IllegalStateException("\u062D\
u0632\u0645\u0629 ADB \u063A\u064A\u0631 \u0635\u0627\u0644\u062D\u0629")
        val data = ByteArray(length)
        input.readFully(data)
        if (data.fold(0L) { sum, b -> sum + (b.toInt() and 0xff) }.toInt() != checksum) {
            throw IllegalStateException("\u0641\u0634\u0644 \u0627\u0644\u062A\u062D\u0642\u0642 \u0645\
u0646 \u062D\u0632\u0645\u0629 ADB")
        }
        return Packet(command, arg0, arg1, data)
    }

    private fun sendPacket(output: DataOutputStream, name: String, arg0: Int, arg1: Int, data: ByteArray) {
        val command = cmd(name)
        writeLeInt(output, command)
        writeLeInt(output, arg0)
        writeLeInt(output, arg1)
        writeLeInt(output, data.size)
        writeLeInt(output, data.fold(0L) { sum, b -> sum + (b.toInt() and 0xff) }.toInt())
        writeLeInt(output, command xor -1)
        output.write(data)
        output.flush()
```

```
    }

    private fun cmd(value: String): Int {
        val b = value.toByteArray(Charsets.US_ASCII)
        require(b.size == 4)
        return (b[0].toInt() and 0xff) or ((b[1].toInt() and 0xff) shl 8) or
            ((b[2].toInt() and 0xff) shl 16) or ((b[3].toInt() and 0xff) shl 24)
    }

    private fun writeAscii(out: ByteArrayOutputStream, text: String) = out.write(text.toByteArray(Charsets.
US_ASCII))
    private fun writeLeInt(out: ByteArrayOutputStream, value: Int) {
        out.write(value and 0xff); out.write((value ushr 8) and 0xff)
        out.write((value ushr 16) and 0xff); out.write((value ushr 24) and 0xff)
    }
    private fun writeLeInt(out: DataOutputStream, value: Int) {
        out.writeByte(value and 0xff); out.writeByte((value ushr 8) and 0xff)
        out.writeByte((value ushr 16) and 0xff); out.writeByte((value ushr 24) and 0xff)
    }
    private fun readLeInt(input: DataInputStream): Int =
        input.readUnsignedByte() or (input.readUnsignedByte() shl 8) or
            (input.readUnsignedByte() shl 16) or (input.readUnsignedByte() shl 24)

    private fun ByteArray.toLeInt(): Int = (this[0].toInt() and 0xff) or
        ((this[1].toInt() and 0xff) shl 8) or ((this[2].toInt() and 0xff) shl 16) or
        ((this[3].toInt() and 0xff) shl 24)

    companion object {
        private const val ADB_VERSION = 0x01000001
        private const val AUTH_TYPE_TOKEN = 1
        private const val AUTH_TYPE_SIGNATURE = 2
        private const val AUTH_TYPE_RSAPUBLICKEY = 3
        private const val LOCAL_ID = 1
        private const val MAX_PACKET_SIZE = 4 * 1024 * 1024
    }
}
```

controller/src/main/java/com/monib/receivercontrol/controller/AndroidTvDiscovery.kt

```
package com.monib.receivercontrol.controller

import android.content.Context
import android.net.nsd.NsdManager
import android.net.nsd.NsdServiceInfo
import android.net.wifi.WifiManager
import android.os.Handler
import android.os.Looper
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import kotlinx.coroutines.suspendCancellableCoroutine
import java.util.concurrent.ConcurrentHashMap
import kotlin.coroutines.resume

/** Discovers Android TV Remote v2 endpoints. This does not require our receiver companion app. */
class AndroidTvDiscovery(context: Context) {
    private val appContext = context.applicationContext
    private val nsdManager = appContext.getSystemService(Context.NSD_SERVICE) as NsdManager
    private val wifiManager = appContext.getSystemService(Context.WIFI_SERVICE) as WifiManager

    suspend fun discover(timeoutMillis: Long = 5_000): List<ReceiverDevice> =
        withContext(Dispatchers.Main.immediate) {
            val found = ConcurrentHashMap<String, ReceiverDevice>()
            val lock = wifiManager.createMulticastLock("AndroidTvRemoteDiscovery")
            runCatching { lock.setReferenceCounted(false); lock.acquire() }

            suspendCancellableCoroutine { continuation ->
                val handler = Handler(Looper.getMainLooper())
                val pending = ArrayDeque<NsdServiceInfo>()
                val queued = mutableSetOf<String>()
                var resolving = false
                var stopped = false
                lateinit var listener: NsdManager.DiscoveryListener

                fun finish() {
                    if (stopped) return
                    stopped = true
                    handler.removeCallbacksAndMessages(null)
                    runCatching { nsdManager.stopServiceDiscovery(listener) }
                    if (lock.isHeld) runCatching { lock.release() }
                    if (continuation.isActive) continuation.resume(found.values.sortedBy { it.name.
lowercase() })
                }

                fun resolveNext() {
                    if (stopped || resolving || pending.isEmpty()) return
                    val service = pending.removeFirst()
                    resolving = true
                    try {
                        @Suppress("DEPRECATION")
                        nsdManager.resolveService(service, object : NsdManager.ResolveListener {
                            override fun onResolveFailed(serviceInfo: NsdServiceInfo, errorCode: Int) {
                                resolving = false
                                resolveNext()
                            }

                            override fun onServiceResolved(serviceInfo: NsdServiceInfo) {
                                val host = serviceInfo.host?.hostAddress.orEmpty()
                                if (host.isNotBlank() && serviceInfo.port == PAIRING_PORT) {
                                    val device = ReceiverDevice(
                                        name = serviceInfo.serviceName.ifBlank { "Android TV" },
                                        host = host,
                                        port = serviceInfo.port,
                                        protocol = ReceiverDevice.ANDROID_TV_REMOTE
                                    )
                                    found[host] = device
                                }
                                resolving = false
                                resolveNext()
                            }
                        })
                    } catch (_: Exception) {
                        resolving = false
                        resolveNext()
                    }
                }

                listener = object : NsdManager.DiscoveryListener {
                    override fun onDiscoveryStarted(serviceType: String) = Unit

                    override fun onServiceFound(serviceInfo: NsdServiceInfo) {
                        if (!serviceInfo.serviceType.orEmpty().trimEnd('.').equals(SERVICE_TYPE.trimEnd('.
'), ignoreCase = true)) return
                        val key = serviceInfo.serviceName + "|" + serviceInfo.serviceType
                        if (queued.add(key)) {
                            pending.addLast(serviceInfo)
                            resolveNext()
                        }
                    }
```

```
                    override fun onServiceLost(serviceInfo: NsdServiceInfo) = Unit
                    override fun onDiscoveryStopped(serviceType: String) = Unit

                    override fun onStartDiscoveryFailed(serviceType: String, errorCode: Int) {
                        finish()
                    }

                    override fun onStopDiscoveryFailed(serviceType: String, errorCode: Int) {
                        finish()
                    }
                }

                continuation.invokeOnCancellation {
                    handler.removeCallbacksAndMessages(null)
                    runCatching { nsdManager.stopServiceDiscovery(listener) }
                    if (lock.isHeld) runCatching { lock.release() }
                }

                try {
                    nsdManager.discoverServices(SERVICE_TYPE, NsdManager.PROTOCOL_DNS_SD, listener)
                    handler.postDelayed({ finish() }, timeoutMillis)
                } catch (_: Exception) {
                    finish()
                }
            }
        }

    private companion object {
        const val SERVICE_TYPE = "_androidtvremote2._tcp."
        const val PAIRING_PORT = 6467
    }
}
```

controller/src/main/java/com/monib/receivercontrol/controller/AndroidTvRemoteClient.kt

```
package com.monib.receivercontrol.controller

import android.content.Context
import com.out.androidtvremote.androidLib.AndroidRemoteContext
import com.out.androidtvremote.androidLib.pairing.PairingListener
import com.out.androidtvremote.androidLib.pairing.PairingSession
import com.out.androidtvremote.androidLib.remote.RemoteSession
import com.out.androidtvremote.androidLib.remote.Remotemessage
import java.io.File
import java.util.concurrent.Executors
import java.util.concurrent.atomic.AtomicBoolean

/** Native Android TV Remote v2 pairing/control. It does not need LocalReceiverService on the TV. */
class AndroidTvRemoteClient(context: Context) {
    private val appContext = context.applicationContext
    private val executor = Executors.newSingleThreadExecutor { task ->
        Thread(task, "AndroidTvRemote").apply { isDaemon = true }
    }

    @Volatile private var pairingSession: PairingSession? = null
    @Volatile private var remoteSession: RemoteSession? = null
    @Volatile private var activeHost: String? = null

    init {
        AndroidRemoteContext.getInstance().keyStoreFile = File(appContext.filesDir, KEYSTORE_NAME)
        AndroidRemoteContext.getInstance().clientName = "Android remote"
    }

    interface Listener {
        fun onPairingCodeRequested()
        fun onPairingAccepted() = Unit
        fun onConnected()
        fun onDisconnected() = Unit
        fun onError(message: String)
    }

    fun startPairing(host: String, listener: Listener) {
        closeSessions()
        val pairing = PairingSession()
        pairingSession = pairing
        executor.execute {
            try {
                pairing.pair(host, PAIRING_PORT, object : PairingListener {
                    override fun onSessionCreated() = Unit
                    override fun onPerformInputDeviceRole() = Unit
                    override fun onPerformOutputDeviceRole(gamma: ByteArray) = Unit
                    override fun onSecretRequested() = listener.onPairingCodeRequested()
                    override fun onSessionEnded() = Unit
                    override fun onError(message: String) = listener.onError(message)
                    override fun onPaired() = listener.onPairingAccepted()
                    override fun onLog(message: String) = Unit
                })
                runCatching { pairing.abort() }
                pairingSession = null
                connectRemote(host, listener)
            } catch (error: Exception) {
                pairingSession = null
                listener.onError(error.message ?: "Android TV pairing failed")
            }
        }
    }

    fun providePairingCode(code: String) {
        val pending = pairingSession ?: throw IllegalStateException("\u0627\u0628\u062F\u0623 \u0627\u0644\
u0627\u0642\u062A\u0631\u0627\u0646 \u0623\u0648\u0644\u0627\u064B")
        require(code.matches(Regex("[0-9A-Fa-f]{6}"))) { "\u0631\u0645\u0632 Android TV \u064A\u062C\u0628 \
u0623\u0646 \u064A\u062A\u0643\u0648\u0651\u0646 \u0645\u0646 6 \u0623\u062D\u0631\u0641 \u0623\u0648 \
u0623\u0631\u0642\u0627\u0645 \u0633\u062F\u0627\u0633\u064A\u0629 \u0639\u0634\u0631\u064A\u0629." }
        try {
            pending.provideSecret(code.uppercase())
        } catch (error: Exception) {
            pairingSession = null
            runCatching { pending.abort() }
            throw IllegalArgumentException("\u0631\u0641\u0636 \u0627\u0644\u062A\u0644\u0641\u0627\u0632 \
u0631\u0645\u0632 \u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646. \u0623\u0639\u062F \u0628\u062F\u0621 \
u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646 \u0648\u0627\u0643\u062A\u0628 \u0627\u0644\u0631\u0645\
u0632 \u0627\u0644\u062C\u062F\u064A\u062F \u0627\u0644\u0638\u0627\u0647\u0631 \u0639\u0644\u0649 \u0627\
u0644\u0634\u0627\u0634\u0629.", error)
        }
    }

    fun reconnect(host: String, listener: Listener) {
        closeSessions()
        executor.execute { connectRemote(host, listener) }
    }

    fun sendKey(host: String, key: String) {
        val session = remoteSession
        if (activeHost != host || session == null || !session.isSocketAlive()) {
```

```
            throw IllegalStateException("\u0627\u062A\u0635\u0627\u0644 Android TV \u063A\u064A\u0631 \
u062C\u0627\u0647\u0632. \u0623\u0639\u062F \u0627\u062E\u062A\u064A\u0627\u0631 \u0627\u0644\u062C\u0647\
u0627\u0632 \u0623\u0648 \u0623\u0639\u062F \u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646.")
        }
        session.sendCommand(toRemoteKey(key), Remotemessage.RemoteDirection.SHORT)
        if (!session.isSocketAlive()) throw IllegalStateException("\u0627\u0646\u0642\u0637\u0639 \u0627\
u062A\u0635\u0627\u0644 Android TV\u061B \u0623\u0639\u062F \u0627\u0644\u0627\u062A\u0635\u0627\u0644 \
u062B\u0645 \u062C\u0631\u0651\u0628 \u0645\u062C\u062F\u062F\u0627\u064B.")
    }

    fun close() {
        closeSessions()
        executor.shutdownNow()
    }

    private fun connectRemote(host: String, listener: Listener) {
        val connectedReported = AtomicBoolean(false)
        val failedReported = AtomicBoolean(false)
        val session = RemoteSession(host, REMOTE_PORT, object : RemoteSession.RemoteSessionListener {
            override fun onConnected() {
                activeHost = host
                if (connectedReported.compareAndSet(false, true)) listener.onConnected()
            }

            override fun onSslError() {
                failOnce("\u062A\u0639\u0630\u0651\u0631 \u0625\u0646\u0634\u0627\u0621 TLS \u0645\u0639 \
u0627\u0644\u062A\u0644\u0641\u0627\u0632. \u0623\u0639\u062F \u0627\u0644\u0627\u0642\u062A\u0631\u0627\
u0646 \u0645\u0646 \u0634\u0627\u0634\u0629 Android TV.")
            }

            override fun onDisconnected() {
                activeHost = null
                listener.onDisconnected()
            }

            override fun onImeShow(text: String, fieldCounter: Int) = Unit

            override fun onError(message: String) {
                failOnce(message.ifBlank { "\u0631\u0641\u0636 \u0627\u0644\u062A\u0644\u0641\u0627\u0632 \
u0627\u062A\u0635\u0627\u0644 \u0627\u0644\u0631\u064A\u0645\u0648\u062A." })
            }

            private fun failOnce(message: String) {
                if (failedReported.compareAndSet(false, true)) listener.onError(message)
            }
        })
        remoteSession = session
        activeHost = null
        try {
            session.connect()
            if (session.isSocketAlive()) {
                activeHost = host
                if (connectedReported.compareAndSet(false, true)) listener.onConnected()
            } else if (failedReported.compareAndSet(false, true)) {
                listener.onError("\u062A\u0639\u0630\u0651\u0631 \u0627\u0644\u0648\u0635\u0648\u0644 \
u0625\u0644\u0649 \u062E\u062F\u0645\u0629 Android TV Remote \u0639\u0644\u0649 \u0627\u0644\u0645\u0646\
u0641\u0630 6466. \u062A\u0623\u0643\u062F \u0623\u0646 \u0627\u0644\u062A\u0644\u0641\u0627\u0632 \u064A\
u062F\u0639\u0645\u0647\u0627 \u0648\u0623\u0646 \u0627\u0644\u0647\u0627\u062A\u0641 \u0639\u0644\u0649 \
u0634\u0628\u0643\u0629 Wi-Fi \u0646\u0641\u0633\u0647\u0627.")
            }
        } catch (error: Exception) {
            if (failedReported.compareAndSet(false, true)) listener.onError(error.message ?: "Android TV
remote connection failed")
        }
    }

    private fun closeSessions() {
        activeHost = null
        runCatching { pairingSession?.abort() }
        pairingSession = null
        runCatching { remoteSession?.abort() }
        remoteSession = null
    }

    private fun toRemoteKey(key: String): Remotemessage.RemoteKeyCode {
        val name = when (key.uppercase()) {
            "UP" -> "KEYCODE_DPAD_UP"
            "DOWN" -> "KEYCODE_DPAD_DOWN"
            "LEFT" -> "KEYCODE_DPAD_LEFT"
            "RIGHT" -> "KEYCODE_DPAD_RIGHT"
            "ENTER", "OK", "SELECT" -> "KEYCODE_DPAD_CENTER"
            "RECENTS" -> "KEYCODE_APP_SWITCH"
            "MUTE" -> "KEYCODE_MUTE"
            "CHANNEL_UP" -> "KEYCODE_CHANNEL_UP"
            "CHANNEL_DOWN" -> "KEYCODE_CHANNEL_DOWN"
            else -> "KEYCODE_${key.uppercase()}"
        }
        return runCatching { Remotemessage.RemoteKeyCode.valueOf(name) }
            .getOrElse { throw IllegalArgumentException("\u0632\u0631 \u063A\u064A\u0631 \u0645\u062F\u0639\
u0648\u0645 \u0641\u064A Android TV Remote: $key") }
    }

    private companion object {
        const val KEYSTORE_NAME = "androidtv-remote.keystore"
        const val PAIRING_PORT = 6467
```

```
        const val REMOTE_PORT = 6466
    }
}
```

controller/src/main/java/com/monib/receivercontrol/controller/ControllerUi.kt

```
package com.monib.receivercontrol.controller

import android.Manifest
import android.content.Context
import android.content.pm.PackageManager
import android.graphics.ImageFormat
import android.graphics.Rect
import android.util.Size
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.camera.core.CameraSelector
import androidx.camera.core.ImageAnalysis
import androidx.camera.core.ImageProxy
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.camera.view.PreviewView
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.spring
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.gestures.detectDragGestures
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.aspectRatio
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.Divider
import androidx.compose.material3.ElevatedButton
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.AssistChip
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberUpdatedState
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.scale
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.compose.ui.window.Dialog
import androidx.compose.ui.window.DialogProperties
import androidx.core.content.ContextCompat
import androidx.lifecycle.LifecycleOwner
import androidx.lifecycle.compose.LocalLifecycleOwner
import androidx.compose.ui.viewinterop.AndroidView
import com.google.zxing.BinaryBitmap
import com.google.zxing.DecodeHintType
```

```
import com.google.zxing.PlanarYUVLuminanceSource
import com.google.zxing.qrcode.QRCodeReader
import com.google.zxing.common.HybridBinarizer
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import com.monib.receivercontrol.controller.widgets.ReferenceDpadView
import java.nio.ByteBuffer
import java.util.EnumMap
import java.util.concurrent.ExecutorService
import java.util.concurrent.Executors
import java.util.concurrent.atomic.AtomicBoolean
import java.util.concurrent.atomic.AtomicLong

fun loc(language: String, arabic: String, english: String): String = if (language == "ar") arabic else
english

private val Purple = Color(0xFF8064DB)
private val DarkScheme = darkColorScheme(
    primary = Color(0xFFB7A3FF),
    onPrimary = Color(0xFF211345),
    secondary = Color(0xFFAC9BCB),
    background = Color(0xFF111117),
    surface = Color(0xFF1D1C25),
    surfaceVariant = Color(0xFF2A2935),
    onSurface = Color(0xFFF0EFF7),
    onSurfaceVariant = Color(0xFFC5C1D2),
    outline = Color(0xFF514E5D)
)
private val WhiteScheme = lightColorScheme(
    primary = Purple,
    onPrimary = Color.White,
    secondary = Color(0xFF65558F),
    background = Color.White,
    surface = Color.White,
    surfaceVariant = Color(0xFFF5F3FA),
    onSurface = Color(0xFF17151C),
    onSurfaceVariant = Color(0xFF5D5966),
    outline = Color(0xFFE3E0EA)
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ReceiverControlApp(
    state: ControllerState,
    onTab: (String) -> Unit,
    onDiscover: () -> Unit,
    onPairPrompt: (ReceiverDevice) -> Unit,
    onPair: (String, String) -> Unit,
    onPairingProtocol: (String) -> Unit,
    onStartAndroidTvPairing: (String) -> Unit,
    onQrScan: (String) -> Unit,
    onScannerVisibility: (Boolean) -> Unit,
    onManualHost: (String) -> Unit,
    onPairCodeInput: (String) -> Unit,
    onSendKey: (String) -> Unit,
    onAdbHost: (String) -> Unit,
    onAdbPort: (String) -> Unit,
    onAdbConnect: () -> Unit,
    onAdbCommand: (String) -> Unit,
    onRunAdbCommand: (String) -> Unit,
    onEditFavorite: (Int, String, String) -> Unit,
    onRunFavorite: (Int) -> Unit,
    onRefreshApps: () -> Unit,
    onLaunchApp: (ReceiverApp) -> Unit,
    onUninstallApp: (ReceiverApp) -> Unit,
    onSendFile: (android.net.Uri) -> Unit,
    onRefreshScreen: () -> Unit,
    onScreenLiveChange: (Boolean) -> Unit,
    onPip: () -> Unit,
    onWebsite: () -> Unit,
    onDismissWebsiteBanner: () -> Unit,
    onTheme: (String) -> Unit,
    onLanguage: (String) -> Unit,
    onCustomJson: () -> Unit,
    onCustomJsonText: (String) -> Unit,
    onSelectProfile: (SavedReceiver) -> Unit,
    onDeleteProfile: (SavedReceiver) -> Unit,
    onSaveMacro: (String?, String, List<String>) -> Unit,
    onDeleteMacro: (String) -> Unit,
    onRunMacro: (RemoteMacro) -> Unit,
    onCastUrl: (String) -> Unit
) {
    val isArabic = state.language == "ar"
    val context = LocalContext.current
    val cameraPermission = rememberLauncherForActivityResult(ActivityResultContracts.RequestPermission()) {
granted ->
        if (granted) onScannerVisibility(true)
    }
    val openFile = rememberLauncherForActivityResult(ActivityResultContracts.OpenDocument()) { uri -> uri?.
let(onSendFile) }
    var uninstallTarget by remember { mutableStateOf<ReceiverApp?>(null) }
    val latestSuccess by rememberUpdatedState(state.success)
    LaunchedEffect(state.screenLive, state.paired?.host, state.tab, state.pictureInPicture) {
        if (!state.screenLive) return@LaunchedEffect
```

```
        if (state.tab != "media" || state.paired == null) {
            onScreenLiveChange(false)
            return@LaunchedEffect
        }
        while (true) {
            onRefreshScreen()
            delay(2500)
            if (latestSuccess == false) {
                onScreenLiveChange(false)
                break
            }
        }
    }

    MaterialTheme(colorScheme = if (state.theme == "white") WhiteScheme else DarkScheme) {
        CompositionDirection(isArabic) {
            if (state.pictureInPicture && state.screenPreview != null) {
                Box(Modifier.fillMaxSize().background(Color.Black), contentAlignment = Alignment.Center) {
                    Image(
                        bitmap = state.screenPreview.asImageBitmap(),
                        contentDescription = loc(state.language, "\u0634\u0627\u0634\u0629 \u0627\u0644\
u0631\u0633\u064A\u0641\u0631", "Receiver screen"),
                        modifier = Modifier.fillMaxSize()
                    )
                }
            } else {
            val selectedTab = when (state.tab) {
                "macros" -> "remote"
                "screen" -> "media"
                else -> state.tab
            }
            Scaffold(
                containerColor = MaterialTheme.colorScheme.background,
                topBar = {
                    TopAppBar(
                        title = { Text("Android remote", style = MaterialTheme.typography.titleLarge) },
                        actions = {
                            TextButton(onClick = { onTab("devices") }) {
                                Text(state.paired?.name ?: loc(state.language, "\u0631\u0628\u0637",
"Connect"), maxLines = 1)
                            }
                        },
                        colors = TopAppBarDefaults.topAppBarColors(containerColor = MaterialTheme.
colorScheme.background)
                    )
                },
                bottomBar = {
                    NavigationBar(containerColor = MaterialTheme.colorScheme.surface) {
                        listOf(
                            Triple("remote", "\u2316", loc(state.language, "\u0631\u064A\u0645\u0648\u062A",
 "Remote")),
                            Triple("devices", "\u25A3", loc(state.language, "\u0623\u062C\u0647\u0632\
u0629", "Devices")),
                            Triple("apps", "\u25A6", loc(state.language, "\u062A\u0637\u0628\u064A\u0642\
u0627\u062A", "Apps")),
                            Triple("media", "\u266B", loc(state.language, "\u0648\u0633\u0627\u0626\u0637",
"Media")),
                            Triple("settings", "\u2699", loc(state.language, "\u0625\u0639\u062F\u0627\
u062F\u0627\u062A", "Settings"))
                        ).forEach { (tab, glyph, label) ->
                            NavigationBarItem(
                                selected = selectedTab == tab,
                                onClick = { onTab(tab) },
                                icon = { Text(glyph, style = MaterialTheme.typography.titleMedium) },
                                label = { Text(label, maxLines = 1) }
                            )
                        }
                    }
                }
            ) { padding ->
                Column(Modifier.fillMaxSize().padding(padding)) {
                        if (!state.websiteBannerDismissed) {
                            Card(
                                onClick = onWebsite,
                                modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 4.
dp),
                                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.
primaryContainer),
                                shape = RoundedCornerShape(18.dp)
                            ) {
                                Row(Modifier.fillMaxWidth().padding(start = 14.dp, end = 6.dp, top = 6.dp,
bottom = 6.dp), verticalAlignment = Alignment.CenterVertically) {
                                    Column(Modifier.weight(1f)) {
                                        Text(loc(state.language, "\u0645\u0648\u0642\u0639 \u0627\u0644\
u062A\u0637\u0628\u064A\u0642", "App website"), style = MaterialTheme.typography.titleSmall)
                                        Text(loc(state.language, "\u0627\u0644\u0641\u0643\u0631\u0629 \
u0648\u0637\u0631\u064A\u0642\u0629 \u0627\u0644\u062A\u062D\u0645\u064A\u0644", "About and downloads"),
style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                                    }
                                    TextButton(onClick = onDismissWebsiteBanner) { Text("\u00D7", style =
MaterialTheme.typography.titleLarge) }
                                }
                            }
                        }
                        Card(
```

```
                            Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 4.dp),
                            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.
surfaceVariant),
                            shape = RoundedCornerShape(18.dp)
                        ) {
                            Row(Modifier.padding(horizontal = 14.dp, vertical = 10.dp), verticalAlignment =
Alignment.CenterVertically) {
                                val color = when (state.success) { true -> Color(0xFF55C98B); false ->
Color(0xFFE47B7B); null -> MaterialTheme.colorScheme.primary }
                                Surface(Modifier.size(9.dp), color = color, shape = CircleShape) {}
                                Spacer(Modifier.width(10.dp))
                                Text(state.status, style = MaterialTheme.typography.bodySmall, color =
MaterialTheme.colorScheme.onSurface, modifier = Modifier.weight(1f))
                            }
                        }
                        Box(Modifier.weight(1f).fillMaxWidth()) {
                            when (state.tab) {
                                "devices" -> DevicesPage(state, onDiscover, onPairPrompt, onManualHost,
onPairCodeInput, onPair, onPairingProtocol, onStartAndroidTvPairing, onSelectProfile, onDeleteProfile, {
                                    if (ContextCompat.checkSelfPermission(context, Manifest.permission.
CAMERA) == PackageManager.PERMISSION_GRANTED) onScannerVisibility(true)
                                    else cameraPermission.launch(Manifest.permission.CAMERA)
                                })
                                "macros" -> MacrosPage(state, onSaveMacro, onDeleteMacro, onRunMacro)
                                "apps" -> AppsPage(state, onRefreshApps, onLaunchApp, { uninstallTarget =
it })
                                "media" -> MediaPage(state, onRefreshScreen, onScreenLiveChange, onPip,
onCastUrl) { openFile.launch(arrayOf("*/*")) }
                                "screen" -> ScreenPage(state, onRefreshScreen, onPip)
                                "settings" -> SettingsPage(state, onTheme, onLanguage, onAdbHost, onAdbPort,
 onAdbConnect, onAdbCommand, onRunAdbCommand, onEditFavorite, onRunFavorite, onCustomJson, onCustomJsonText)
                                else -> RemotePage(state, onSendKey, onTab, onLaunchApp)
                            }
                        }
                    }
                }
            if (state.showQrScanner) QrScannerDialog(state.language, onDismiss = { onScannerVisibility(
false) }, onScanned = onQrScan)
            uninstallTarget?.let { target ->
                AlertDialog(
                    onDismissRequest = { uninstallTarget = null },
                    title = { Text(loc(state.language, "\u0625\u0644\u063A\u0627\u0621 \u062A\u062B\u0628\
u064A\u062A \u0627\u0644\u062A\u0637\u0628\u064A\u0642\u061F", "Uninstall app?")) },
                    text = { Text("${loc(state.language, "\u0633\u064A\u064F\u0637\u0644\u0628 \u062A\u0623\
u0643\u064A\u062F \u0627\u0644\u062D\u0630\u0641 \u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\
u0631:", "The receiver will ask you to confirm removal:")}\n${target.packageName}") },
                    confirmButton = { TextButton(onClick = { onUninstallApp(target); uninstallTarget = null
}) { Text(loc(state.language, "\u062D\u0630\u0641", "Uninstall")) } },
                    dismissButton = { TextButton(onClick = { uninstallTarget = null }) { Text(loc(state.
language, "\u0625\u0644\u063A\u0627\u0621", "Cancel")) } }
                )
            }
            }
        }
    }
}

@Composable
private fun CompositionDirection(isArabic: Boolean, content: @Composable () -> Unit) {
    androidx.compose.runtime.CompositionLocalProvider(
        androidx.compose.ui.platform.LocalLayoutDirection provides if (isArabic) androidx.compose.ui.unit.
LayoutDirection.Rtl else androidx.compose.ui.unit.LayoutDirection.Ltr,
        content = content
    )
}

@Composable
private fun RemotePage(
    state: ControllerState,
    onSendKey: (String) -> Unit,
    onTab: (String) -> Unit,
    onLaunchApp: (ReceiverApp) -> Unit
) {
    val ar = state.language
    var mode by rememberSaveable { mutableStateOf("touchpad") }
    Column(
        Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(horizontal = 18.dp, vertical =
10.dp),
        verticalArrangement = Arrangement.spacedBy(14.dp)
    ) {
        Row(
            Modifier.fillMaxWidth().horizontalScroll(rememberScrollState()),
            horizontalArrangement = Arrangement.spacedBy(8.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            if (state.apps.isEmpty()) {
                AssistChip(onClick = { onTab("apps") }, label = { Text(loc(ar, "\u062A\u0637\u0628\u064A\
u0642\u0627\u062A \u0633\u0631\u064A\u0639\u0629", "Quick apps")) })
            } else {
                state.apps.take(8).forEach { app ->
                    AssistChip(onClick = { onLaunchApp(app) }, label = { Text(app.name, maxLines = 1) })
                }
            }
        }
```

```
        Card(
            Modifier.fillMaxWidth().height(310.dp),
            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant),
            shape = RoundedCornerShape(30.dp),
            border = androidx.compose.foundation.BorderStroke(1.dp, MaterialTheme.colorScheme.outline.copy(
alpha = 0.35f))
        ) {
            Column(Modifier.fillMaxSize().padding(15.dp), verticalArrangement = Arrangement.spacedBy(8.dp))
{
                Row(
                    Modifier.fillMaxWidth().horizontalScroll(rememberScrollState()),
                    horizontalArrangement = Arrangement.spacedBy(7.dp)
                ) {
                    FilterChip(selected = mode == "touchpad", onClick = { mode = "touchpad" }, label = {
Text(loc(ar, "\u0644\u0648\u062D\u0629 \u0644\u0645\u0633", "Touchpad")) })
                    FilterChip(selected = mode == "dpad", onClick = { mode = "dpad" }, label = { Text("D-
Pad") })
                    FilterChip(selected = mode == "numbers", onClick = { mode = "numbers" }, label = { Text(
loc(ar, "\u0623\u0631\u0642\u0627\u0645", "Number pad")) })
                }
                Box(Modifier.fillMaxSize().weight(1f), contentAlignment = Alignment.Center) {
                    when (mode) {
                        "dpad" -> ReferenceDpad(onSendKey)
                        "numbers" -> NumberPadContent(ar, onSendKey)
                        else -> TouchPadContent(ar, onSendKey)
                    }
                }
            }
        }

        RemoteControlCluster(ar, onSendKey) { mode = "numbers" }

        Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(9.dp)) {
            OutlinedButton(onClick = { onTab("macros") }, modifier = Modifier.weight(1f)) {
                Text(loc(ar, "\u0627\u062E\u062A\u0635\u0627\u0631\u0627\u062A", "Shortcuts"))
            }
            OutlinedButton(onClick = { onTab("screen") }, modifier = Modifier.weight(1f)) {
                Text(loc(ar, "\u0645\u0639\u0627\u064A\u0646\u0629 \u0627\u0644\u0634\u0627\u0634\u0629",
"Screen preview"))
            }
        }
    }
}

@Composable
private fun RemoteControlCluster(language: String, onSendKey: (String) -> Unit, onShowNumbers: () -> Unit) {
    Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(7.dp), verticalAlignment =
Alignment.CenterVertically) {
        VerticalRemotePill(loc(language, "\u0635\u0648\u062A", "VOL"), "+", "VOLUME_UP", "\u2212",
"VOLUME_DOWN", onSendKey)
        Column(
            Modifier.weight(1f),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(10.dp)
        ) {
            Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                RemoteActionKey("\u25A6") { onSendKey("RECENTS") }
                RemoteKey("\u2302", "HOME", onSendKey, compact = true, modifier = Modifier.size(52.dp))
                RemoteActionKey("123", onShowNumbers)
            }
            Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                RemoteKey("\u25D6", "MUTE", onSendKey, compact = true, modifier = Modifier.size(52.dp))
                RemoteKey("\u23FB", "POWER", onSendKey, compact = true, tint = Color(0xFFD95362), modifier
= Modifier.size(56.dp))
                RemoteKey("\u21B6", "BACK", onSendKey, compact = true, modifier = Modifier.size(52.dp))
            }
        }
        VerticalRemotePill(loc(language, "\u0642\u0646\u0627\u0629", "CH"), "\u25B2", "CHANNEL_UP", "\
u25BC", "CHANNEL_DOWN", onSendKey)
    }
}

@Composable
private fun VerticalRemotePill(
    title: String,
    topLabel: String,
    topKey: String,
    bottomLabel: String,
    bottomKey: String,
    onSendKey: (String) -> Unit
) {
    Card(
        Modifier.width(58.dp).height(174.dp),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
        shape = RoundedCornerShape(28.dp),
        border = androidx.compose.foundation.BorderStroke(1.dp, MaterialTheme.colorScheme.outline.copy(
alpha = 0.3f))
    ) {
        Column(Modifier.fillMaxSize().padding(vertical = 7.dp), horizontalAlignment = Alignment.
CenterHorizontally) {
            Text(title, style = MaterialTheme.typography.labelSmall, color = MaterialTheme.colorScheme.
onSurfaceVariant)
            Spacer(Modifier.weight(1f))
            RemoteActionKey(topLabel) { onSendKey(topKey) }
            Spacer(Modifier.weight(1f))
```

```
            RemoteActionKey(bottomLabel) { onSendKey(bottomKey) }
            Spacer(Modifier.weight(1f))
        }
    }
}

@Composable
private fun RemoteActionKey(label: String, onClick: () -> Unit) {
    var pressed by remember { mutableStateOf(false) }
    val scale by animateFloatAsState(if (pressed) 0.92f else 1f, spring(dampingRatio = 0.52f, stiffness =
520f), label = "remote-action")
    val scope = rememberCoroutineScope()
    ElevatedButton(
        onClick = { pressed = true; onClick(); scope.launch { delay(110); pressed = false } },
        modifier = Modifier.size(52.dp).scale(scale),
        shape = CircleShape,
        contentPadding = androidx.compose.foundation.layout.PaddingValues(2.dp),
        colors = androidx.compose.material3.ButtonDefaults.elevatedButtonColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
    ) { Text(label, style = MaterialTheme.typography.labelMedium, maxLines = 1) }
}

@Composable
private fun ReferenceDpad(onSendKey: (String) -> Unit) {
    val colors = MaterialTheme.colorScheme
    val listener = remember(onSendKey) {
        object : ReferenceDpadView.DpadListener {
            override fun onUp() = onSendKey("UP")
            override fun onDown() = onSendKey("DOWN")
            override fun onLeft() = onSendKey("LEFT")
            override fun onRight() = onSendKey("RIGHT")
            override fun onCenter() = onSendKey("ENTER")
        }
    }
    AndroidView(
        modifier = Modifier.fillMaxSize(),
        factory = { context ->
            ReferenceDpadView(context).apply {
                this.listener = listener
                setPalette(
                    colors.surface.toArgb(), colors.surfaceVariant.toArgb(), colors.onSurface.toArgb(),
                    colors.primary.toArgb(), colors.primaryContainer.toArgb(), colors.onPrimaryContainer.
toArgb(),
                    colors.outline.toArgb()
                )
            }
        },
        update = { view ->
            view.listener = listener
            view.setPalette(
                colors.surface.toArgb(), colors.surfaceVariant.toArgb(), colors.onSurface.toArgb(),
                colors.primary.toArgb(), colors.primaryContainer.toArgb(), colors.onPrimaryContainer.toArgb(
),
                colors.outline.toArgb()
            )
        }
    )
}

@Composable
private fun NumberPadContent(language: String, onSendKey: (String) -> Unit) {
    Column(Modifier.fillMaxSize(), verticalArrangement = Arrangement.spacedBy(5.dp)) {
        listOf(listOf("1", "2", "3"), listOf("4", "5", "6"), listOf("7", "8", "9"), listOf("\u232B", "0",
"OK")).forEach { row ->
            Row(Modifier.weight(1f).fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                row.forEach { label ->
                    val key = when (label) {
                        "\u232B" -> "DELETE"
                        "OK" -> "ENTER"
                        else -> "DIGIT_$label"
                    }
                    RemoteKey(label, key, onSendKey, large = true, modifier = Modifier.weight(1f).
fillMaxHeight())
                }
            }
        }
    }
}

@Composable
private fun TouchPadContent(language: String, onSendKey: (String) -> Unit) {
    var dragDistance by remember { mutableStateOf(Offset.Zero) }
    Box(
        Modifier
            .fillMaxSize()
            .background(MaterialTheme.colorScheme.surface, RoundedCornerShape(23.dp))
            .pointerInput(onSendKey) {
                var total = Offset.Zero
                detectDragGestures(
                    onDragStart = { total = Offset.Zero },
                    onDragEnd = {
                        val dx = total.x; val dy = total.y
                        if (kotlin.math.abs(dx) < 36f && kotlin.math.abs(dy) < 36f) onSendKey("ENTER")
                        else if (kotlin.math.abs(dx) > kotlin.math.abs(dy)) onSendKey(if (dx > 0) "RIGHT"
else "LEFT")
```

```
                        else onSendKey(if (dy > 0) "DOWN" else "UP")
                        dragDistance = Offset.Zero
                    },
                    onDragCancel = { dragDistance = Offset.Zero },
                    onDrag = { change, amount -> change.consume(); total += amount; dragDistance = total }
                )
            }
            .pointerInput(onSendKey) { detectTapGestures(onTap = { onSendKey("ENTER") }) },
        contentAlignment = Alignment.Center
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally, verticalArrangement = Arrangement.
spacedBy(7.dp)) {
            Text("\u2316", style = MaterialTheme.typography.displaySmall, color = MaterialTheme.colorScheme.
primary)
            Text(
                if (dragDistance == Offset.Zero) loc(language, "\u0627\u0633\u062D\u0628 \u0644\u0644\u062A\
u0646\u0642\u0644 \u00B7 \u0627\u0636\u063A\u0637 \u0644\u0644\u0627\u062E\u062A\u064A\u0627\u0631", "Swipe
to navigate \u00B7 tap to select") else loc(language, "\u0625\u0631\u0633\u0627\u0644 \u062D\u0631\u0643\
u0629 \u0627\u0644\u0644\u0645\u0633", "Sending touch movement"),
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                style = MaterialTheme.typography.labelMedium
            )
        }
    }
}

@Composable
private fun RemoteKey(
    label: String,
    key: String,
    onSend: (String) -> Unit,
    compact: Boolean = false,
    selected: Boolean = false,
    large: Boolean = false,
    tint: Color? = null,
    modifier: Modifier? = null
) {
    var pressed by remember { mutableStateOf(false) }
    val scale by animateFloatAsState(if (pressed) 0.92f else 1f, spring(dampingRatio = 0.52f, stiffness =
520f), label = "remote-button")
    val scope = rememberCoroutineScope()
    val color = tint ?: if (selected) MaterialTheme.colorScheme.primary else MaterialTheme.colorScheme.
surfaceVariant
    ElevatedButton(
        onClick = { pressed = true; onSend(key); scope.launch { delay(110); pressed = false } },
        modifier = Modifier.scale(scale).then(modifier ?: if (compact) Modifier.size(58.dp) else Modifier.
width(95.dp).height(62.dp)),
        shape = if (compact) CircleShape else RoundedCornerShape(22.dp),
        colors = androidx.compose.material3.ButtonDefaults.elevatedButtonColors(
            containerColor = color,
            contentColor = if (selected || tint != null) Color.White else MaterialTheme.colorScheme.
onSurface
        )
    ) {
        Text(label, style = when { large -> MaterialTheme.typography.titleMedium; compact -> MaterialTheme.
typography.labelLarge; else -> MaterialTheme.typography.labelMedium }, maxLines = 1)
    }
}

@Composable
private fun DevicesPage(
    state: ControllerState,
    onDiscover: () -> Unit,
    onPairPrompt: (ReceiverDevice) -> Unit,
    onManualHost: (String) -> Unit,
    onCode: (String) -> Unit,
    onPair: (String, String) -> Unit,
    onPairingProtocol: (String) -> Unit,
    onStartAndroidTvPairing: (String) -> Unit,
    onSelectProfile: (SavedReceiver) -> Unit,
    onDeleteProfile: (SavedReceiver) -> Unit,
    onScan: () -> Unit
) {
    val ar = state.language
    Column(Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(horizontal = 16.dp,
vertical = 4.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(10.dp)) {
            Button(onClick = onDiscover, modifier = Modifier.weight(1f), enabled = !state.discovering) {
                if (state.discovering) CircularProgressIndicator(Modifier.size(18.dp), strokeWidth = 2.dp)
else Text("\u2315  ${loc(ar, "\u0628\u062D\u062B Wi-Fi", "Search Wi-Fi")}")
            }
            OutlinedButton(onClick = onScan, modifier = Modifier.weight(1f)) { Text("\u25A6  ${loc(ar, "\
u0645\u0633\u062D QR", "Scan QR")}") }
        }
        Text(loc(ar, "\u064A\u0628\u062D\u062B \u0639\u0646 \u0623\u062C\u0647\u0632\u0629 Android TV \
u0627\u0644\u0645\u062A\u0648\u0627\u0641\u0642\u0629 \u0645\u0628\u0627\u0634\u0631\u0629\u060C \u0648\
u0639\u0646 \u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0627\u0644\u0645\
u0631\u0627\u0641\u0642 \u0644\u0644\u0645\u064A\u0632\u0627\u062A \u0627\u0644\u0625\u0636\u0627\u0641\
u064A\u0629.", "Finds compatible Android TVs directly and the companion receiver app for advanced features.
"), color = MaterialTheme.colorScheme.onSurfaceVariant)
        state.paired?.let {
            Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.
surfaceVariant)) {
                Column(Modifier.fillMaxWidth().padding(14.dp)) {
                    Text(loc(ar, "\u0645\u0642\u062A\u0631\u0646", "Paired"), color = Color(0xFF55C98B))
```

```
                    Text("${it.name}  \u00B7  ${it.host}")
                }
            }
        }
        if (state.profiles.isNotEmpty()) {
            Text(loc(ar, "\u0645\u0644\u0641\u0627\u062A \u0627\u0644\u0631\u0633\u064A\u0641\u0631\u0627\
u062A \u0627\u0644\u0645\u062D\u0641\u0648\u0638\u0629", "Saved receiver profiles"), style = MaterialTheme.
typography.titleSmall)
            state.profiles.forEach { profile ->
                Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
                    Row(Modifier.fillMaxWidth().padding(10.dp), verticalAlignment = Alignment.
CenterVertically) {
                        Column(Modifier.weight(1f)) {
                            Text(profile.name, style = MaterialTheme.typography.titleSmall)
                            Text(profile.host, color = MaterialTheme.colorScheme.onSurfaceVariant, style =
MaterialTheme.typography.bodySmall)
                        }
                        TextButton(onClick = { onSelectProfile(profile) }) { Text(loc(ar, "\u062A\u062D\
u062F\u064A\u062F", "Select")) }
                        TextButton(onClick = { onDeleteProfile(profile) }) { Text("\u00D7") }
                    }
                }
            }
        }
        state.devices.forEach { device ->
            Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
                Row(Modifier.fillMaxWidth().padding(14.dp), verticalAlignment = Alignment.CenterVertically,
horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Column(Modifier.weight(1f)) {
                        Text(device.name, style = MaterialTheme.typography.titleSmall)
                        Text(device.host, color = MaterialTheme.colorScheme.onSurfaceVariant)
                        Text(
                            if (device.protocol == ReceiverDevice.ANDROID_TV_REMOTE) loc(ar, "Android TV \
u0645\u0628\u0627\u0634\u0631 \u00B7 \u0644\u0627 \u064A\u062D\u062A\u0627\u062C \u0627\u0644\u062A\u0637\
u0628\u064A\u0642 \u0627\u0644\u0645\u0631\u0627\u0641\u0642", "Android TV direct \u00B7 no companion app")
                            else loc(ar, "\u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\
u0641\u0631 \u00B7 \u0645\u064A\u0632\u0627\u062A \u0627\u0644\u0645\u0644\u0641\u0627\u062A \u0648\u0627\
u0644\u062A\u0637\u0628\u064A\u0642\u0627\u062A", "Receiver app \u00B7 files and apps"),
                            color = MaterialTheme.colorScheme.primary,
                            style = MaterialTheme.typography.bodySmall
                        )
                    }
                    OutlinedButton(onClick = { onPairPrompt(device) }) { Text(loc(ar, "\u0627\u0633\u062A\
u062E\u062F\u0645", "Use")) }
                }
            }
        }
        Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)) {
            Column(Modifier.fillMaxWidth().padding(14.dp), verticalArrangement = Arrangement.spacedBy(8.
dp)) {
                Text(loc(ar, "\u0637\u0631\u064A\u0642\u0629 \u0627\u0644\u0627\u062A\u0635\u0627\u0644",
"Connection method"), style = MaterialTheme.typography.titleSmall)
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    FilterChip(
                        selected = state.pairingProtocol == ReceiverDevice.ANDROID_TV_REMOTE,
                        onClick = { onPairingProtocol(ReceiverDevice.ANDROID_TV_REMOTE) },
                        label = { Text(loc(ar, "Android TV \u0645\u0628\u0627\u0634\u0631", "Android TV
direct")) }
                    )
                    FilterChip(
                        selected = state.pairingProtocol == ReceiverDevice.RECEIVER_CONTROL,
                        onClick = { onPairingProtocol(ReceiverDevice.RECEIVER_CONTROL) },
                        label = { Text(loc(ar, "\u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\
u064A\u0641\u0631", "Receiver app")) }
                    )
                }
                OutlinedTextField(value = state.pairingHost, onValueChange = onManualHost, label = { Text(
"IP") }, placeholder = { Text("192.168.1.20") }, modifier = Modifier.fillMaxWidth(), singleLine = true)
                if (state.pairingProtocol == ReceiverDevice.ANDROID_TV_REMOTE) {
                    Text(loc(ar, "\u0644\u0627 \u064A\u062D\u062A\u0627\u062C \u0647\u0630\u0627 \u0627\
u0644\u0645\u0633\u0627\u0631 \u062A\u062B\u0628\u064A\u062A \u062A\u0637\u0628\u064A\u0642 \u0639\u0644\
u0649 \u0627\u0644\u062A\u0644\u0641\u0627\u0632. \u0627\u062E\u062A\u0631 \u062C\u0647\u0627\u0632\u0627\
u064B \u0645\u0643\u062A\u0634\u0641\u0627\u064B \u0623\u0648 \u0623\u062F\u062E\u0644 IP\u060C \u062B\
u0645 \u0648\u0627\u0641\u0642 \u0639\u0644\u0649 \u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646 \u0641\
u064A \u0627\u0644\u062A\u0644\u0641\u0627\u0632.", "This path needs no TV app. Choose a discovered device
or enter its IP, then approve pairing on the TV."), color = MaterialTheme.colorScheme.onSurfaceVariant,
style = MaterialTheme.typography.bodySmall)
                    if (!state.androidTvPairingStarted) {
                        Button(onClick = { onStartAndroidTvPairing(state.pairingHost) }, modifier =
Modifier.fillMaxWidth(), enabled = state.pairingHost.isNotBlank()) {
                            Text(loc(ar, "\u0627\u0628\u062F\u0623 \u0627\u0644\u0627\u0642\u062A\u0631\
u0627\u0646 \u0639\u0644\u0649 \u0627\u0644\u062A\u0644\u0641\u0627\u0632", "Start pairing on TV"))
                        }
                    } else {
                        OutlinedTextField(
                            value = state.pairingCodeInput,
                            onValueChange = onCode,
                            label = { Text(loc(ar, "\u0631\u0645\u0632 \u0627\u0644\u062A\u0644\u0641\u0627\
u0632", "TV pairing code")) },
                            placeholder = { Text("123456") },
                            modifier = Modifier.fillMaxWidth(),
                            singleLine = true
                        )
                        Button(onClick = { onPair(state.pairingHost, state.pairingCodeInput) }, modifier =
Modifier.fillMaxWidth(), enabled = state.androidTvAwaitingCode && state.pairingCodeInput.length == 6) {
```

```
                        Text(loc(ar, "\u062A\u0623\u0643\u064A\u062F \u0631\u0645\u0632 \u0627\u0644\
u062A\u0644\u0641\u0627\u0632", "Submit TV code"))
                    }
                }
            } else {
                Text(loc(ar, "\u064A\u062C\u0628 \u062A\u0634\u063A\u064A\u0644 Android TV
responsiveness \u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631. \u0623\u062F\u062E\u0644 \
u0627\u0644\u0643\u0648\u062F \u0627\u0644\u0645\u0639\u0631\u0648\u0636 \u0647\u0646\u0627\u0643.", "Start
Android TV responsiveness on the receiver, then enter its displayed code."), color = MaterialTheme.
colorScheme.onSurfaceVariant, style = MaterialTheme.typography.bodySmall)
                OutlinedTextField(value = state.pairingCodeInput, onValueChange = onCode, label = {
Text(loc(ar, "\u0643\u0648\u062F \u0645\u0646 7 \u0623\u0631\u0642\u0627\u0645", "7-digit code")) },
placeholder = { Text("1234567") }, modifier = Modifier.fillMaxWidth(), singleLine = true)
                Button(onClick = { onPair(state.pairingHost, state.pairingCodeInput) }, modifier =
Modifier.fillMaxWidth(), enabled = state.pairingHost.isNotBlank() && state.pairingCodeInput.length == 7) {
                    Text(loc(ar, "\u0627\u062A\u0635\u0627\u0644 \u0628\u0627\u0644\u062A\u0637\u0628\
u064A\u0642 \u0627\u0644\u0645\u0631\u0627\u0641\u0642", "Connect to companion app"))
                }
            }
        }
        if (state.paired == null && state.devices.isEmpty()) Text(loc(ar, "\u0644\u0627 \u062A\u0648\u062C\
u062F \u0623\u062C\u0647\u0632\u0629 \u0645\u0643\u062A\u0634\u0641\u0629 \u062D\u0627\u0644\u064A\u0627\
u064B.", "No devices discovered yet."), color = MaterialTheme.colorScheme.onSurfaceVariant)
    }
}

@Composable
private fun MacrosPage(
    state: ControllerState,
    onSave: (String?, String, List<String>) -> Unit,
    onDelete: (String) -> Unit,
    onRun: (RemoteMacro) -> Unit
) {
    val ar = state.language
    var editingId by rememberSaveable { mutableStateOf<String?>(null) }
    var name by rememberSaveable { mutableStateOf("") }
    var keysText by rememberSaveable { mutableStateOf("") }
    Column(Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(16.dp), verticalArrangement
= Arrangement.spacedBy(10.dp)) {
        Text(loc(ar, "\u0627\u062E\u062A\u0635\u0627\u0631\u0627\u062A \u0627\u0644\u062A\u062D\u0643\
u0645", "Remote macros"), style = MaterialTheme.typography.titleLarge)
        Text(loc(ar, "\u0623\u0646\u0634\u0626 \u062A\u0633\u0644\u0633\u0644\u0627\u064B \u0645\u0646 \
u0627\u0644\u0623\u0632\u0631\u0627\u0631 \u0648\u0646\u0641\u0651\u0630\u0647\u0627 \u0628\u0636\u063A\u0637\
u0629 \u0648\u0627\u062D\u062F\u0629. \u0645\u062B\u0627\u0644: HOME, LEFT, ENTER.", "Create a sequence of
remote buttons and run it with one tap. Example: HOME, LEFT, ENTER."), color = MaterialTheme.colorScheme.
onSurfaceVariant)
        Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
            Column(Modifier.fillMaxWidth().padding(14.dp), verticalArrangement = Arrangement.spacedBy(8.
dp)) {
                Text(if (editingId == null) loc(ar, "\u0627\u062E\u062A\u0635\u0627\u0631 \u062C\u062F\
u064A\u062F", "New macro") else loc(ar, "\u062A\u0639\u062F\u064A\u0644 \u0627\u0644\u0627\u062E\u062A\
u0635\u0627\u0631", "Edit macro"), style = MaterialTheme.typography.titleSmall)
                OutlinedTextField(value = name, onValueChange = { name = it }, label = { Text(loc(ar, "\
u0627\u0644\u0627\u0633\u0645", "Name")) }, modifier = Modifier.fillMaxWidth(), singleLine = true)
                OutlinedTextField(value = keysText, onValueChange = { keysText = it }, label = { Text(loc(
ar, "\u0627\u0644\u0623\u0632\u0631\u0627\u0631 \u0628\u0627\u0644\u062A\u0631\u062A\u064A\u0628", "Keys in
order")) }, placeholder = { Text("HOME, LEFT, ENTER") }, modifier = Modifier.fillMaxWidth(), minLines = 2)
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Button(onClick = {
                        val keys = keysText.split(Regex("[,\\s]+" )).filter(String::isNotBlank).map { it.
uppercase() }
                        onSave(editingId, name, keys)
                        editingId = null; name = ""; keysText = ""
                    }, modifier = Modifier.weight(1f)) { Text(loc(ar, "\u062D\u0641\u0638", "Save")) }
                    if (editingId != null) OutlinedButton(onClick = { editingId = null; name = ""; keysText
= "" }) { Text(loc(ar, "\u0625\u0644\u063A\u0627\u0621", "Cancel")) }
                }
            }
        }
        state.macros.forEach { macro ->
            Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
                Column(Modifier.fillMaxWidth().padding(13.dp), verticalArrangement = Arrangement.spacedBy(6.
dp)) {
                    Text(macro.name, style = MaterialTheme.typography.titleSmall)
                    Text(macro.keys.joinToString(" \u2192 "), color = MaterialTheme.colorScheme.
onSurfaceVariant, style = MaterialTheme.typography.bodySmall)
                    Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                        Button(onClick = { onRun(macro) }, modifier = Modifier.weight(1f)) { Text("\u25B6
${loc(ar, "\u062A\u0634\u063A\u064A\u0644", "Run")}") }
                        TextButton(onClick = { editingId = macro.id; name = macro.name; keysText = macro.
keys.joinToString(", ") }) { Text(loc(ar, "\u062A\u0639\u062F\u064A\u0644", "Edit")) }
                        TextButton(onClick = { onDelete(macro.id) }) { Text("\u00D7  ${loc(ar, "\u062D\
u0630\u0641", "Delete")}") }
                    }
                }
            }
        }
        if (state.macros.isEmpty()) Text(loc(ar, "\u0644\u0645 \u062A\u064F\u0646\u0634\u0623 \u0627\u062E\
u062A\u0635\u0627\u0631\u0627\u062A \u0628\u0639\u062F.", "No macros saved yet."), color = MaterialTheme.
colorScheme.onSurfaceVariant)
    }
}

@Composable
```

```
private fun AppsPage(state: ControllerState, onRefresh: () -> Unit, onLaunch: (ReceiverApp) -> Unit,
onUninstall: (ReceiverApp) -> Unit) {
    val ar = state.language
    Column(Modifier.fillMaxSize().padding(horizontal = 16.dp, vertical = 4.dp)) {
        Row(Modifier.fillMaxWidth(), verticalAlignment = Alignment.CenterVertically, horizontalArrangement
= Arrangement.SpaceBetween) {
            Text(loc(ar, "\u062A\u0637\u0628\u064A\u0642\u0627\u062A \u0627\u0644\u0631\u0633\u064A\u0641\
u0631", "Receiver apps"), style = MaterialTheme.typography.titleMedium)
            Button(onClick = onRefresh, enabled = !state.loadingApps) { if (state.loadingApps)
CircularProgressIndicator(Modifier.size(18.dp), strokeWidth = 2.dp) else Text(loc(ar, "\u062A\u062D\u062F\
u064A\u062B", "Refresh")) }
        }
        Text(loc(ar, "\u064A\u0645\u0643\u0646 \u0641\u062A\u062D \u0627\u0644\u062A\u0637\u0628\u064A\
u0642\u0627\u062A \u0648\u062D\u0630\u0641\u0647\u0627. \u0627\u0644\u062D\u0630\u0641 \u0639\u0628\u0631 \
u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u064A\u062D\u062A\u0627\u062C \
u062A\u0623\u0643\u064A\u062F\u0627\u064B \u0639\u0644\u0649 \u0627\u0644\u062C\u0647\u0627\u0632.",
"Launch or uninstall apps. Uninstalling through the receiver app requires confirmation on that device."),
color = MaterialTheme.colorScheme.onSurfaceVariant, modifier = Modifier.padding(vertical = 8.dp))
        Column(Modifier.fillMaxSize().verticalScroll(rememberScrollState()), verticalArrangement =
Arrangement.spacedBy(8.dp)) {
            state.apps.forEach { app ->
                Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
                    Column(Modifier.fillMaxWidth().padding(12.dp)) {
                        Text(app.name, style = MaterialTheme.typography.titleSmall)
                        Text(app.packageName, color = MaterialTheme.colorScheme.onSurfaceVariant, style =
MaterialTheme.typography.bodySmall)
                        Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                            OutlinedButton(onClick = { onLaunch(app) }, modifier = Modifier.weight(1f)) {
Text("\u25B6  ${loc(ar, "\u0641\u062A\u062D", "Open")}") }
                            TextButton(onClick = { onUninstall(app) }, modifier = Modifier.weight(1f)) {
Text("\u232B  ${loc(ar, "\u0625\u0644\u063A\u0627\u0621 \u0627\u0644\u062A\u062B\u0628\u064A\u062A",
"Uninstall")}") }
                        }
                    }
                }
            }
            if (!state.loadingApps && state.apps.isEmpty()) Text(loc(ar, "\u0627\u0636\u063A\u0637 \u062A\
u062D\u062F\u064A\u062B \u0644\u062C\u0644\u0628 \u0627\u0644\u062A\u0637\u0628\u064A\u0642\u0627\u062A \
u0645\u0646 \u0627\u0644\u0631\u0633\u064A\u0641\u0631.", "Tap Refresh to load apps from the receiver."),
color = MaterialTheme.colorScheme.onSurfaceVariant)
        }
    }
}

@Composable
private fun MediaPage(state: ControllerState, onRefreshScreen: () -> Unit, onScreenLiveChange: (Boolean) ->
Unit, onPip: () -> Unit, onCastUrl: (String) -> Unit, onChoose: () -> Unit) {
    val ar = state.language
    var mediaUrl by rememberSaveable { mutableStateOf("") }
    Column(Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(16.dp), verticalArrangement
= Arrangement.spacedBy(14.dp)) {
        Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
            Column(Modifier.fillMaxWidth().padding(13.dp), verticalArrangement = Arrangement.spacedBy(8.
dp)) {
                Text(loc(ar, "\u0639\u0631\u0636 \u0634\u0627\u0634\u0629 \u0627\u0644\u0631\u0633\u064A\
u0641\u0631", "Receiver screen preview"), style = MaterialTheme.typography.titleMedium)
                Text(loc(ar, "\u062D\u062F\u0651\u062B \u0627\u0644\u0645\u0639\u0627\u064A\u0646\u0629 \
u064A\u062F\u0648\u064A\u064B\u0627 \u0639\u0646\u062F \u0627\u0644\u062D\u0627\u062C\u0629\u060C \u0648\
u064A\u0645\u0643\u0646 \u0641\u062A\u062D\u0647\u0627 \u0641\u064A \u0646\u0627\u0641\u0630\u0629 \u0639\
u0627\u0626\u0645\u0629.", "Refresh the preview manually when needed, or open it in a floating window."),
color = MaterialTheme.colorScheme.onSurfaceVariant, style = MaterialTheme.typography.bodySmall)
                if (state.screenPreview != null) Image(state.screenPreview.asImageBitmap(),
contentDescription = loc(ar, "\u0634\u0627\u0634\u0629 \u0627\u0644\u0631\u0633\u064A\u0641\u0631",
"Receiver screen"), modifier = Modifier.fillMaxWidth().aspectRatio(16f / 9f))
                else Box(Modifier.fillMaxWidth().aspectRatio(16f / 9f).background(MaterialTheme.colorScheme.
surfaceVariant, RoundedCornerShape(14.dp)), contentAlignment = Alignment.Center) {
                    if (state.screenBusy) CircularProgressIndicator() else Text(loc(ar, "\u0627\u0631\u0628\
u0637 \u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0648\u0641\u0639\u0651\
u0644 \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\u062D\u0643\u0645 \u0627\u0644\u0645\u0633\u0627\u0639\
u062F", "Pair the receiver app and enable its Accessibility service"), color = MaterialTheme.colorScheme.
onSurfaceVariant)
                }
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedButton(onClick = onRefreshScreen, modifier = Modifier.weight(1f), enabled =
!state.screenBusy && !state.screenLive) { Text(loc(ar, "\u0644\u0642\u0637\u0629 \u0627\u0644\u0622\u0646",
"Capture now")) }
                    Button(onClick = onPip, modifier = Modifier.weight(1f), enabled = state.screenPreview
!= null) { Text(loc(ar, "\u0646\u0627\u0641\u0630\u0629 \u0639\u0627\u0626\u0645\u0629", "Floating
window")) }
                }
                Button(
                    onClick = { onScreenLiveChange(!state.screenLive) },
                    modifier = Modifier.fillMaxWidth(),
                    enabled = state.paired != null && !state.screenBusy
                ) {
                    Text(if (state.screenLive) loc(ar, "\u0625\u064A\u0642\u0627\u0641 \u0627\u0644\u0639\
u0631\u0636 \u0627\u0644\u0645\u0628\u0627\u0634\u0631", "Stop live preview") else loc(ar, "\u0628\u062F\
u0621 \u0627\u0644\u0639\u0631\u0636 \u0627\u0644\u0645\u0628\u0627\u0634\u0631", "Start live preview"))
                }
            }
        }
        Text(loc(ar, "\u0625\u0631\u0633\u0627\u0644 \u0645\u0644\u0641\u0627\u062A \u0648\u0648\u0633\
u0627\u0626\u0637", "Send files and media"), style = MaterialTheme.typography.titleLarge)
        Text(loc(ar, "\u0627\u062E\u062A\u0631 \u0635\u0648\u0631\u0629 \u0623\u0648 \u0641\u064A\u062F\
u064A\u0648 \u0623\u0648 \u0645\u0644\u0641\u0627\u064B. \u0627\u0644\u0644\u0625\u0631\u0633\u0627\u0644 \u064A\
```

```
u0639\u0645\u0644 \u0639\u0628\u0631 ADB TCP \u0623\u0648 \u0639\u0628\u0631 \u062A\u0637\u0628\u064A\u0642
Android TV responsiveness \u0627\u0644\u0645\u0642\u062A\u0631\u0646.", "Choose an image, video, or file.
Transfer uses ADB TCP or the paired Android TV responsiveness app."), color = MaterialTheme.colorScheme.
onSurfaceVariant)
        Button(onClick = onChoose, modifier = Modifier.fillMaxWidth().height(54.dp)) { Text("\u21E7  ${loc(
ar, "\u0627\u062E\u062A\u064A\u0627\u0631 \u0645\u0644\u0641 \u0644\u0644\u0625\u0631\u0633\u0627\u0644",
"Choose file to send")}") }
        Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
            Column(Modifier.fillMaxWidth().padding(14.dp), verticalArrangement = Arrangement.spacedBy(8.
dp)) {
                Text(loc(ar, "\u0641\u062A\u062D \u0631\u0627\u0628\u0637 \u0648\u0633\u0627\u0626\u0637 \
u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631", "Open media URL on receiver"), style =
MaterialTheme.typography.titleSmall)
                OutlinedTextField(value = mediaUrl, onValueChange = { mediaUrl = it }, modifier = Modifier.
fillMaxWidth(), singleLine = true, label = { Text("HTTP / HTTPS") })
                OutlinedButton(onClick = { onCastUrl(mediaUrl) }, modifier = Modifier.fillMaxWidth()) {
Text(loc(ar, "\u062A\u0634\u063A\u064A\u0644 \u0627\u0644\u0631\u0627\u0628\u0637", "Open URL")) }
            }
        }
        Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
            Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
                Text(loc(ar, "\u0648\u062C\u0647\u0629 \u0627\u0644\u0644\u0646\u0642\u0644", "Transfer
destination"), style = MaterialTheme.typography.titleSmall)
                Text(if (state.adbHost.isNotBlank()) "ADB \u00B7 ${state.adbHost}:${state.adbPort}" else
state.paired?.let { "${it.name} \u00B7 ${it.host}" } ?: loc(ar, "\u0644\u0627 \u064A\u0648\u062C\u062F \
u062C\u0647\u0627\u0632 \u0645\u062A\u0635\u0644", "No receiver connected"), color = MaterialTheme.
colorScheme.onSurfaceVariant)
                Text(loc(ar, "\u062D\u062F \u0627\u0644\u062D\u062C\u0645 \u0639\u0646\u062F \u0627\u0644\
u0646\u0642\u0644 \u0639\u0628\u0631 \u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\u0641\
u0631: 512 \u0645\u064A\u063A\u0627\u0628\u0627\u064A\u062A \u0644\u0644\u0645\u0644\u0641 \u0627\u0644\
u0648\u0627\u062D\u062F.", "File size limit through the receiver app: 512 MB per file."), style =
MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
            }
        }
    }
}

@Composable
private fun ScreenPage(state: ControllerState, onRefresh: () -> Unit, onPip: () -> Unit) {
    val ar = state.language
    Column(Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(16.dp), verticalArrangement
= Arrangement.spacedBy(12.dp)) {
        Text(loc(ar, "\u0639\u0631\u0636 \u0634\u0627\u0634\u0629 \u0627\u0644\u0631\u0633\u064A\u0641\
u0631", "Receiver screen preview"), style = MaterialTheme.typography.titleLarge)
        Text(loc(ar, "\u062A\u0635\u0644 \u0627\u0644\u0644\u0642\u0637\u0629 \u0639\u0628\u0631 \u0627\
u062A\u0635\u0627\u0644 \u0627\u0644\u062A\u0637\u0628\u064A\u0642\u064A\u0646 \u0627\u0644\u0645\u062D\
u0644\u064A\u061B \u0641\u0639\u0651\u0644 \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\u062D\u0643\u0645 \
u0627\u0644\u0645\u0633\u0627\u0639\u062F \u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631. \
u0627\u0644\u0646\u0627\u0641\u0630\u0629 \u0627\u0644\u0639\u0627\u0626\u0645\u0629 \u062A\u0633\u062A\
u062E\u062F\u0645 \u0648\u0636\u0639 \u0635\u0648\u0631\u0629 \u062F\u0627\u062E\u0644 \u0635\u0648\u0631\
u0629.", "The snapshot comes over the paired app connection. Enable Accessibility on the receiver. Floating
preview uses picture-in-picture."), color = MaterialTheme.colorScheme.onSurfaceVariant)
        if (state.screenPreview != null) {
            Image(state.screenPreview.asImageBitmap(), contentDescription = loc(ar, "\u0634\u0627\u0634\
u0629 \u0627\u0644\u0631\u0633\u064A\u0641\u0631", "Receiver screen"), modifier = Modifier.fillMaxWidth().
aspectRatio(16f / 9f))
            Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                OutlinedButton(onClick = onRefresh, modifier = Modifier.weight(1f)) { Text(loc(ar, "\u062A\
u062D\u062F\u064A\u062B \u0627\u0644\u0622\u0646", "Refresh now")) }
                Button(onClick = onPip, modifier = Modifier.weight(1f)) { Text(loc(ar, "\u0646\u0627\u0641\
u0630\u0629 \u0639\u0627\u0626\u0645\u0629", "Floating window")) }
            }
        } else {
            Box(Modifier.fillMaxWidth().aspectRatio(16f / 9f).background(MaterialTheme.colorScheme.
surfaceVariant, RoundedCornerShape(18.dp)), contentAlignment = Alignment.Center) {
                if (state.screenBusy) CircularProgressIndicator() else Text(loc(ar, "\u0627\u0631\u0628\
u0637 \u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0648\u0641\u0639\u0651\
u0644 \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\u062D\u0643\u0645 \u0627\u0644\u0645\u0633\u0627\u0639\
u062F", "Pair the receiver app and enable its Accessibility service"), color = MaterialTheme.colorScheme.
onSurfaceVariant)
            }
            Button(onClick = onRefresh, modifier = Modifier.fillMaxWidth(), enabled = !state.screenBusy) {
Text(loc(ar, "\u0627\u0644\u062A\u0642\u0627\u0637 \u0623\u0648\u0644 \u0635\u0648\u0631\u0629", "Capture
first frame")) }
        }
    }
}

@Composable
private fun SettingsPage(
    state: ControllerState,
    onTheme: (String) -> Unit,
    onLanguage: (String) -> Unit,
    onAdbHost: (String) -> Unit,
    onAdbPort: (String) -> Unit,
    onAdbConnect: () -> Unit,
    onAdbCommand: (String) -> Unit,
    onRunAdbCommand: (String) -> Unit,
    onEditFavorite: (Int, String, String) -> Unit,
    onRunFavorite: (Int) -> Unit,
    onCustomJson: () -> Unit,
    onCustomJsonText: (String) -> Unit
) {
    val ar = state.language
    var advanced by rememberSaveable { mutableStateOf(false) }
```

```
    Column(Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(16.dp), verticalArrangement
= Arrangement.spacedBy(14.dp)) {
        Text(loc(ar, "\u0627\u0644\u0625\u0639\u062F\u0627\u062F\u0627\u062A", "Settings"), style =
MaterialTheme.typography.titleLarge)
        Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
            Column(Modifier.fillMaxWidth().padding(16.dp), verticalArrangement = Arrangement.spacedBy(10.
dp)) {
                Text("\u25D0  ${loc(ar, "\u0645\u0638\u0647\u0631 \u0627\u0644\u062A\u0637\u0628\u064A\
u0642", "Appearance")}", style = MaterialTheme.typography.titleMedium)
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    FilterChip(selected = state.theme == "purple", onClick = { onTheme("purple") }, label =
{ Text(loc(ar, "\u0628\u0646\u0641\u0633\u062C\u064A \u0631\u0645\u0627\u062F\u064A", "Purple gray")) })
                    FilterChip(selected = state.theme == "white", onClick = { onTheme("white") }, label = {
Text(loc(ar, "\u0623\u0628\u064A\u0636 \u0643\u0627\u0645\u0644", "Pure white")) })
                }
                Divider()
                Text("\u6587  ${loc(ar, "\u0627\u0644\u0644\u063A\u0629", "Language")}", style =
MaterialTheme.typography.titleMedium)
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    FilterChip(selected = ar == "ar", onClick = { onLanguage("ar") }, label = { Text("\
u0627\u0644\u0639\u0631\u0628\u064A\u0629") })
                    FilterChip(selected = ar == "en", onClick = { onLanguage("en") }, label = { Text(
"English") })
                }
            }
        }
        Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
            Column(Modifier.fillMaxWidth().padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.
dp)) {
                Text(loc(ar, "\u0627\u0644\u0627\u062A\u0635\u0627\u0644 \u0627\u0644\u0639\u0627\u062F\
u064A", "Standard connection"), style = MaterialTheme.typography.titleMedium)
                Text(state.paired?.let { "${it.name} \u00B7 ${it.host}" } ?: loc(ar, "\u064A\u0645\u0643\
u0646 \u0631\u0628\u0637 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0628\u0645\u0633\u062D QR \u0623\
u0648 \u0643\u0648\u062F 7 \u0623\u0631\u0642\u0627\u0645.", "Pair the receiver by scanning its QR code or
entering its 7-digit code."), color = MaterialTheme.colorScheme.onSurfaceVariant)
                Divider()
                Text(loc(ar, "\u0625\u0631\u0633\u0627\u0644 JSON \u0645\u062E\u0635\u0635", "Custom JSON
sender"), style = MaterialTheme.typography.titleSmall)
                OutlinedTextField(value = state.customJson, onValueChange = onCustomJsonText, modifier =
Modifier.fillMaxWidth(), minLines = 3, label = { Text("JSON") })
                OutlinedButton(onClick = onCustomJson, modifier = Modifier.fillMaxWidth()) { Text(loc(ar, "\
u0625\u0631\u0633\u0627\u0644 \u0625\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631", "Send to
receiver")) }
            }
        }
        Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)) {
            Column(Modifier.fillMaxWidth().padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.
dp)) {
                TextButton(onClick = { advanced = !advanced }, modifier = Modifier.fillMaxWidth()) {
                    Text("\u26A0  ${loc(ar, "\u0625\u0639\u062F\u0627\u062F\u0627\u062A \u0645\u062A\u0642\
u062F\u0645\u0629 \u00B7 ADB", "Advanced settings \u00B7 ADB")}  ${if (advanced) "\u25B2" else "\u25BC"}")
                }
                if (!advanced) Text(loc(ar, "\u0645\u062E\u0641\u064A\u0629 \u0627\u0641\u062A\u0631\u0627\
u0636\u064A\u0627\u064B. \u0627\u0641\u062A\u062D\u0647\u0627 \u0641\u0642\u0637 \u0625\u0630\u0627 \u0643\
u0646\u062A \u062A\u0639\u0631\u0641 \u0623\u0648\u0627\u0645\u0631 ADB.", "Hidden by default. Open only if
you understand ADB commands."), color = MaterialTheme.colorScheme.onSurfaceVariant)
                if (advanced) {
                    Text(loc(ar, "ADB \u064A\u0645\u0646\u062D \u0623\u0648\u0627\u0645\u0631\u0643 \u0635\
u0644\u0627\u062D\u064A\u0627\u062A shell \u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \
u0628\u0639\u062F \u0645\u0648\u0627\u0641\u0642\u0629 \u0628\u0635\u0645\u0629 \u0627\u0644\u0645\u0641\
u062A\u0627\u062D. \u0627\u062A\u0635\u0627\u0644 \u0647\u0630\u0627 \u0627\u0644\u062A\u0637\u0628\u064A\
u0642 \u064A\u062F\u0639\u0645 ADB TCP \u0627\u0644\u062A\u062A\u0642\u0644\u064A\u062F\u064A \u0641\u0642\u0637\
u061B \u0644\u0627 \u064A\u062F\u0639\u0645 \u0634\u0627\u0634\u0629 \u0627\u0644\u0627\u062A\u0635\u0627\
u0627\u0646 \u0627\u0644\u0644\u0644\u0627\u0633\u0644\u0643\u064A \u0630\u0627\u062A \u0627\u0644\u0645\u0646\
u0641\u0630 \u0627\u0644\u0645\u0624\u0642\u062A.", "ADB commands run with the receiver's shell privileges
after you approve its key. This app supports classic ADB TCP only; it does not support the newer wireless-
debugging pairing port."), color = MaterialTheme.colorScheme.onSurfaceVariant, style = MaterialTheme.
typography.bodySmall)
                    Row(horizontalArrangement = Arrangement.spacedBy(8.dp), verticalAlignment = Alignment.
CenterVertically) {
                        OutlinedTextField(value = state.adbHost, onValueChange = onAdbHost, label = { Text(
"IP") }, modifier = Modifier.weight(1f), singleLine = true)
                        OutlinedTextField(value = state.adbPort, onValueChange = onAdbPort, label = { Text(
loc(ar, "\u0627\u0644\u0645\u0646\u0641\u0630", "Port")) }, modifier = Modifier.width(104.dp), singleLine =
true)
                    }
                    Button(onClick = onAdbConnect, modifier = Modifier.fillMaxWidth()) { Text(if (state.
adbConnected) loc(ar, "\u0625\u0639\u0627\u062F\u0629 \u0627\u0644\u0627\u062A\u0635\u0627\u0644 \u0628\
u0640 ADB", "Reconnect ADB") else loc(ar, "\u0627\u062A\u0635\u0627\u0644 ADB", "Connect ADB")) }
                    OutlinedTextField(
                        value = state.adbCommand,
                        onValueChange = onAdbCommand,
                        label = { Text(loc(ar, "\u0623\u0645\u0631 ADB", "ADB command")) },
                        placeholder = { Text("input keyevent 3") },
                        modifier = Modifier.fillMaxWidth(),
                        minLines = 2
                    )
                    OutlinedButton(onClick = { onRunAdbCommand(state.adbCommand) }, modifier = Modifier.
fillMaxWidth()) { Text(loc(ar, "\u062A\u0646\u0641\u064A\u0630 \u0627\u0644\u0623\u0645\u0631", "Run
command")) }
                    Text(loc(ar, "\u0627\u0644\u0623\u0648\u0627\u0645\u0631 \u0627\u0644\u0645\u0641\u0636\
u0644\u0629 (10 \u0642\u0627\u0628\u0644\u0629 \u0644\u0644\u062A\u0639\u062F\u064A\u0644)", "Favorite
commands (10 editable slots)"), style = MaterialTheme.typography.titleSmall)
                    state.favorites.forEachIndexed { index, favorite ->
                        FavoriteEditor(index, favorite, ar, { name, command -> onEditFavorite(index, name,
```

```
command) }, { onRunFavorite(index) })
                    }
                }
            }
        }
    }
}

@Composable
private fun FavoriteEditor(index: Int, favorite: FavoriteCommand, language: String, onChange: (String,
String) -> Unit, onRun: () -> Unit) {
    Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface), shape =
RoundedCornerShape(14.dp)) {
        Column(Modifier.fillMaxWidth().padding(10.dp), verticalArrangement = Arrangement.spacedBy(5.dp)) {
            Text("${index + 1}. ${loc(language, "\u0645\u0641\u0636\u0644", "Favorite")}", color =
MaterialTheme.colorScheme.onSurfaceVariant, style = MaterialTheme.typography.labelSmall)
            OutlinedTextField(value = favorite.name, onValueChange = { onChange(it, favorite.command) },
label = { Text(loc(language, "\u0627\u0644\u0627\u0633\u0645", "Name")) }, modifier = Modifier.fillMaxWidth(
), singleLine = true)
            OutlinedTextField(value = favorite.command, onValueChange = { onChange(favorite.name, it) },
label = { Text("ADB") }, modifier = Modifier.fillMaxWidth(), singleLine = true)
            OutlinedButton(onClick = onRun, modifier = Modifier.fillMaxWidth()) { Text("\u25B6  ${loc(
language, "\u062A\u0646\u0641\u064A\u0630", "Run")}") }
        }
    }
}

@Composable
private fun QrScannerDialog(language: String, onDismiss: () -> Unit, onScanned: (String) -> Unit) {
    Dialog(onDismissRequest = onDismiss, properties = DialogProperties(usePlatformDefaultWidth = false)) {
        Surface(Modifier.fillMaxWidth(0.92f).height(520.dp), shape = RoundedCornerShape(24.dp), color =
MaterialTheme.colorScheme.surface) {
            Box(Modifier.fillMaxSize()) {
                CameraQrPreview(onScanned)
                Column(Modifier.align(Alignment.TopCenter).fillMaxWidth().background(Color.Black.copy(alpha
= 0.52f)).padding(16.dp)) {
                    Text(loc(language, "\u0648\u062C\u0651\u0647 \u0627\u0644\u0643\u0627\u0645\u064A\u0631\
u0627 \u0625\u0644\u0649 \u0631\u0645\u0632 QR \u0641\u064A \u0627\u0644\u0631\u0633\u064A\u0641\u0631",
"Point the camera at the receiver QR"), color = Color.White)
                }
                TextButton(onClick = onDismiss, modifier = Modifier.align(Alignment.BottomCenter).padding(
12.dp)) { Text(loc(language, "\u0625\u063A\u0644\u0627\u0642", "Close"), color = Color.White) }
            }
        }
    }
}

@Composable
private fun CameraQrPreview(onScanned: (String) -> Unit) {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current
    val executor = remember { Executors.newSingleThreadExecutor() }
    val decoded = remember { AtomicBoolean(false) }
    val lastFrame = remember { AtomicLong(0) }
    var cameraProvider by remember { mutableStateOf<ProcessCameraProvider?>(null) }
    var cameraError by remember { mutableStateOf<String?>(null) }
    Box(Modifier.fillMaxSize()) {
        androidx.compose.ui.viewinterop.AndroidView(
            modifier = Modifier.fillMaxSize(),
            factory = { ctx ->
                PreviewView(ctx).also { previewView ->
                    previewView.scaleType = PreviewView.ScaleType.FILL_CENTER
                    val future = ProcessCameraProvider.getInstance(ctx)
                    future.addListener({
                        try {
                            val provider = future.get()
                            cameraProvider = provider
                            val preview = Preview.Builder().setTargetResolution(Size(1280, 720)).build().
also { it.setSurfaceProvider(previewView.surfaceProvider) }
                            val analysis = ImageAnalysis.Builder().setBackpressureStrategy(ImageAnalysis.
STRATEGY_KEEP_ONLY_LATEST).build()
                            analysis.setAnalyzer(executor) { image ->
                                val now = System.currentTimeMillis()
                                if (!decoded.get() && now - lastFrame.get() > 350) {
                                    lastFrame.set(now)
                                    decodeQr(image)?.let { content ->
                                        if (decoded.compareAndSet(false, true)) android.os.Handler(android.
os.Looper.getMainLooper()).post { onScanned(content) }
                                    }
                                }
                                image.close()
                            }
                            provider.unbindAll()
                            provider.bindToLifecycle(lifecycleOwner, CameraSelector.DEFAULT_BACK_CAMERA,
preview, analysis)
                        } catch (e: Exception) {
                            cameraError = e.message ?: "Camera unavailable"
                        }
                    }, ContextCompat.getMainExecutor(ctx))
                }
            }
        )
        cameraError?.let { Text(it, Modifier.align(Alignment.Center).padding(20.dp), color = Color.White) }
    }
    DisposableEffect(Unit) {
```

```
        onDispose {
            cameraProvider?.unbindAll()
            executor.shutdown()
        }
    }
}

private fun decodeQr(image: ImageProxy): String? {
    if (image.format != ImageFormat.YUV_420_888) return null
    return try {
        val plane = image.planes[0]
        val sourceBuffer = plane.buffer.duplicate()
        val width = image.width
        val height = image.height
        val bytes = ByteArray(width * height)
        val rowStride = plane.rowStride
        val pixelStride = plane.pixelStride
        for (y in 0 until height) for (x in 0 until width) {
            bytes[y * width + x] = sourceBuffer.get(y * rowStride + x * pixelStride)
        }
        val hints = EnumMap<DecodeHintType, Any>(DecodeHintType::class.java).apply { put(DecodeHintType.
TRY_HARDER, true) }
        var rotated = bytes
        var rotatedWidth = width
        var rotatedHeight = height
        repeat(4) { turn ->
            try {
                val source = PlanarYUVLuminanceSource(rotated, rotatedWidth, rotatedHeight, 0, 0,
rotatedWidth, rotatedHeight, false)
                return QRCodeReader().decode(BinaryBitmap(HybridBinarizer(source)), hints).text
            } catch (_: Exception) {
                if (turn < 3) {
                    val next = ByteArray(rotated.size)
                    val nextWidth = rotatedHeight
                    val nextHeight = rotatedWidth
                    for (y in 0 until rotatedHeight) for (x in 0 until rotatedWidth) {
                        val newX = rotatedHeight - 1 - y
                        val newY = x
                        next[newY * nextWidth + newX] = rotated[y * rotatedWidth + x]
                    }
                    rotated = next
                    rotatedWidth = nextWidth
                    rotatedHeight = nextHeight
                }
            }
        }
        null
    } catch (_: Exception) {
        null
    }
}
```

controller/src/main/java/com/monib/receivercontrol/controller/LanClient.kt

```
package com.monib.receivercontrol.controller

import android.content.Context
import android.net.wifi.WifiManager
import com.google.gson.Gson
import com.google.gson.JsonObject
import com.google.gson.JsonParser
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.async
import kotlinx.coroutines.coroutineScope
import kotlinx.coroutines.withContext
import java.io.DataInputStream
import java.io.DataOutputStream
import java.net.DatagramPacket
import java.net.DatagramSocket
import java.net.InetAddress
import java.net.InetSocketAddress
import java.net.Socket

data class ReceiverDevice(
    val name: String,
    val host: String,
    val port: Int = LanClient.COMMAND_PORT,
    val protocol: String = RECEIVER_CONTROL
) {
    companion object {
        const val RECEIVER_CONTROL = "receivercontrol"
        const val ANDROID_TV_REMOTE = "androidtvremote"
    }
}
data class BridgeReply(val ok: Boolean, val message: String, val raw: JsonObject)

object LanClient {
    private const val DISCOVERY_PORT = 39000
    const val COMMAND_PORT = 39001
    private const val MAX_FRAME = 2 * 1024 * 1024
    private val gson = Gson()

    suspend fun discover(context: Context): List<ReceiverDevice> = coroutineScope {
        val companion = async(Dispatchers.IO) { discoverReceiverControl(context) }
        val androidTv = async(Dispatchers.IO) { AndroidTvDiscovery(context).discover() }
        (companion.await() + androidTv.await()).distinctBy { "${it.protocol}:${it.host}" }
    }

    private suspend fun discoverReceiverControl(context: Context): List<ReceiverDevice> = withContext(
Dispatchers.IO) {
        val found = linkedMapOf<String, ReceiverDevice>()
        val payload = "{\"type\":\"DISCOVER\"}".toByteArray(Charsets.UTF_8)
        DatagramSocket().use { socket ->
            socket.broadcast = true
            socket.soTimeout = 350
            val targets = buildList {
                directedBroadcast(context)?.let(::add)
                add(InetAddress.getByName("255.255.255.255"))
            }.distinctBy { it.hostAddress }
            for (target in targets) {
                runCatching { socket.send(DatagramPacket(payload, payload.size, target, DISCOVERY_PORT)) }
            }

            val deadline = System.currentTimeMillis() + 2800
            val buffer = ByteArray(4096)
            while (System.currentTimeMillis() < deadline) {
                try {
                    val packet = DatagramPacket(buffer, buffer.size)
                    socket.receive(packet)
                    val json = JsonParser.parseString(String(packet.data, packet.offset, packet.length,
Charsets.UTF_8)).asJsonObject
                    if (json.get("type")?.asString == "RECEIVER") {
                        val host = packet.address.hostAddress ?: continue
                        val port = json.get("port")?.asInt ?: COMMAND_PORT
                        found[host] = ReceiverDevice(json.get("name")?.asString ?: "Android Receiver", host,
 port, ReceiverDevice.RECEIVER_CONTROL)
                    }
                } catch (_: java.net.SocketTimeoutException) {
                    // Continue until the discovery window has elapsed.
                } catch (_: Exception) {
                    // Ignore malformed packets and keep collecting valid receiver replies.
                }
            }
        }
        found.values.toList()
    }

    suspend fun pair(device: ReceiverDevice, code: String): BridgeReply = request(device, code, JsonObject(
).apply { addProperty("type", "PAIR") })

    suspend fun command(device: ReceiverDevice, code: String, action: String, value: String = ""):
BridgeReply = request(
        device, code, JsonObject().apply {
            addProperty("type", "COMMAND")
            addProperty("action", action)
```

```
            addProperty("value", value)
        }
    )

    suspend fun custom(device: ReceiverDevice, code: String, jsonText: String): BridgeReply = request(
        device, code, JsonObject().apply {
            addProperty("type", "COMMAND")
            addProperty("action", "CUSTOM")
            addProperty("json", jsonText)
        }
    )

    suspend fun sendFile(device: ReceiverDevice, code: String, name: String, mime: String, size: Long, open:
 () -> java.io.InputStream?): BridgeReply = withContext(Dispatchers.IO) {
        require(name.isNotBlank() && name.length <= 200) { "\u0627\u0633\u0645 \u0627\u0644\u0645\u0644\
u0641 \u063A\u064A\u0631 \u0635\u0627\u0644\u062D" }
        require(size in 0..(512L * 1024 * 1024)) { "\u062D\u062C\u0645 \u0627\u0644\u0645\u0644\u0641 \
u064A\u062A\u062C\u0627\u0648\u0632 \u0627\u0644\u062D\u062F 512 \u0645\u064A\u063A\u0627\u0628\u0627\u064A\
u062A" }
        val socket = Socket()
        socket.connect(InetSocketAddress(device.host, device.port), 5000)
        socket.soTimeout = 20_000
        socket.use { client ->
            val output = DataOutputStream(client.getOutputStream())
            val input = DataInputStream(client.getInputStream())
            val header = JsonObject().apply {
                addProperty("type", "FILE")
                addProperty("code", code)
                addProperty("name", name)
                addProperty("mime", mime)
                addProperty("size", size)
            }.toString().toByteArray(Charsets.UTF_8)
            output.writeInt(header.size)
            output.write(header)
            output.flush()
            val stream = open() ?: throw IllegalStateException("\u062A\u0639\u0630\u0631 \u0641\u062A\u062D
\u0627\u0644\u0645\u0644\u0641 \u0627\u0644\u0645\u062D\u062F\u062F")
            stream.use { source ->
                val buffer = ByteArray(32 * 1024)
                var remaining = size
                while (remaining > 0) {
                    val count = source.read(buffer, 0, minOf(buffer.size.toLong(), remaining).toInt())
                    if (count < 0) throw java.io.EOFException("\u0627\u0646\u062A\u0647\u0649 \u0627\u0644\
u0645\u0644\u0641 \u0642\u0628\u0644 \u0627\u0643\u062A\u0645\u0627\u0644 \u0627\u0644\u0625\u0631\u0633\
u0627\u0644")
                    output.write(buffer, 0, count)
                    remaining -= count
                }
                output.flush()
            }
            readReply(input)
        }
    }

    private suspend fun request(device: ReceiverDevice, code: String, request: JsonObject): BridgeReply =
withContext(Dispatchers.IO) {
        request.addProperty("code", code)
        val socket = Socket()
        socket.connect(InetSocketAddress(device.host, device.port), 4500)
        socket.soTimeout = 8000
        socket.use { client ->
            val output = DataOutputStream(client.getOutputStream())
            val input = DataInputStream(client.getInputStream())
            val bytes = gson.toJson(request).toByteArray(Charsets.UTF_8)
            output.writeInt(bytes.size)
            output.write(bytes)
            output.flush()
            readReply(input)
        }
    }

    private fun readReply(input: DataInputStream): BridgeReply {
        val length = input.readInt()
        if (length !in 1..MAX_FRAME) throw IllegalStateException("\u0631\u062F \u063A\u064A\u0631 \u0635\
u0627\u0644\u062D \u0645\u0646 \u0627\u0644\u0631\u0633\u064A\u0641\u0631")
        val bytes = ByteArray(length)
        input.readFully(bytes)
        val json = JsonParser.parseString(String(bytes, Charsets.UTF_8)).asJsonObject
        return BridgeReply(json.get("ok")?.asBoolean == true, json.get("message")?.asString.orEmpty(), json)
    }

    @Suppress("DEPRECATION")
    private fun directedBroadcast(context: Context): InetAddress? = runCatching {
        val manager = context.applicationContext.getSystemService(Context.WIFI_SERVICE) as WifiManager
        val dhcp = manager.dhcpInfo ?: return null
        val broadcast = (dhcp.ipAddress and dhcp.netmask) or dhcp.netmask.inv()
        InetAddress.getByAddress(byteArrayOf(
            (broadcast and 0xff).toByte(),
            (broadcast shr 8 and 0xff).toByte(),
            (broadcast shr 16 and 0xff).toByte(),
            (broadcast shr 24 and 0xff).toByte()
        ))
    }.getOrNull()
}
```

controller/src/main/java/com/monib/receivercontrol/controller/MainActivity.kt

```
package com.monib.receivercontrol.controller

import android.app.PictureInPictureParams
import android.content.ContentResolver
import android.content.res.Configuration
import android.content.Intent
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.net.Uri
import android.os.Build
import android.os.Bundle
import android.util.Base64
import android.util.Rational
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.lifecycle.lifecycleScope
import com.google.gson.Gson
import com.google.gson.JsonParser
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.io.IOException
import java.net.ConnectException
import java.net.SocketTimeoutException

data class FavoriteCommand(val name: String, val command: String)
data class ReceiverApp(val name: String, val packageName: String)
data class SavedReceiver(
    val name: String,
    val host: String,
    val port: Int,
    val code: String,
    val protocol: String = ReceiverDevice.RECEIVER_CONTROL
)
data class RemoteMacro(val id: String, val name: String, val keys: List<String>)
data class ControllerState(
    val tab: String = "remote",
    val theme: String = "purple",
    val language: String = "ar",
    val devices: List<ReceiverDevice> = emptyList(),
    val discovering: Boolean = false,
    val paired: ReceiverDevice? = null,
    val pairCode: String = "",
    val profiles: List<SavedReceiver> = emptyList(),
    val macros: List<RemoteMacro> = emptyList(),
    val pairingHost: String = "",
    val pairingRequested: Boolean = false,
    val pairingCodeInput: String = "",
    val pairingProtocol: String = ReceiverDevice.RECEIVER_CONTROL,
    val androidTvPairingStarted: Boolean = false,
    val androidTvAwaitingCode: Boolean = false,
    val adbHost: String = "",
    val adbPort: String = "5555",
    val adbConnected: Boolean = false,
    val adbCommand: String = "",
    val customJson: String = "{\"action\":\"INFO\"}",
    val favorites: List<FavoriteCommand> = defaultFavorites(),
    val apps: List<ReceiverApp> = emptyList(),
    val loadingApps: Boolean = false,
    val status: String = loc("ar", "\u0644\u0645 \u064A\u062A\u0645 \u0627\u0644\u0627\u062A\u0635\u0627\
u0644 \u0628\u0623\u064A \u0631\u0633\u064A\u0641\u0631", "No receiver connected"),
    val success: Boolean? = null,
    val screenPreview: Bitmap? = null,
    val screenBusy: Boolean = false,
    val screenLive: Boolean = false,
    val pictureInPicture: Boolean = false,
    val websiteBannerDismissed: Boolean = false,
    val showQrScanner: Boolean = false
)

private fun defaultFavorites(language: String = "ar") = listOf(
    FavoriteCommand(loc(language, "\u062A\u0634\u063A\u064A\u0644/\u0625\u064A\u0642\u0627\u0641 \u0627\
u0644\u0634\u0627\u0634\u0629", "Screen power"), "input keyevent 26"),
    FavoriteCommand(loc(language, "\u0627\u0644\u0631\u0626\u064A\u0633\u064A\u0629", "Home"), "input
keyevent 3"),
    FavoriteCommand(loc(language, "\u0631\u062C\u0648\u0639", "Back"), "input keyevent 4"),
    FavoriteCommand(loc(language, "\u064A\u0633\u0627\u0631", "Left"), "input keyevent 21"),
    FavoriteCommand(loc(language, "\u064A\u0645\u064A\u0646", "Right"), "input keyevent 22"),
    FavoriteCommand(loc(language, "\u0623\u0639\u0644\u0649", "Up"), "input keyevent 19"),
    FavoriteCommand(loc(language, "\u0623\u0633\u0641\u0644", "Down"), "input keyevent 20"),
    FavoriteCommand(loc(language, "\u0645\u0648\u0627\u0641\u0642", "Enter"), "input keyevent 66"),
    FavoriteCommand(loc(language, "\u0631\u0641\u0639 \u0627\u0644\u0635\u0648\u062A", "Volume up"), "input
keyevent 24"),
    FavoriteCommand(loc(language, "\u062E\u0641\u0636 \u0627\u0644\u0635\u0648\u062A", "Volume down"),
"input keyevent 25")
)

class MainActivity : ComponentActivity() {
```

```
    private val adb by lazy { AdbShellClient() }
    private val androidTvRemote by lazy { AndroidTvRemoteClient(applicationContext) }
    private val gson = Gson()
    private var ui by androidx.compose.runtime.mutableStateOf(ControllerState())
    private val prefs by lazy { getSharedPreferences("controller", MODE_PRIVATE) }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        ui = loadSavedState()
        setContent {
            ReceiverControlApp(
                state = ui,
                onTab = { ui = ui.copy(tab = it) },
                onDiscover = ::discoverDevices,
                onPairPrompt = ::selectDiscoveredDevice,
                onPair = ::pairDevice,
                onPairingProtocol = ::changePairingProtocol,
                onStartAndroidTvPairing = ::startAndroidTvPairing,
                onQrScan = ::handleQrScan,
                onScannerVisibility = { ui = ui.copy(showQrScanner = it) },
                onManualHost = { ui = ui.copy(pairingHost = it) },
                onPairCodeInput = { input ->
                    val code = if (ui.pairingProtocol == ReceiverDevice.ANDROID_TV_REMOTE) {
                        input.filter { it.isDigit() || it.uppercaseChar() in 'A'..'F' }.uppercase().take(6)
                    } else input.filter(Char::isDigit).take(7)
                    ui = ui.copy(pairingCodeInput = code)
                },
                onSendKey = ::sendRemoteKey,
                onAdbHost = { ui = ui.copy(adbHost = it) },
                onAdbPort = { ui = ui.copy(adbPort = it.filter(Char::isDigit).take(5)) },
                onAdbConnect = ::connectAdb,
                onAdbCommand = { ui = ui.copy(adbCommand = it) },
                onRunAdbCommand = ::runAdbCommand,
                onEditFavorite = ::editFavorite,
                onRunFavorite = { index -> ui.favorites.getOrNull(index)?.command?.let(::runAdbCommand) },
                onRefreshApps = ::refreshApps,
                onLaunchApp = ::launchApp,
                onUninstallApp = ::uninstallApp,
                onSendFile = ::sendFile,
                onRefreshScreen = ::refreshScreen,
                onScreenLiveChange = { ui = ui.copy(screenLive = it) },
                onPip = ::enterPictureInPicture,
                onWebsite = { startActivity(Intent(this, WebsiteActivity::class.java)) },
                onDismissWebsiteBanner = {
                    prefs.edit().putBoolean("website.banner.dismissed", true).apply()
                    ui = ui.copy(websiteBannerDismissed = true)
                },
                onTheme = ::changeTheme,
                onLanguage = ::changeLanguage,
                onCustomJson = { sendCustomJson(ui.customJson) },
                onCustomJsonText = { ui = ui.copy(customJson = it) },
                onSelectProfile = ::selectProfile,
                onDeleteProfile = ::deleteProfile,
                onSaveMacro = ::saveMacro,
                onDeleteMacro = ::deleteMacro,
                onRunMacro = ::runMacro,
                onCastUrl = ::castUrl
            )
        }
        ui.paired?.takeIf { it.protocol == ReceiverDevice.ANDROID_TV_REMOTE }?.let { connectSavedAndroidTv(
it.host) }
    }

    override fun onDestroy() {
        androidTvRemote.close()
        super.onDestroy()
    }

    override fun onUserLeaveHint() {
        super.onUserLeaveHint()
        if (ui.tab in setOf("screen", "media") && ui.screenPreview != null) enterPictureInPicture()
    }

    override fun onPictureInPictureModeChanged(isInPictureInPictureMode: Boolean, newConfig: Configuration)
{
        super.onPictureInPictureModeChanged(isInPictureInPictureMode, newConfig)
        ui = ui.copy(pictureInPicture = isInPictureInPictureMode)
    }

    private fun loadSavedState(): ControllerState {
        val host = prefs.getString("pair.host", "").orEmpty()
        val code = prefs.getString("pair.code", "").orEmpty()
        val savedName = prefs.getString("pair.name", "").orEmpty()
        val port = prefs.getInt("pair.port", LanClient.COMMAND_PORT)
        val protocol = prefs.getString("pair.protocol", ReceiverDevice.RECEIVER_CONTROL).orEmpty()
        val language = prefs.getString("language", "ar").orEmpty()
        val profiles = runCatching { gson.fromJson(prefs.getString("profiles", null), Array<SavedReceiver>::
class.java)?.toList() }
            .getOrNull().orEmpty().map { it.copy(protocol = it.protocol.ifBlank { ReceiverDevice.
RECEIVER_CONTROL }) }
        val macros = runCatching { gson.fromJson(prefs.getString("macros", null), Array<RemoteMacro>::class.
java)?.toList() }.getOrNull().orEmpty()
        val savedFavorites = runCatching {
            gson.fromJson(prefs.getString("adb.favorites", null), Array<FavoriteCommand>::class.java)?.
toList()
```

```
        }.getOrNull()?.takeIf { it.size == 10 } ?: defaultFavorites(language)
        return ControllerState(
            theme = prefs.getString("theme", "purple").orEmpty(),
            language = language,
            paired = if (host.isNotBlank() && (protocol == ReceiverDevice.ANDROID_TV_REMOTE || code.length
== 7)) {
                ReceiverDevice(savedName.ifBlank { "\u0631\u0633\u064A\u0641\u0631 \u0645\u062D\u0641\u0648\
u0638" }, host, if (protocol == ReceiverDevice.ANDROID_TV_REMOTE) 6466 else port, protocol)
            } else null,
            pairCode = if (protocol == ReceiverDevice.ANDROID_TV_REMOTE) "" else code,
            websiteBannerDismissed = prefs.getBoolean("website.banner.dismissed", false),
            profiles = profiles,
            macros = macros,
            adbHost = prefs.getString("adb.host", "").orEmpty(),
            adbPort = prefs.getInt("adb.port", 5555).toString(),
            favorites = savedFavorites
        )
    }

    private fun selectDiscoveredDevice(device: ReceiverDevice) {
        ui = ui.copy(
            pairingHost = device.host,
            pairingProtocol = device.protocol,
            pairingRequested = true,
            pairingCodeInput = "",
            androidTvPairingStarted = false,
            androidTvAwaitingCode = false,
            status = if (device.protocol == ReceiverDevice.ANDROID_TV_REMOTE) {
                loc(ui.language, "\u062A\u0645 \u0627\u062E\u062A\u064A\u0627\u0631 ${device.name}. \u0627\
u0628\u062F\u0623 \u0627\u0642\u062A\u0631\u0627\u0646 Android TV \u0627\u0644\u0645\u0628\u0627\u0634\
u0631.", "Selected ${device.name}. Start Android TV pairing.")
            } else {
                loc(ui.language, "\u0623\u062F\u062E\u0644 \u0643\u0648\u062F \u062A\u0637\u0628\u064A\
u0642 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0630\u064A 7 \u0623\u0631\u0642\u0627\u0645.", "Enter
the receiver app's 7-digit code.")
            },
            success = null
        )
    }

    private fun changePairingProtocol(protocol: String) {
        if (protocol !in setOf(ReceiverDevice.RECEIVER_CONTROL, ReceiverDevice.ANDROID_TV_REMOTE)) return
        ui = ui.copy(
            pairingProtocol = protocol,
            pairingCodeInput = "",
            androidTvPairingStarted = false,
            androidTvAwaitingCode = false,
            status = loc(ui.language, "\u0627\u062E\u062A\u0631 \u0627\u0644\u062C\u0647\u0627\u0632 \u0648\
u0623\u0643\u0645\u0644 \u062E\u0637\u0648\u0627\u062A \u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646 \
u0627\u0644\u0638\u0627\u0647\u0631\u0629.", "Select a device and follow its pairing steps."),
            success = null
        )
    }

    private fun startAndroidTvPairing(host: String) {
        val cleanHost = host.trim()
        if (cleanHost.isBlank()) {
            ui = ui.copy(status = loc(ui.language, "\u0623\u062F\u062E\u0644 \u0639\u0646\u0648\u0627\u0646
IP \u0644\u0644\u062A\u0644\u0641\u0627\u0632 \u0623\u0648 \u0627\u062E\u062A\u0631 \u062C\u0647\u0627\
u0632\u0627\u064B \u0645\u0643\u062A\u0634\u0641\u0627\u064B.", "Enter the TV's IP address or choose a
discovered device."), success = false)
            return
        }
        ui = ui.copy(
            pairingHost = cleanHost,
            pairingProtocol = ReceiverDevice.ANDROID_TV_REMOTE,
            pairingCodeInput = "",
            androidTvPairingStarted = true,
            androidTvAwaitingCode = false,
            status = loc(ui.language, "\u062C\u0627\u0631\u064D \u0628\u062F\u0621 \u0627\u0644\u0627\u0642\
u062A\u0631\u0627\u0646. \u0648\u0627\u0641\u0642 \u0639\u0644\u0649 \u0627\u0644\u0637\u0644\u0628 \u0641\
u064A \u0627\u0644\u062A\u0644\u0641\u0627\u0632 \u0648\u0627\u0646\u062A\u0638\u0631 \u0627\u0644\u0631\
u0645\u0632.", "Starting pairing. Approve the request on the TV and wait for its code."),
            success = null
        )
        androidTvRemote.startPairing(cleanHost, androidTvListener(cleanHost))
    }

    private fun submitAndroidTvPairingCode(code: String) {
        val cleanCode = code.trim().uppercase()
        if (!cleanCode.matches(Regex("[0-9A-F]{6}"))) {
            ui = ui.copy(status = loc(ui.language, "\u0623\u062F\u062E\u0644 \u0631\u0645\u0632 \u0627\
u0644\u062A\u0644\u0641\u0627\u0632 \u0627\u0644\u0645\u0643\u0648\u0651\u0646 \u0645\u0646 6 \u062E\u0627\
u0646\u0627\u062A (0-9 \u0623\u0648 A-F).", "Enter the TV's 6-character code (0-9 or A-F)."), success =
false)
            return
        }
        try {
            androidTvRemote.providePairingCode(cleanCode)
            ui = ui.copy(androidTvAwaitingCode = false, status = loc(ui.language, "\u062A\u0645 \u0625\
u0631\u0633\u0627\u0644 \u0627\u0644\u0631\u0645\u0632\u060C \u062C\u0627\u0631\u064D \u0627\u0644\u062A\
u062D\u0642\u0642 \u0648\u0627\u0644\u0627\u062A\u0635\u0627\u0644\u2026", "Code sent. Verifying and
connecting\u2026"), success = null)
        } catch (error: Exception) {
            ui = ui.copy(androidTvPairingStarted = false, androidTvAwaitingCode = false, status = error.
```

```
message ?: loc(ui.language, "\u062A\u0639\u0630\u0631 \u0625\u0631\u0633\u0627\u0644 \u0631\u0645\u0632 \
u0627\u0644\u062A\u0644\u0641\u0627\u0632.", "Could not submit the TV code."), success = false)
        }
    }

    private fun connectSavedAndroidTv(host: String) {
        ui = ui.copy(status = loc(ui.language, "\u062C\u0627\u0631\u064D \u0625\u0639\u0627\u062F\u0629 \
u0627\u0644\u0627\u062A\u0635\u0627\u0644 \u0628\u0627\u0644\u062A\u0644\u0641\u0627\u0632 \u0627\u0644\
u0645\u062D\u0641\u0648\u0638\u0629", "Reconnecting to the saved TV\u2026"), success = null)
        androidTvRemote.reconnect(host, androidTvListener(host))
    }

    private fun androidTvListener(host: String) = object : AndroidTvRemoteClient.Listener {
        override fun onPairingCodeRequested() = runOnUiThread {
            if (ui.pairingHost == host) {
                ui = ui.copy(
                    androidTvAwaitingCode = true,
                    status = loc(ui.language, "\u0627\u0641\u062A\u062D \u0634\u0627\u0634\u0629 \u0627\
u0644\u062A\u0644\u0641\u0627\u0632\u060C \u0648\u0627\u0641\u0642 \u0639\u0644\u0649 \u0627\u0644\u0627\
u0642\u062A\u0631\u0627\u0646\u060C \u062B\u0645 \u0623\u062F\u062E\u0644 \u0627\u0644\u0631\u0645\u0632 \
u0630\u064A 6 \u062E\u0627\u0646\u0627\u062A.", "Approve pairing on the TV, then enter the 6-character code
shown there."),
                    success = null
                )
            }
        }

        override fun onPairingAccepted() = runOnUiThread {
            ui = ui.copy(status = loc(ui.language, "\u0642\u064E\u0628\u0650\u0644 \u0627\u0644\u062A\u0644\
u0641\u0627\u0632 \u0627\u0644\u0631\u0645\u0632. \u062C\u0627\u0631\u064D \u0641\u062A\u062D \u0642\u0646\
u0627\u0629 \u0627\u0644\u062A\u062D\u0643\u0645\u2026", "The TV accepted the code. Opening the remote
session\u2026"), success = null)
        }

        override fun onConnected() = runOnUiThread { saveAndroidTvConnection(host) }

        override fun onDisconnected() = runOnUiThread {
            if (ui.paired?.host == host && ui.paired?.protocol == ReceiverDevice.ANDROID_TV_REMOTE) {
                ui = ui.copy(status = loc(ui.language, "\u0627\u0646\u0642\u0637\u0639 \u0627\u062A\u0635\
u0627\u0644 \u0627\u0644\u062A\u0644\u0641\u0627\u0632. \u0623\u0639\u062F \u0627\u0644\u0627\u062A\u0635\
u0627\u0644 \u0623\u0648 \u0623\u0639\u062F \u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646.", "The TV
connection dropped. Reconnect or pair again."), success = false)
            }
        }

        override fun onError(message: String) = runOnUiThread {
            ui = ui.copy(
                androidTvPairingStarted = false,
                androidTvAwaitingCode = false,
                status = message.ifBlank { loc(ui.language, "\u062A\u0639\u0630\u0631 \u0627\u0644\u0627\
u062A\u0635\u0627\u0644 \u0628\u0627\u0644\u062A\u0644\u0641\u0627\u0632. \u062A\u062D\u0642\u0642 \u0645\
u0646 \u0627\u0644\u0634\u0628\u0643\u0629 \u0648\u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646.", "Could
not connect to the TV. Check the network and pairing.") },
                success = false
            )
        }
    }

    private fun saveAndroidTvConnection(host: String) {
        val found = ui.devices.firstOrNull { it.host == host && it.protocol == ReceiverDevice.
ANDROID_TV_REMOTE }
        val device = ReceiverDevice(found?.name ?: "Android TV ($host)", host, 6466, ReceiverDevice.
ANDROID_TV_REMOTE)
        val saved = SavedReceiver(device.name, host, 6466, "", ReceiverDevice.ANDROID_TV_REMOTE)
        val profiles = (listOf(saved) + ui.profiles.filterNot { it.host == host && it.protocol ==
ReceiverDevice.ANDROID_TV_REMOTE }).take(20)
        prefs.edit().putString("pair.host", host).putString("pair.code", "").putString("pair.name", device.
name)
            .putInt("pair.port", 6466).putString("pair.protocol", ReceiverDevice.ANDROID_TV_REMOTE)
            .putString("profiles", gson.toJson(profiles)).apply()
        ui = ui.copy(
            paired = device,
            pairCode = "",
            profiles = profiles,
            pairingRequested = false,
            androidTvPairingStarted = false,
            androidTvAwaitingCode = false,
            status = loc(ui.language, "\u062A\u0645 \u0627\u0644\u0627\u062A\u0635\u0627\u0644 \u0628\u0640
${device.name} \u0639\u0628\u0631 Android TV Remote.", "Connected to ${device.name} with Android TV Remote.
"),
            success = true
        )
    }

    private fun discoverDevices() {
        if (ui.discovering) return
        ui = ui.copy(discovering = true, status = loc(ui.language, "\u062C\u0627\u0631\u064D \u0627\u0644\
u0628\u062D\u062B \u0639\u0646 Android TV \u0648\u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u064A\
u064A\u0641\u0631 \u0639\u0644\u0649 \u0634\u0628\u0643\u0629 Wi-Fi\u2026", "Searching Wi-Fi for Android
TVs and receiver companion apps\u2026"), success = null)
        lifecycleScope.launch {
            try {
                val devices = LanClient.discover(this@MainActivity)
                ui = ui.copy(
                    discovering = false,
```

```
                    devices = devices,
                    status = if (devices.isEmpty()) {
                        loc(ui.language, "\u0644\u0645 \u064A\u064F\u0639\u062B\u0631 \u0639\u0644\u0649 \
u062E\u062F\u0645\u0629 Android TV Remote \u0623\u0648 \u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\
u0633\u064A\u0641\u0631. \u062A\u0623\u0643\u062F \u0645\u0646 Wi-Fi \u0646\u0641\u0633\u0647\u0627\u061B \
u0628\u0639\u0636 \u0627\u0644\u0623\u062C\u0647\u0632\u0629 \u0644\u0627 \u062A\u0639\u0644\u0646 \u062E\
u062F\u0645\u0629 \u0627\u0644\u0631\u064A\u0645\u0648\u062A.", "No Android TV Remote service or companion
app found. Check the same Wi-Fi; some devices do not advertise the remote service.")
                    } else {
                        val nativeCount = devices.count { it.protocol == ReceiverDevice.ANDROID_TV_REMOTE }
                        val companionCount = devices.size - nativeCount
                        loc(ui.language, "\u0639\u064F\u062B\u0631 \u0639\u0644\u0649 $nativeCount \u062A\
u0644\u0641\u0627\u0632 Android TV \u0648$companionCount \u062A\u0637\u0628\u064A\u0642 \u0631\u0633\u064A\
u0641\u0631. \u0627\u062E\u062A\u0631 \u0645\u0633\u0627\u0631 \u0627\u0644\u0627\u062A\u0635\u0627\u0644 \
u0627\u0644\u0645\u0646\u0627\u0633\u0628.", "Found $nativeCount Android TV device(s) and $companionCount
companion app(s). Choose the matching connection path.")
                    },
                    success = devices.isNotEmpty()
                )
            } catch (e: Exception) {
                ui = ui.copy(discovering = false, status = friendlyError(e), success = false)
            }
        }
    }

    private fun pairDevice(host: String, code: String) {
        if (ui.pairingProtocol == ReceiverDevice.ANDROID_TV_REMOTE) {
            submitAndroidTvPairingCode(code)
            return
        }
        val cleanHost = host.trim()
        if (cleanHost.isBlank() || !code.matches(Regex("\\d{7}"))) {
            ui = ui.copy(status = loc(ui.language, "\u0623\u062F\u062E\u0644 \u0639\u0646\u0648\u0627\u0646
IP \u0648\u0643\u0648\u062F\u0627\u064B \u0645\u0646 7 \u0623\u0631\u0642\u0627\u0645.", "Enter an IP
address and a 7-digit code."), success = false)
            return
        }
        val known = ui.devices.firstOrNull { it.host == cleanHost && it.protocol == ReceiverDevice.
RECEIVER_CONTROL }
            ?: ReceiverDevice("\u0631\u0633\u064A\u0641\u0631 $cleanHost", cleanHost)
        ui = ui.copy(status = loc(ui.language, "\u062C\u0627\u0631\u064D \u0627\u0644\u062A\u062D\u0642\
u0642 \u0645\u0646 \u0643\u0648\u062F \u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646\u2026", "Checking
pairing code\u2026"), success = null)
        lifecycleScope.launch {
            try {
                val reply = LanClient.pair(known, code)
                if (!reply.ok) throw IllegalStateException(reply.message.ifBlank { loc(ui.language, "\u0631\
u0641\u0636 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646.",
"The receiver rejected pairing.") })
                val named = known.copy(name = reply.raw.get("name")?.asString ?: known.name)
                val saved = SavedReceiver(named.name, named.host, named.port, code, ReceiverDevice.
RECEIVER_CONTROL)
                val profiles = (listOf(saved) + ui.profiles.filterNot { it.host == saved.host && it.
protocol == saved.protocol }).take(20)
                prefs.edit().putString("pair.host", named.host).putString("pair.code", code)
                    .putString("pair.name", named.name).putInt("pair.port", named.port).putString("pair.
protocol", ReceiverDevice.RECEIVER_CONTROL)
                    .putString("profiles", gson.toJson(profiles)).apply()
            ui = ui.copy(paired = named.copy(protocol = ReceiverDevice.RECEIVER_CONTROL), pairCode = code,
profiles = profiles, pairingHost = "", pairingRequested = false, pairingCodeInput = "", status = loc(ui.
language, "\u062A\u0645 \u0627\u0644\u0627\u0642\u062A\u0631\u0627\u0646 \u0645\u0639 ${named.name} \u0628\
u0646\u062C\u0627\u062D.", "Paired with ${named.name}."), success = true)
            } catch (e: Exception) {
                ui = ui.copy(status = friendlyError(e), success = false)
            }
        }
    }

    private fun handleQrScan(value: String) {
        try {
            val qr = JsonParser.parseString(value).asJsonObject
            require(qr.get("type")?.asString == "ReceiverControlPairing") { loc(ui.language, "\u0631\u0645\
u0632 QR \u0644\u064A\u0633 \u0631\u0645\u0632 \u0627\u0642\u062A\u0631\u0627\u0646 ReceiverControl", "This
QR code is not a ReceiverControl pairing code.") }
            val host = qr.get("ip")?.asString.orEmpty()
            require(host.matches(Regex("(?:[0-9]{1,3}\\.){3}[0-9]{1,3}"))) { loc(ui.language, "\u0639\u0646\
u0648\u0627\u0646 IP \u063A\u064A\u0631 \u0635\u0627\u0644\u062D \u0641\u064A \u0627\u0644\u0631\u0645\
u0632", "The QR code contains an invalid IP address.") }
            val port = qr.get("port")?.asInt ?: LanClient.COMMAND_PORT
            require(port == LanClient.COMMAND_PORT) { loc(ui.language, "\u0645\u0646\u0641\u0630 \u0627\
u0644\u062E\u062F\u0645\u0629 \u063A\u064A\u0631 \u0645\u062F\u0639\u0648\u0645", "The service port is not
supported.") }
            val code = qr.get("code")?.asString.orEmpty()
            require(code.matches(Regex("\\d{7}"))) { loc(ui.language, "\u0643\u0648\u062F \u0627\u0644\
u0627\u0642\u062A\u0631\u0627\u0646 \u0641\u064A \u0627\u0644\u0631\u0645\u0632 \u063A\u064A\u0631 \u0635\
u0627\u0644\u062D", "The QR code contains an invalid pairing code.") }
            val name = qr.get("name")?.asString ?: "Android Receiver"
            val device = ReceiverDevice(name, host, port, ReceiverDevice.RECEIVER_CONTROL)
            ui = ui.copy(showQrScanner = false, devices = (listOf(device) + ui.devices.filterNot { it.host
== host }))
            pairDevice(host, code)
        } catch (e: Exception) {
            ui = ui.copy(showQrScanner = false, status = e.message ?: loc(ui.language, "\u062A\u0639\u0630\
u0631 \u0642\u0631\u0627\u0621\u0629 \u0631\u0645\u0632 \u0627\u0644\u0627\u0642\u062A\u0631\u0646.",
"Could not read pairing code."), success = false)
```

```
        }
    }

    private fun sendRemoteKey(key: String) {
        lifecycleScope.launch { runCatching { sendRemoteKeyNow(key) }.onFailure { ui = ui.copy(status =
friendlyError(it), success = false) } }
    }

    private suspend fun sendRemoteKeyNow(key: String) {
        val host = ui.adbHost.trim()
        if (host.isNotBlank()) {
            withContext(Dispatchers.IO) { adb.shell(host, ui.adbPort.toIntOrNull() ?: 5555, keyToAdbCommand(
key)) }
            ui = ui.copy(adbConnected = true, status = loc(ui.language, "\u062A\u0645 \u0625\u0631\u0633\
u0627\u0644 \u0627\u0644\u0623\u0645\u0631.", "Command sent."), success = true)
            return
        }
        val receiver = ui.paired
        if (receiver == null) throw IllegalStateException(loc(ui.language, "\u0627\u0631\u0628\u0637 \u0627\
u0644\u0631\u0633\u064A\u0641\u0631 \u0623\u0648 \u0641\u0639\u0651\u0644 ADB \u0627\u0644\u0634\u0628\
u0643\u064A \u0623\u0648\u0644\u0627\u064B.", "Connect to a receiver or enable network ADB first."))
        if (receiver.protocol == ReceiverDevice.ANDROID_TV_REMOTE) {
            withContext(Dispatchers.IO) { androidTvRemote.sendKey(receiver.host, key) }
            ui = ui.copy(status = loc(ui.language, "\u062A\u0645 \u0625\u0631\u0633\u0627\u0644 \u0627\
u0644\u0623\u0645\u0631 \u0625\u0644\u0649 Android TV.", "Command sent to Android TV."), success = true)
            return
        }
        if (ui.pairCode.length != 7) throw IllegalStateException(loc(ui.language, "\u0623\u0639\u062F \
u0627\u0642\u062A\u0631\u0627\u0646 \u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\u0641\
u0631 \u0628\u0643\u0648\u062F \u0645\u0646 7 \u0623\u0631\u0642\u0627\u0645.", "Pair the receiver app
again with its 7-digit code."))
        val reply = LanClient.command(receiver, ui.pairCode, "KEY", key)
        if (!reply.ok) throw IllegalStateException(reply.message.ifBlank { loc(ui.language, "\u0631\u0641\
u0636 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0627\u0644\u0623\u0645\u0631.", "The receiver rejected
the command.") })
        ui = ui.copy(status = reply.message.ifBlank { loc(ui.language, "\u062A\u0645 \u0625\u0631\u0633\
u0627\u0644 \u0627\u0644\u0623\u0645\u0631.", "Command sent.") }, success = true)
    }

    private fun selectProfile(profile: SavedReceiver) {
        prefs.edit().putString("pair.host", profile.host).putString("pair.code", profile.code)
            .putString("pair.name", profile.name).putInt("pair.port", profile.port).putString("pair.
protocol", profile.protocol).apply()
        val device = ReceiverDevice(profile.name, profile.host, profile.port, profile.protocol)
        ui = ui.copy(paired = device, pairCode = profile.code, pairingProtocol = profile.protocol,
            status = loc(ui.language, "\u062A\u0645 \u0627\u062E\u062A\u064A\u0627\u0631 ${profile.name}. \
u062C\u0627\u0631\u064D \u0627\u0644\u0627\u062A\u0635\u0627\u0644\u2026", "Selected ${profile.name}.
Connecting\u2026"), success = null)
        if (profile.protocol == ReceiverDevice.ANDROID_TV_REMOTE) connectSavedAndroidTv(profile.host)
    }

    private fun deleteProfile(profile: SavedReceiver) {
        val profiles = ui.profiles.filterNot { it.host == profile.host && it.protocol == profile.protocol }
        if (ui.paired?.host == profile.host && ui.paired?.protocol == profile.protocol) {
            prefs.edit().remove("pair.host").remove("pair.code").remove("pair.name").remove("pair.
protocol").apply()
            ui = ui.copy(profiles = profiles, paired = null, pairCode = "", status = loc(ui.language, "\
u062D\u064F\u0630\u0641 \u0645\u0644\u0641 \u0627\u0644\u0627\u062A\u0635\u0627\u0644.", "Connection
profile removed."), success = true)
        } else ui = ui.copy(profiles = profiles)
        prefs.edit().putString("profiles", gson.toJson(profiles)).apply()
    }

    private fun saveMacro(id: String?, name: String, keys: List<String>) {
        val allowed = setOf("POWER", "HOME", "BACK", "UP", "DOWN", "LEFT", "RIGHT", "ENTER", "VOLUME_UP",
"VOLUME_DOWN", "MUTE", "RECENTS")
        val safeKeys = keys.filter { it in allowed }.take(20)
        if (name.isBlank() || safeKeys.isEmpty()) {
            ui = ui.copy(status = loc(ui.language, "\u0623\u062F\u062E\u0644 \u0627\u0633\u0645\u0627\u064B
\u0648\u062E\u0637\u0648\u0629 \u062A\u062D\u0643\u0645 \u0648\u0627\u062D\u062F\u0629 \u0639\u0644\u0649 \
u0627\u0644\u0623\u0642\u0644.", "Enter a name and at least one valid control step."), success = false)
            return
        }
        val macro = RemoteMacro(id ?: java.util.UUID.randomUUID().toString(), name.trim(), safeKeys)
        val macros = if (ui.macros.any { it.id == macro.id }) ui.macros.map { if (it.id == macro.id) macro
else it } else listOf(macro) + ui.macros
        prefs.edit().putString("macros", gson.toJson(macros)).apply()
        ui = ui.copy(macros = macros, status = loc(ui.language, "\u062A\u0645 \u062D\u0641\u0638 \u0627\
u0644\u0627\u062E\u062A\u0635\u0627\u0631.", "Macro saved."), success = true)
    }

    private fun deleteMacro(id: String) {
        val macros = ui.macros.filterNot { it.id == id }
        prefs.edit().putString("macros", gson.toJson(macros)).apply()
        ui = ui.copy(macros = macros, status = loc(ui.language, "\u062A\u0645 \u062D\u0630\u0641 \u0627\
u0644\u0627\u062E\u062A\u0635\u0627\u0631.", "Macro deleted."), success = true)
    }

    private fun runMacro(macro: RemoteMacro) {
        lifecycleScope.launch {
            try {
                ui = ui.copy(status = loc(ui.language, "\u062C\u0627\u0631\u064D \u062A\u0646\u0641\u064A\
u0630 ${macro.name}\u2026", "Running ${macro.name}\u2026"), success = null)
                macro.keys.forEach { key -> sendRemoteKeyNow(key); delay(600) }
                ui = ui.copy(status = loc(ui.language, "\u0627\u0643\u062A\u0645\u0644 ${macro.name}.", "${
```

```
macro.name} completed."), success = true)
            } catch (e: Exception) {
                ui = ui.copy(status = friendlyError(e), success = false)
            }
        }
    }

    private fun connectAdb() {
        val host = ui.adbHost.trim()
        val port = ui.adbPort.toIntOrNull()
        if (host.isBlank() || port == null || port !in 1..65535) {
            ui = ui.copy(status = "\u0623\u062F\u062E\u0644 \u0639\u0646\u0648\u0627\u0646 IP \u0648\u0645\
u0646\u0641\u0630\u0627\u064B \u0635\u0627\u0644\u062D\u0627\u064B.", success = false)
            return
        }
        ui = ui.copy(status = loc(ui.language, "\u062C\u0627\u0631\u064D \u0627\u0644\u0627\u062A\u0635\
u0627\u0644 \u0639\u0628\u0631 ADB TCP\u2026 \u0648\u0627\u0641\u0642 \u0639\u0644\u0649 \u0628\u0635\u0645\
u0629 \u0627\u0644\u0645\u0641\u062A\u0627\u062D \u0639\u0646\u062F \u0638\u0647\u0648\u0631\u0627\u0647 \
u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631.", "Connecting over ADB TCP\u2026 approve the
key prompt on the receiver."), success = null)
        lifecycleScope.launch {
            try {
                val model = withContext(Dispatchers.IO) { adb.shell(host, port, "getprop ro.product.model")
}
                prefs.edit().putString("adb.host", host).putInt("adb.port", port).apply()
                ui = ui.copy(adbConnected = true, status = loc(ui.language, "\u0627\u062A\u0635\u0627\u0644
ADB \u0646\u0627\u062C\u062D${model.takeIf(String::isNotBlank)?.let { ": $it" } ?: ""}.", "ADB connected${
model.takeIf(String::isNotBlank)?.let { ": $it" } ?: ""}."), success = true)
            } catch (e: Exception) {
                ui = ui.copy(adbConnected = false, status = friendlyError(e), success = false)
            }
        }
    }

    private fun changeTheme(theme: String) {
        ui = ui.copy(theme = theme)
        prefs.edit().putString("theme", theme).apply()
    }

    private fun changeLanguage(language: String) {
        ui = ui.copy(language = language, status = loc(language, "\u062A\u0645 \u062A\u063A\u064A\u064A\
u0631 \u0627\u0644\u0644\u063A\u0629 \u0625\u0644\u0649 \u0627\u0644\u0639\u0631\u0628\u064A\u0629.",
"Language changed to English."))
        prefs.edit().putString("language", language).apply()
    }

    private fun runAdbCommand(command: String) {
        val host = ui.adbHost.trim()
        val port = ui.adbPort.toIntOrNull() ?: 5555
        if (host.isBlank()) {
            ui = ui.copy(status = loc(ui.language, "\u0623\u062F\u062E\u0644 IP \u0627\u0644\u0631\u0633\
u064A\u0641\u0631 \u0648\u0627\u062A\u0635\u0644 \u0639\u0628\u0631 ADB \u0623\u0648\u0644\u064B.",
"Enter the receiver IP and connect ADB first."), success = false)
            return
        }
        if (command.isBlank()) return
        ui = ui.copy(adbCommand = command, status = loc(ui.language, "\u062C\u0627\u0631\u064D \u062A\u0646\
u0641\u064A\u0630 \u0627\u0644\u0623\u0645\u0631\u2026", "Running command\u2026"), success = null)
        lifecycleScope.launch {
            try {
                val result = withContext(Dispatchers.IO) { adb.shell(host, port, command) }
                ui = ui.copy(adbConnected = true, status = result.ifBlank { loc(ui.language, "\u0627\u0643\
u062A\u0645\u0644 \u0627\u0644\u0623\u0645\u0631.", "Command completed.") }, success = true)
            } catch (e: Exception) {
                ui = ui.copy(adbConnected = false, status = friendlyError(e), success = false)
            }
        }
    }

    private fun editFavorite(index: Int, name: String, command: String) {
        if (index !in ui.favorites.indices) return
        val updated = ui.favorites.toMutableList().also { it[index] = FavoriteCommand(name, command) }
        ui = ui.copy(favorites = updated)
        prefs.edit().putString("adb.favorites", gson.toJson(updated)).apply()
    }

    private fun refreshApps() {
        if (ui.loadingApps) return
        ui = ui.copy(loadingApps = true, status = loc(ui.language, "\u062C\u0627\u0631\u064D \u0642\u0631\
u0627\u0621\u0629 \u0627\u0644\u062A\u0637\u0628\u064A\u0642\u0627\u062A \u0627\u0644\u0645\u062B\u0628\
u062A\u0629\u2026", "Loading installed apps\u2026"), success = null)
        lifecycleScope.launch {
            try {
                val apps = if (ui.adbHost.isNotBlank()) {
                    val output = withContext(Dispatchers.IO) { adb.shell(ui.adbHost, ui.adbPort.toIntOrNull(
) ?: 5555, "pm list packages --user 0") }
                    output.lineSequence().mapNotNull { it.trim().removePrefix("package:").takeIf(String::
isNotBlank) }
                        .map { ReceiverApp(it.substringAfterLast('.'), it) }.toList()
                } else {
                    val receiver = ui.paired ?: throw IllegalStateException(loc(ui.language, "\u0627\u0631\
u0628\u0637 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0628\u0643\u0648\u062F \u0623\u0648 \u0627\u062A\
u0635\u0644 \u0639\u0628\u0631 ADB \u0623\u0648\u0644\u064B", "Pair the receiver or connect through
ADB first."))
                    if (receiver.protocol == ReceiverDevice.ANDROID_TV_REMOTE) throw IllegalStateException(
```

```
loc(ui.language, "\u0639\u0631\u0636 \u0627\u0644\u062A\u0637\u0628\u064A\u0642\u0627\u062A \u064A\u062D\
u062A\u0627\u062C \u062A\u0637\u0628\u064A\u0642 Android TV responsiveness \u0623\u0648 ADB. \u0627\u062A\
u0635\u0627\u0644 Android TV \u0627\u0644\u0645\u0628\u0627\u0634\u0631 \u064A\u062F\u0639\u0645 \u0623\
u0632\u0631\u0627\u0631 \u0627\u0644\u0631\u064A\u0645\u0648\u062A.", "App management needs Android TV
responsiveness or ADB. Direct Android TV pairing supports remote buttons."))
                    val reply = LanClient.command(receiver, ui.pairCode, "APPS")
                    if (!reply.ok) throw IllegalStateException(reply.message)
                    reply.raw.getAsJsonArray("apps").map { it.asJsonObject }.map {
                        ReceiverApp(it.get("name").asString, it.get("package").asString)
                    }
                }
                ui = ui.copy(apps = apps, loadingApps = false, status = loc(ui.language, "\u062A\u0645 \
u062C\u0644\u0628 ${apps.size} \u062A\u0637\u0628\u064A\u0642\u0627\u064B.", "Loaded ${apps.size} app(s)."),
 success = true)
            } catch (e: Exception) {
                ui = ui.copy(loadingApps = false, status = friendlyError(e), success = false)
            }
        }
    }

    private fun launchApp(app: ReceiverApp) {
        if (!app.packageName.matches(Regex("[A-Za-z0-9_]+(\\.[A-Za-z0-9_]+)+"))) return
        if (ui.adbHost.isNotBlank()) runAdbCommand("monkey -p ${app.packageName} 1")
        else bridgeAction("LAUNCH", app.packageName)
    }

    private fun uninstallApp(app: ReceiverApp) {
        if (!app.packageName.matches(Regex("[A-Za-z0-9_]+(\\.[A-Za-z0-9_]+)+"))) return
        if (ui.adbHost.isNotBlank()) runAdbCommand("pm uninstall ${app.packageName}")
        else bridgeAction("UNINSTALL", app.packageName)
    }

    private fun bridgeAction(action: String, value: String) {
        val receiver = ui.paired
        if (receiver == null) {
            ui = ui.copy(status = loc(ui.language, "\u0644\u0627 \u064A\u0648\u062C\u062F \u0627\u062A\
u0635\u0627\u0644 \u0628\u0627\u0644\u0631\u0633\u064A\u0641\u0631.", "No receiver connection."), success =
false)
            return
        }
        if (receiver.protocol == ReceiverDevice.ANDROID_TV_REMOTE && ui.adbHost.isBlank()) {
            ui = ui.copy(status = loc(ui.language, "\u0647\u0630\u0627 \u0627\u0644\u0623\u0645\u0631 \
u064A\u062D\u062A\u0627\u062C \u062A\u0637\u0628\u064A\u0642 Android TV responsiveness \u0639\u0644\u0649 \
u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0623\u0648 \u0627\u062A\u0635\u0627\u0644 ADB. \u0627\u0644\
u0631\u064A\u0645\u0648\u062A \u0627\u0644\u0623\u0633\u0627\u0633\u064A \u064A\u0639\u0645\u0644 \u0628\
u0627\u0644\u0627\u062A\u0635\u0627\u0644 \u0627\u0644\u0644\u0645\u0628\u0627\u0634\u0631.", "This feature needs
Android TV responsiveness on the receiver or ADB. Basic remote buttons work through direct pairing."),
success = false)
            return
        }
        lifecycleScope.launch {
            try {
                val reply = LanClient.command(receiver, ui.pairCode, action, value)
                ui = ui.copy(status = reply.message, success = reply.ok)
            } catch (e: Exception) {
                ui = ui.copy(status = friendlyError(e), success = false)
            }
        }
    }

    private fun sendCustomJson(text: String) {
        val receiver = ui.paired
        if (receiver == null) {
            ui = ui.copy(status = loc(ui.language, "\u0627\u0642\u0631\u0646 \u0627\u0644\u0631\u0633\u064A\
u0641\u0631 \u0623\u0648\u0644\u0627\u064B \u0644\u0625\u0631\u0633\u0627\u0644 JSON.", "Pair a receiver
before sending JSON."), success = false)
            return
        }
        if (receiver.protocol == ReceiverDevice.ANDROID_TV_REMOTE && ui.adbHost.isBlank()) {
            ui = ui.copy(status = loc(ui.language, "\u0625\u0631\u0633\u0627\u0644 JSON \u064A\u062D\u062A\
u0627\u062C \u062A\u0637\u0628\u064A\u0642 Android TV responsiveness \u0623\u0648 \u0627\u062A\u0635\u0627\
u0644 ADB.", "Sending JSON needs Android TV responsiveness or ADB."), success = false)
            return
        }
        lifecycleScope.launch {
            try {
                val response = LanClient.custom(receiver, ui.pairCode, text)
                ui = ui.copy(status = response.message.ifBlank { response.raw.toString() }, success =
response.ok)
            } catch (e: Exception) {
                ui = ui.copy(status = friendlyError(e), success = false)
            }
        }
    }

    private fun sendFile(uri: Uri) {
        val resolver = contentResolver
        val name = queryDisplayName(resolver, uri) ?: "media_${System.currentTimeMillis()}"
        val mime = resolver.getType(uri) ?: "application/octet-stream"
        val size = querySize(resolver, uri)
        if (size < 0) {
            ui = ui.copy(status = loc(ui.language, "\u062A\u0639\u0630\u0631 \u0645\u0639\u0631\u0641\u0629
\u062D\u062C\u0645 \u0627\u0644\u0645\u0644\u0641. \u0627\u062E\u062A\u0631 \u0627\u0644\u0644\u0645\u0644\u0641 \
u0645\u0646 \u0645\u062F\u064A\u0631 \u0627\u0644\u0645\u0644\u0641\u0627\u062A \u062B\u0645 \u0623\u0639\
u062F \u0627\u0644\u0645\u062D\u0627\u0648\u0644\u0629.", "Could not read file size. Choose it from a file
```

```
manager and try again."), success = false)
            return
        }
        val host = ui.adbHost.trim()
        val port = ui.adbPort.toIntOrNull() ?: 5555
        val receiver = ui.paired
        if (host.isBlank() && receiver == null) {
            ui = ui.copy(status = loc(ui.language, "\u0627\u0631\u0628\u0637 \u0627\u0644\u0631\u0633\u064A\
u0641\u0631 \u0623\u0648 \u0627\u062A\u0635\u0644 \u0639\u0628\u0631 ADB \u0642\u0628\u0644 \u0625\u0631\
u0633\u0627\u0644 \u0627\u0644\u0645\u0644\u0641\u0627\u062A.", "Pair the receiver or connect ADB before
sending files."), success = false)
            return
        }
        if (host.isBlank() && receiver?.protocol == ReceiverDevice.ANDROID_TV_REMOTE) {
            ui = ui.copy(status = loc(ui.language, "\u0625\u0631\u0633\u0627\u0644 \u0627\u0644\u0645\u0644\
u0641\u0627\u062A \u064A\u062D\u062A\u0627\u062C \u062A\u0637\u0628\u064A\u0642 Android TV responsiveness \
u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0623\u0648 \u0627\u062A\u0635\u0627\u0644
ADB.", "File transfer needs Android TV responsiveness on the receiver or ADB."), success = false)
            return
        }
        ui = ui.copy(status = loc(ui.language, "\u062C\u0627\u0631\u064D \u0625\u0631\u0633\u0627\u0644
$name (${formatBytes(size)})\u2026", "Sending $name (${formatBytes(size)})\u2026"), success = null)
        lifecycleScope.launch {
            try {
                if (host.isNotBlank()) {
                    withContext(Dispatchers.IO) { resolver.openInputStream(uri)?.use { adb.push(host, port,
name, it, size) } ?: error("\u062A\u0639\u0630\u0631 \u0641\u062A\u062D \u0627\u0644\u0645\u0644\u0641") }
                    ui = ui.copy(status = loc(ui.language, "\u062A\u0645 \u0646\u0633\u062E $name \u0625\
u0644\u0649 \u0645\u062C\u0644\u062F Download \u0641\u064A \u0627\u0644\u0631\u0633\u064A\u0641\u0631.",
"Copied $name to the receiver's Download folder."), success = true)
                } else {
                    val reply = LanClient.sendFile(receiver!!, ui.pairCode, name, mime, size) { resolver.
openInputStream(uri) }
                    if (!reply.ok) throw IllegalStateException(reply.message)
                    ui = ui.copy(status = reply.message.ifBlank { loc(ui.language, "\u062A\u0645 \u0625\
u0631\u0633\u0627\u0644 \u0627\u0644\u0645\u0644\u0641 \u0625\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\
u0641\u0631.", "File sent to receiver.") }, success = true)
                }
            } catch (e: Exception) {
                ui = ui.copy(status = friendlyError(e), success = false)
            }
        }
    }

    private fun castUrl(rawUrl: String) {
        val uri = runCatching { Uri.parse(rawUrl.trim()) }.getOrNull()
        if (uri == null || uri.scheme !in setOf("http", "https") || uri.host.isNullOrBlank()) {
            ui = ui.copy(status = loc(ui.language, "\u0623\u062F\u062E\u0644 \u0631\u0627\u0628\u0637 HTTP \
u0623\u0648 HTTPS \u0635\u0627\u0644\u062D\u0627\u064B.", "Enter a valid HTTP or HTTPS URL."), success =
false)
            return
        }
        if (ui.adbHost.isNotBlank()) runAdbCommand("am start -a android.intent.action.VIEW -d ${shellQuote(
uri.toString())}")
        else bridgeAction("OPEN_URL", uri.toString())
    }

    private fun shellQuote(value: String): String = "'" + value.replace("'", "'\\''") + "'"

    private fun refreshScreen() {
        if (ui.screenBusy) return
        val receiver = ui.paired
        val code = ui.pairCode
        if (receiver?.protocol == ReceiverDevice.ANDROID_TV_REMOTE) {
            ui = ui.copy(status = loc(ui.language, "\u0639\u0631\u0636 \u0627\u0644\u0634\u0627\u0634\u0629
\u064A\u062D\u062A\u0627\u062C \u062A\u0637\u0628\u064A\u0642 Android TV responsiveness \u0623\u0648 ADB\
u061B \u0627\u0644\u0627\u062A\u0635\u0627\u0644 \u0627\u0644\u0645\u0628\u0627\u0634\u0631 \u064A\u062F\
u0639\u0645 \u0627\u0644\u0631\u064A\u0645\u0648\u062A \u0641\u0642\u0637 \u0627\u0644\u0645\u0639\u062F.", "Screen viewing needs Android
TV responsiveness or ADB; direct pairing supports remote control only."), success = false)
            return
        }
        if (receiver == null || code.length != 7) {
            ui = ui.copy(status = loc(ui.language, "\u0627\u0631\u0628\u0637 \u062A\u0637\u0628\u064A\u0642
\u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0623\u0648\u0644\u064B \u0644\u0639\u0631\u0636 \u0627\
u0644\u0634\u0627\u0634\u0629 \u0628\u062F\u0648\u0646 ADB.", "Pair the receiver app first to view its
screen without ADB."), success = false)
            return
        }
        ui = ui.copy(screenBusy = true)
        lifecycleScope.launch {
            try {
                val response = LanClient.command(receiver, code, "SCREENSHOT")
                if (!response.ok) throw IllegalStateException(response.message)
                val encoded = response.raw.get("image")?.asString
                    ?: throw IOException("\u0644\u0645 \u062A\u0635\u0644 \u0635\u0648\u0631\u0629 \u0645\
u0646 \u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\u0641\u0631")
                val bytes = Base64.decode(encoded, Base64.DEFAULT)
                val bitmap = withContext(Dispatchers.Default) { decodeScreenPreview(bytes) }
                    ?: throw IOException("\u062A\u0639\u0630\u0631 \u0641\u0643 \u0635\u0648\u0631\u0629 \
u0634\u0627\u0634\u0629 \u0627\u0644\u0631\u0633\u064A\u0641\u0631")
                ui = ui.copy(screenPreview = bitmap, screenBusy = false, success = true)
            } catch (e: Exception) {
                ui = ui.copy(screenBusy = false, status = friendlyError(e), success = false)
            }
        }
```

```
    }

    private fun decodeScreenPreview(bytes: ByteArray): Bitmap? {
        val bounds = BitmapFactory.Options().apply { inJustDecodeBounds = true }
        BitmapFactory.decodeByteArray(bytes, 0, bytes.size, bounds)
        if (bounds.outWidth <= 0 || bounds.outHeight <= 0) return null

        val options = BitmapFactory.Options()
        var sampleSize = 1
        while (bounds.outWidth / sampleSize > 1280 || bounds.outHeight / sampleSize > 720) {
            sampleSize *= 2
        }
        options.inSampleSize = sampleSize
        return BitmapFactory.decodeByteArray(bytes, 0, bytes.size, options)
    }

    private fun enterPictureInPicture() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O && ui.screenPreview != null) {
            runCatching { enterPictureInPictureMode(PictureInPictureParams.Builder().setAspectRatio(
Rational(16, 9)).build()) }
        }
    }

    private fun keyToAdbCommand(key: String): String {
        val digitCode = key.removePrefix("DIGIT_").singleOrNull()?.digitToIntOrNull()?.let { 7 + it }
        val code = digitCode ?: when (key) {
            "POWER" -> 26; "HOME" -> 3; "BACK" -> 4; "UP" -> 19; "DOWN" -> 20
            "LEFT" -> 21; "RIGHT" -> 22; "ENTER" -> 66; "VOLUME_UP" -> 24
            "VOLUME_DOWN" -> 25; "MUTE" -> 164; "RECENTS" -> 187
            "CHANNEL_UP" -> 166; "CHANNEL_DOWN" -> 167; "DELETE" -> 67
            else -> throw IllegalArgumentException("${loc(ui.language, "\u0632\u0631 \u063A\u064A\u0631 \
u0645\u062F\u0639\u0648\u0645", "Unsupported remote key")}: $key")
        }
        return "input keyevent $code"
    }

    private fun friendlyError(error: Throwable): String = when (error) {
        is ConnectException -> loc(ui.language, "\u0631\u0641\u0636 \u0627\u0644\u062C\u0647\u0627\u0632 \
u0627\u0644\u0627\u062A\u0635\u0627\u0644. \u062A\u0623\u0643\u062F \u0623\u0646 \u062E\u0627\u0631\u0645 \
u0627\u0644\u062A\u0637\u0628\u064A\u0642 \u064A\u0639\u0645\u0644 \u0623\u0648 \u0623\u0646 ADB TCP \u0645\
u0641\u0639\u0651\u0644 \u0639\u0644\u0649 \u0627\u0644\u0645\u0646\u0641\u0630 \u0627\u0644\u0635\u062D\
u064A\u062D.", "The device refused the connection. Check that its server is running or ADB TCP uses the
selected port.")
        is SocketTimeoutException -> loc(ui.language, "\u0627\u0646\u062A\u0647\u062A \u0645\u0647\u0644\
u0629 \u0627\u0644\u0627\u062A\u0635\u0627\u0644. \u062A\u062D\u0642\u0642 \u0645\u0646 \u0627\u0644\u0634\
u0628\u0643\u0629 \u0648\u0643\u0648\u062F \u0627\u0644\u0627\u062A\u0635\u0627\u0644 \u0648\u060C \u062B\
u0645 \u0623\u0639\u062F \u0627\u0644\u0645\u062D\u0627\u0648\u0644\u0629.", "Connection timed out. Check
the network and pairing code, then try again.")
        is java.net.UnknownHostException -> loc(ui.language, "\u062A\u0639\u0630\u0631 \u0627\u0644\u0639\
u062B\u0648\u0631 \u0639\u0644\u0649 \u0639\u0646\u0648\u0627\u0646 \u0627\u0644\u062C\u0647\u0627\u0632. \
u062A\u062D\u0642\u0642 \u0645\u0646 IP.", "Could not resolve the device address. Check the IP.")
        is SecurityException -> loc(ui.language, "\u0644\u0645 \u064A\u064F\u0633\u0645\u062D \u0628\u0627\
u0644\u0648\u0635\u0648\u0644 \u0625\u0644\u0649 \u0627\u0644\u0634\u0628\u0643\u0629 \u0627\u0644\u0645\
u062D\u0644\u064A\u0629. \u0631\u0627\u062C\u0639 \u0623\u0630\u0648\u0646\u0627\u062A \u0627\u0644\u062A\
u0637\u0628\u064A\u0642.", "Local network access was denied. Check app permissions.")
        else -> error.message?.takeIf(String::isNotBlank) ?: loc(ui.language, "\u062A\u0639\u0630\u0631 \
u0627\u0644\u0627\u062A\u0635\u0627\u0644 \u0628\u0627\u0644\u0631\u0633\u064A\u0641\u0631.", "Could not
connect to the receiver.")
    }

    private fun queryDisplayName(resolver: ContentResolver, uri: Uri): String? = runCatching {
        resolver.query(uri, arrayOf(android.provider.OpenableColumns.DISPLAY_NAME), null, null, null)?.use {
 cursor ->
            if (cursor.moveToFirst()) cursor.getString(0) else null
        }
    }.getOrNull()

    private fun querySize(resolver: ContentResolver, uri: Uri): Long = runCatching {
        resolver.query(uri, arrayOf(android.provider.OpenableColumns.SIZE), null, null, null)?.use { cursor
->
            if (cursor.moveToFirst() && !cursor.isNull(0)) cursor.getLong(0) else -1L
        } ?: -1L
    }.getOrDefault(-1L)

    private fun formatBytes(bytes: Long): String = when {
        bytes >= 1024L * 1024 * 1024 -> "%.1f GB".format(bytes / (1024.0 * 1024 * 1024))
        bytes >= 1024L * 1024 -> "%.1f MB".format(bytes / (1024.0 * 1024))
        else -> "%.0f KB".format(bytes / 1024.0)
    }
}
```

controller/src/main/java/com/monib/receivercontrol/controller/WebsiteActivity.kt

```
package com.monib.receivercontrol.controller

import android.app.DownloadManager
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.os.Environment
import android.webkit.WebResourceRequest
import android.webkit.WebView
import android.webkit.WebViewClient
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.ui.Modifier
import androidx.compose.ui.viewinterop.AndroidView

class WebsiteActivity : ComponentActivity() {
    @OptIn(ExperimentalMaterial3Api::class)
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val language = getSharedPreferences("controller", MODE_PRIVATE).getString("language", "ar") ?: "ar"
        setContent {
            MaterialTheme {
                Scaffold(
                    topBar = {
                        TopAppBar(
                            title = { Text(loc(language, "\u0645\u0648\u0642\u0639 \u0627\u0644\u062A\u0637\
u0628\u064A\u0642", "App website")) },
                            navigationIcon = {
                                TextButton(onClick = { finish() }) { Text(loc(language, "\u0631\u062C\u0648\
u0639", "Back")) }
                            }
                        )
                    }
                ) { padding ->
                    AndroidView(
                        modifier = Modifier.fillMaxSize().padding(padding),
                        factory = { context ->
                            WebView(context).apply {
                                settings.javaScriptEnabled = true
                                settings.domStorageEnabled = true
                                settings.allowFileAccess = false
                                setDownloadListener { url, _, _, _, _ -> downloadApk(url, language) }
                                webViewClient = object : WebViewClient() {
                                    override fun shouldOverrideUrlLoading(view: WebView, request:
WebResourceRequest): Boolean {
                                        val uri = request.url
                                        if (uri.path.orEmpty().endsWith(".apk", ignoreCase = true)) {
                                            downloadApk(uri.toString(), language)
                                            return true
                                        }
                                        if (uri.scheme in setOf("http", "https") && uri.host == SITE_HOST)
return false
                                        if (uri.scheme in setOf("http", "https")) {
                                            startActivity(Intent(Intent.ACTION_VIEW, uri))
                                            return true
                                        }
                                        return true
                                    }
                                }
                                loadUrl(SITE_URL)
                            }
                        }
                    )
                }
            }
        }
    }

    private fun downloadApk(rawUrl: String, language: String) {
        val uri = runCatching { Uri.parse(rawUrl) }.getOrNull()
        if (uri == null || uri.scheme != "https" || uri.host != SITE_HOST || !uri.path.orEmpty().endsWith(".
apk", ignoreCase = true)) {
            Toast.makeText(this, loc(language, "\u0631\u0627\u0628\u0637 \u0627\u0644\u062A\u0646\u0632\
u064A\u0644 \u063A\u064A\u0631 \u0635\u0627\u0644\u062D.", "This download link is not valid."), Toast.
LENGTH_LONG).show()
            return
        }

        val fileName = uri.lastPathSegment?.takeIf { it.endsWith(".apk", ignoreCase = true) } ?:
"AndroidRemote.apk"
```

```
        val request = DownloadManager.Request(uri)
            .setTitle(fileName)
            .setDescription(loc(language, "\u062C\u0627\u0631\u064D \u062A\u0646\u0632\u064A\u0644 \u0627\
u0644\u062A\u0637\u0628\u064A\u0642", "Downloading app"))
            .setMimeType("application/vnd.android.package-archive")
            .setNotificationVisibility(DownloadManager.Request.VISIBILITY_VISIBLE_NOTIFY_COMPLETED)
            .setDestinationInExternalPublicDir(Environment.DIRECTORY_DOWNLOADS, fileName)
        val manager = getSystemService(DOWNLOAD_SERVICE) as DownloadManager
        manager.enqueue(request)
        Toast.makeText(this, loc(language, "\u0628\u062F\u0623 \u062A\u0646\u0632\u064A\u0644 \u0627\u0644\
u062A\u0637\u0628\u064A\u0642. \u0633\u062A\u062C\u062F\u0647 \u0641\u064A \u0627\u0644\u062A\u0646\u0632\
u064A\u0644\u0627\u062A.", "Download started. Find it in Downloads."), Toast.LENGTH_LONG).show()
    }

    private companion object {
        const val SITE_HOST = "brimiomxf-arch.github.io"
        const val SITE_URL = "https://brimiomxf-arch.github.io/android-remote-site1/"
    }
}
```

controller/src/main/java/com/monib/receivercontrol/controller/widgets/ReferenceDpadView.kt

```
package com.monib.receivercontrol.controller.widgets


import android.animation.AnimatorSet
import android.animation.ObjectAnimator
import android.animation.ValueAnimator
import android.content.Context
import android.graphics.*
import android.util.AttributeSet
import android.view.HapticFeedbackConstants
import android.view.MotionEvent
import android.view.View
import android.view.animation.AccelerateDecelerateInterpolator
import android.view.animation.OvershootInterpolator
// Adapted from Outofcontext Team's Android TV Remote DpadView (MIT License).

/**
 * DpadView \u2014 Material You 3 Expressive D-Pad
 *
 * A clean, minimal D-Pad with:
 *  \u2022 Organic petal-shaped directional zones
 *  \u2022 Ripple press feedback per zone
 *  \u2022 Center OK button with subtle bounce
 *  \u2022 Adaptive color from M3 color scheme
 *  \u2022 Haptic on every press
 *
 * Usage in XML:
 *   <com.out.androidtvremote.views.DpadView
 *       android:id="@+id/dpad"
 *       android:layout_width="match_parent"
 *       android:layout_height="match_parent"
 *       android:clickable="true"
 *       android:focusable="true" />
 */
class ReferenceDpadView @JvmOverloads constructor(
    context: Context,
    attrs: AttributeSet? = null,
    defStyleAttr: Int = 0
) : View(context, attrs, defStyleAttr) {

    // \u2500\u2500 Listener \u2500\u2500\u2500\
```

```
    // \u2500\u2500 State \u2500\u2500\u2500\
```

```
        }
        okTextPaint.apply {
            color = colorOnSurface
            textAlign = Paint.Align.CENTER
            isFakeBoldText = true
        }
        ripplePaint.apply {
            color = colorPrimary
            style = Paint.Style.FILL
        }
    }

    // \u2500\u2500 Layout \u2500\u2500\u2500\
```

```
        outerRingPaint.color = colorOutlineVariant
        canvas.drawCircle(cx, cy, innerR, outerRingPaint)
        outerRingPaint.color = colorOutlineVariant

        // OK text
        okTextPaint.color = if (pressedZone == Zone.CENTER) colorOnPrimaryContainer else colorOnSurface
        canvas.drawText("OK", cx, cy + okTextPaint.textSize * 0.38f, okTextPaint)
        canvas.restore()

        // 6. Arrows
        drawArrow(canvas, Zone.UP)
        drawArrow(canvas, Zone.DOWN)
        drawArrow(canvas, Zone.LEFT)
        drawArrow(canvas, Zone.RIGHT)

        // 7. Ripple
        if (rippleAlpha > 0f) {
            ripplePaint.alpha = (rippleAlpha * 60).toInt()
            canvas.drawCircle(rippleCx, rippleCy, rippleRadius, ripplePaint)
        }
    }

    private fun drawDividers(canvas: Canvas) {
        // Vertical line (top arc \u2192 bottom arc, interrupted by center)
        val gap = innerR + 2.dp
        canvas.drawLine(cx, cy - outerR, cx, cy - gap, dividerPaint)
        canvas.drawLine(cx, cy + gap, cx, cy + outerR, dividerPaint)
        // Horizontal line
        canvas.drawLine(cx - outerR, cy, cx - gap, cy, dividerPaint)
        canvas.drawLine(cx + gap, cy, cx + outerR, cy, dividerPaint)
    }

    private fun drawArrow(canvas: Canvas, zone: Zone) {
        val idx = when (zone) { Zone.UP -> 0; Zone.RIGHT -> 1; Zone.DOWN -> 2; else -> 3 }
        val pressed = sectorAlpha[idx] > 0.5f

        arrowPaint.color = if (pressed) colorPrimary else colorOnSurface
        arrowPaint.alpha = if (pressed) 255 else (255 * 0.7f).toInt()
        arrowPaint.style = Paint.Style.FILL

        val size = outerR * 0.11f   // arrowhead half-size
        val (ax, ay) = when (zone) {
            Zone.UP    -> Pair(cx, cy - arrowR)
            Zone.DOWN  -> Pair(cx, cy + arrowR)
            Zone.LEFT  -> Pair(cx - arrowR, cy)
            Zone.RIGHT -> Pair(cx + arrowR, cy)
            else -> return
        }

        arrowPath.reset()
        when (zone) {
            Zone.UP -> {
                arrowPath.moveTo(ax, ay - size)
                arrowPath.lineTo(ax + size * 1.2f, ay + size * 0.7f)
                arrowPath.lineTo(ax - size * 1.2f, ay + size * 0.7f)
            }
            Zone.DOWN -> {
                arrowPath.moveTo(ax, ay + size)
                arrowPath.lineTo(ax + size * 1.2f, ay - size * 0.7f)
                arrowPath.lineTo(ax - size * 1.2f, ay - size * 0.7f)
            }
            Zone.LEFT -> {
                arrowPath.moveTo(ax - size, ay)
                arrowPath.lineTo(ax + size * 0.7f, ay - size * 1.2f)
                arrowPath.lineTo(ax + size * 0.7f, ay + size * 1.2f)
            }
            Zone.RIGHT -> {
                arrowPath.moveTo(ax + size, ay)
                arrowPath.lineTo(ax - size * 0.7f, ay - size * 1.2f)
                arrowPath.lineTo(ax - size * 0.7f, ay + size * 1.2f)
            }
            else -> return
        }
        arrowPath.close()
        canvas.drawPath(arrowPath, arrowPaint)
    }

    // \u2500\u2500 Touch handling \u2500\u2500\u2500\
```

```
                if (zone != pressedZone) {
                    animatePress(pressedZone, false, tx, ty)
                    pressedZone = zone
                    animatePress(zone, true, tx, ty)
                }
            }
            MotionEvent.ACTION_UP, MotionEvent.ACTION_CANCEL -> {
                if (event.actionMasked == MotionEvent.ACTION_UP) {
                    fireAction(pressedZone)
                    launchRipple(pressedZone, tx, ty)
                }
                animatePress(pressedZone, false, tx, ty)
                pressedZone = Zone.NONE
            }
        }
        invalidate()
        return true
    }

    private fun hitZone(x: Float, y: Float): Zone {
        val dx = x - cx
        val dy = y - cy
        val dist = Math.hypot(dx.toDouble(), dy.toDouble()).toFloat()
        if (dist > outerR) return Zone.NONE
        if (dist < innerR) return Zone.CENTER
        // Quadrant by angle
        val angle = Math.toDegrees(Math.atan2(dy.toDouble(), dx.toDouble())).toFloat()
        return when {
            angle in -135f..-45f -> Zone.UP
            angle in -45f..45f   -> Zone.RIGHT
            angle in 45f..135f   -> Zone.DOWN
            else                  -> Zone.LEFT
        }
    }

    private fun fireAction(zone: Zone) {
        when (zone) {
            Zone.UP     -> listener?.onUp()
            Zone.DOWN   -> listener?.onDown()
            Zone.LEFT   -> listener?.onLeft()
            Zone.RIGHT  -> listener?.onRight()
            Zone.CENTER -> listener?.onCenter()
            Zone.NONE   -> {}
        }
    }

    // \u2500\u2500 Animation helpers \u2500\u2500\u2500\
```

```
        }
    }

    // property for ObjectAnimator
    private fun setCenterScale(v: Float) { centerScale = v; invalidate() }
    private fun getCenterScale() = centerScale

    private fun zoneIdx(zone: Zone) = when (zone) {
        Zone.UP -> 0; Zone.RIGHT -> 1; Zone.DOWN -> 2; Zone.LEFT -> 3; else -> null
    }

    // \u2500\u2500 dp helper \u2500\u2500\u2500\
```

receiver/src/main/java/com/monib/receivercontrol/receiver/LocalReceiverService.kt

```
package com.monib.receivercontrol.receiver

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.Service
import android.content.Context
import android.content.Intent
import android.media.AudioManager
import android.os.Build
import android.os.IBinder
import android.os.PowerManager
import android.util.Base64
import android.content.pm.PackageManager
import android.content.pm.ServiceInfo
import android.net.Uri
import com.google.gson.Gson
import com.google.gson.JsonObject
import com.google.gson.JsonParser
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.cancel
import kotlinx.coroutines.launch
import java.io.BufferedInputStream
import java.io.DataInputStream
import java.io.DataOutputStream
import java.io.File
import java.net.DatagramPacket
import java.net.DatagramSocket
import java.net.ServerSocket
import java.net.Socket
import java.util.concurrent.CountDownLatch
import java.util.concurrent.ConcurrentHashMap
import java.util.concurrent.TimeUnit

class LocalReceiverService : Service() {
    private val gson = Gson()
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.IO)
    @Volatile private var pairingCode = ""
    @Volatile private var udpSocket: DatagramSocket? = null
    @Volatile private var tcpServer: ServerSocket? = null
    private val failedPairings = ConcurrentHashMap<String, Pair<Int, Long>>()

    override fun onCreate() {
        super.onCreate()
        pairingCode = ReceiverPairing.getOrCreateCode(this)
        getSharedPreferences(PREFS, Context.MODE_PRIVATE).edit().putBoolean(SERVER_RUNNING_KEY, true).apply(
)
        running = true
        createNotificationChannel()
        val notification = buildNotification()
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
            startForeground(NOTIFICATION_ID, notification, ServiceInfo.
FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE)
        } else {
            startForeground(NOTIFICATION_ID, notification)
        }
        scope.launch { discoveryLoop() }
        scope.launch { commandLoop() }
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int = START_STICKY
    override fun onBind(intent: Intent?): IBinder? = null

    override fun onDestroy() {
        udpSocket?.close()
        tcpServer?.close()
        running = false
        getSharedPreferences(PREFS, Context.MODE_PRIVATE).edit().putBoolean(SERVER_RUNNING_KEY, false).
apply()
        scope.cancel()
        super.onDestroy()
    }

    private suspend fun discoveryLoop() {
        val socket = try {
            DatagramSocket(null).apply {
                reuseAddress = true
                broadcast = true
                bind(java.net.InetSocketAddress(DISCOVERY_PORT))
                soTimeout = 1000
            }
        } catch (_: Exception) { stopSelf(); return }
        udpSocket = socket
        val buffer = ByteArray(2048)
        while (true) {
```

```
            try {
                val packet = DatagramPacket(buffer, buffer.size)
                socket.receive(packet)
                val request = JsonParser.parseString(String(packet.data, packet.offset, packet.length,
Charsets.UTF_8)).asJsonObject
                if (request.get("type")?.asString != "DISCOVER") continue
                val response = JsonObject().apply {
                    addProperty("type", "RECEIVER")
                    addProperty("name", Build.MODEL.orEmpty().ifBlank { "Android Receiver" })
                    addProperty("port", COMMAND_PORT)
                }.toString().toByteArray(Charsets.UTF_8)
                socket.send(DatagramPacket(response, response.size, packet.address, packet.port))
            } catch (_: java.net.SocketTimeoutException) {
                // Recheck cancellation and service state.
            } catch (cancelled: CancellationException) {
                throw cancelled
            } catch (_: Exception) {
                if (socket.isClosed) break
            }
        }
    }

    private suspend fun commandLoop() {
        val server = try { ServerSocket(COMMAND_PORT) } catch (_: Exception) { stopSelf(); return }
        tcpServer = server
        while (true) {
            try {
                val socket = server.accept()
                scope.launch { handleClient(socket) }
            } catch (cancelled: CancellationException) {
                throw cancelled
            } catch (_: Exception) {
                if (server.isClosed) break
            }
        }
    }

    private fun handleClient(socket: Socket) {
        socket.use { client ->
            client.soTimeout = 10_000
            val input = DataInputStream(BufferedInputStream(client.getInputStream()))
            val output = DataOutputStream(client.getOutputStream())
            try {
                val headerSize = input.readInt()
                require(headerSize in 1..MAX_HEADER_SIZE) { "\u0631\u0623\u0633 \u0627\u0644\u0637\u0644\
u0628 \u063A\u064A\u0631 \u0635\u0627\u0644\u062D" }
                val headerBytes = ByteArray(headerSize)
                input.readFully(headerBytes)
                val request = JsonParser.parseString(String(headerBytes, Charsets.UTF_8)).asJsonObject
                val type = request.get("type")?.asString.orEmpty()
                val peer = client.inetAddress.hostAddress ?: "unknown"
                val attempt = failedPairings[peer]
                if (attempt != null && attempt.second > System.currentTimeMillis()) {
                    reply(output, error("\u0645\u062D\u0627\u0648\u0644\u0627\u062A \u0643\u062B\u064A\
u0631\u0629. \u0623\u0639\u062F \u0627\u0644\u0645\u062D\u0627\u0648\u0644\u0629 \u0628\u0639\u062F \u062F\
u0642\u064A\u0642\u0629."))
                    return
                }
                if (request.get("code")?.asString != pairingCode) {
                    val count = if (attempt == null || attempt.second > 0L) 1 else attempt.first + 1
                    failedPairings[peer] = count to if (count >= 5) System.currentTimeMillis() + 60_000
else 0L
                    reply(output, error("\u0643\u0648\u062F \u0627\u0644\u0627\u0642\u062A\u0631\u0627\
u0646 \u063A\u064A\u0631 \u0635\u062D\u064A\u062D"))
                    return
                }
                failedPairings.remove(peer)
                when (type) {
                    "PAIR" -> reply(output, JsonObject().apply { addProperty("ok", true); addProperty(
"name", Build.MODEL) })
                    "FILE" -> receiveFile(input, output, request)
                    "COMMAND" -> if (request.get("action")?.asString == "SCREENSHOT") captureScreen(output)
else reply(output, execute(request))
                    else -> reply(output, error("\u0646\u0648\u0639 \u0627\u0644\u0637\u0644\u0628 \u063A\
u064A\u0631 \u0645\u0639\u0631\u0648\u0641"))
                }
            } catch (e: Exception) {
                runCatching { reply(output, error(e.message ?: "\u062A\u0639\u0630\u0631 \u062A\u0646\u0641\
u064A\u0630 \u0627\u0644\u0637\u0644\u0628")) }
            }
        }
    }

    private fun captureScreen(output: DataOutputStream) {
        if (Build.VERSION.SDK_INT < 30) {
            reply(output, error("\u0639\u0631\u0636 \u0627\u0644\u0634\u0627\u0634\u0629 \u064A\u062D\u062A\
u0627\u062C Android 11 \u0623\u0648 \u0623\u062D\u062F\u062B \u0639\u0644\u0649 \u0627\u0644\u0631\u0633\
u064A\u0641\u0631"))
            return
        }
        val latch = CountDownLatch(1)
        var image: ByteArray? = null
        val started = ReceiverAccessibilityService.captureScreenshot { bytes ->
            image = bytes
            latch.countDown()
```

```
        }
        if (!started) {
            reply(output, error("\u0641\u0639\u0651\u0644 \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\u062D\
u0643\u0645 \u0627\u0644\u0645\u0633\u0627\u0639\u062F \u0641\u064A \u0625\u0639\u062F\u0627\u062F\u0627\
u062A \u0625\u0645\u0643\u0627\u0646\u064A\u0629 \u0627\u0644\u0648\u0635\u0648\u0644 \u0639\u0644\u0649 \
u0627\u0644\u0631\u0633\u0627\u0644\u0629"))
            return
        }
        if (!latch.await(8, TimeUnit.SECONDS)) {
            reply(output, error("\u0627\u0646\u062A\u0647\u062A \u0645\u0647\u0644\u0629 \u0627\u0644\u062A\
u0642\u0627\u0637 \u0627\u0644\u0634\u0627\u0634\u0629 \u0645\u0646 \u0627\u0644\u0631\u0633\u0627\u0644\
u0631"))
            return
        }
        val bytes = image ?: run {
            reply(output, error("\u062A\u0639\u0630\u0631 \u0627\u0644\u062A\u0642\u0627\u0637 \u0627\u0644\
u0634\u0627\u0634\u0629. \u062A\u0623\u0643\u062F \u0645\u0646 \u062A\u0641\u0639\u064A\u0644 \u062E\u062F\
u0645\u0629 \u0627\u0644\u062A\u062D\u0643\u0645 \u0627\u0644\u0645\u0633\u0627\u0639\u062F."))
            return
        }
        reply(output, JsonObject().apply {
            addProperty("ok", true)
            addProperty("message", "\u062A\u0645 \u0627\u0644\u062A\u0642\u0627\u0637 \u0627\u0644\u0634\
u0627\u0634\u0629")
            addProperty("image", Base64.encodeToString(bytes, Base64.NO_WRAP))
        })
    }

    private fun receiveFile(input: DataInputStream, output: DataOutputStream, request: JsonObject) {
        val name = safeFileName(request.get("name")?.asString.orEmpty())
        val size = request.get("size")?.asLong ?: -1L
        require(size in 0..MAX_FILE_SIZE) { "\u062D\u062C\u0645 \u0627\u0644\u0645\u0644\u0641 \u063A\u064A\
u0631 \u0645\u0633\u0645\u0648\u062D" }
        val directory = File(getExternalFilesDir("Download") ?: filesDir, "ReceiverControl").apply { mkdirs(
) }
        val destination = File(directory, name)
        destination.outputStream().buffered().use { stream ->
            val buffer = ByteArray(32 * 1024)
            var remaining = size
            while (remaining > 0) {
                val read = input.read(buffer, 0, minOf(buffer.size.toLong(), remaining).toInt())
                if (read < 0) throw java.io.EOFException("\u0627\u0646\u0642\u0637\u0639 \u0646\u0642\u0644
\u0627\u0644\u0645\u0644\u0641")
                stream.write(buffer, 0, read)
                remaining -= read
            }
        }
        reply(output, JsonObject().apply {
            addProperty("ok", true)
            addProperty("message", "\u062A\u0645 \u0627\u0633\u062A\u0644\u0627\u0645 \u0627\u0644\u0645\
u0644\u0641")
            addProperty("name", destination.name)
            addProperty("path", destination.absolutePath)
        })
    }

    private fun execute(request: JsonObject): JsonObject {
        val action = request.get("action")?.asString.orEmpty()
        val value = request.get("value")?.asString.orEmpty()
        return try {
            when (action) {
                "KEY" -> executeKey(value)
                "APPS" -> listApps()
                "LAUNCH" -> launchApp(value)
                "UNINSTALL" -> uninstallApp(value)
                "OPEN_URL" -> openUrl(value)
                "INFO" -> JsonObject().apply {
                    addProperty("ok", true)
                    addProperty("name", Build.MODEL)
                    addProperty("android", Build.VERSION.RELEASE)
                    addProperty("accessibility", ReceiverAccessibilityService.isEnabled())
                }
                "CUSTOM" -> {
                    val custom = JsonParser.parseString(request.get("json")?.asString.orEmpty()).
asJsonObject
                    when (custom.get("action")?.asString) {
                        "KEY" -> executeKey(custom.get("value")?.asString.orEmpty())
                        "INFO" -> execute(JsonObject().apply { addProperty("action", "INFO") })
                        "APPS" -> listApps()
                        "LAUNCH" -> launchApp(custom.get("package")?.asString.orEmpty())
                        "UNINSTALL" -> uninstallApp(custom.get("package")?.asString.orEmpty())
                        "OPEN_URL" -> openUrl(custom.get("url")?.asString.orEmpty())
                        else -> error("\u0627\u0644\u0623\u0645\u0631 \u0627\u0644\u0645\u062E\u0635\u0635 \
u063A\u064A\u0631 \u0645\u062F\u0639\u0648\u0645 \u0645\u0646 \u062A\u0637\u0628\u064A\u0642 \u0627\u0644\
u0631\u0633\u064A\u0644\u0631")
                    }
                }
                else -> error("\u0627\u0644\u0623\u0645\u0631 \u063A\u064A\u0631 \u0645\u0639\u0631\u0648\
u0641")
            }
        } catch (e: Exception) {
            error(e.message ?: "\u062A\u0639\u0630\u0631 \u062A\u0646\u0641\u064A\u0630 \u0627\u0644\u0623\
u0645\u0631")
        }
    }
```

```
    private fun executeKey(key: String): JsonObject {
        val audio = getSystemService(AUDIO_SERVICE) as AudioManager
        when (key) {
            "HOME", "BACK", "LOCK", "RECENTS" -> {
                if (!ReceiverAccessibilityService.perform(key)) {
                    return error("\u0641\u0639\u0651\u0644 \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\
u062D\u0643\u0645 \u0627\u0644\u0645\u0633\u0627\u0639\u062F \u0641\u064A \u0634\u0627\u0634\u0629 \u0627\
u0644\u0631\u0633\u064A\u0641\u0631 \u0644\u062A\u0646\u0641\u064A\u0630 \u0623\u0632\u0631\u0627\u0631 \
u0627\u0644\u0646\u0638\u0627\u0645")
                }
            }
            "VOLUME_UP" -> audio.adjustStreamVolume(AudioManager.STREAM_MUSIC, AudioManager.ADJUST_RAISE, 0)
            "VOLUME_DOWN" -> audio.adjustStreamVolume(AudioManager.STREAM_MUSIC, AudioManager.ADJUST_LOWER,
0)
            "MUTE" -> audio.adjustStreamVolume(AudioManager.STREAM_MUSIC, AudioManager.ADJUST_TOGGLE_MUTE,
0)
            "POWER" -> {
                val power = getSystemService(POWER_SERVICE) as PowerManager
                if (!power.isInteractive) {
                    @Suppress("DEPRECATION")
                    power.newWakeLock(PowerManager.SCREEN_BRIGHT_WAKE_LOCK or PowerManager.
ACQUIRE_CAUSES_WAKEUP, "ReceiverControl:Wake").acquire(1500)
                } else if (!ReceiverAccessibilityService.perform("LOCK")) {
                    return error("\u0644\u062A\u0634\u063A\u064A\u0644/\u0625\u0637\u0641\u0627\u0621 \
u0627\u0644\u0634\u0627\u0634\u0629 \u0641\u0639\u0651\u0644 \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\
u062D\u0643\u0645 \u0627\u0644\u0645\u0633\u0627\u0639\u062F \u0639\u0644\u0649 \u0627\u0644\u0631\u0633\
u064A\u0641\u0631")
                }
            }
            "UP", "DOWN", "LEFT", "RIGHT", "ENTER" -> {
                if (!ReceiverAccessibilityService.dispatchTouch(key)) {
                    return error("\u0641\u0639\u0651\u0644 \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\
u062D\u0643\u0645 \u0627\u0644\u0645\u0633\u0627\u0639\u062F \u0644\u062A\u0646\u0641\u064A\u0630 \u0625\
u064A\u0645\u0627\u0621\u0627\u062A \u0627\u0644\u062A\u0646\u0642\u0644 \u0648\u0627\u0644\u0627\u062E\
u062A\u064A\u0627\u0631")
                }
            }
            "CHANNEL_UP", "CHANNEL_DOWN", "DELETE" -> return error("\u0647\u0630\u0627 \u0627\u0644\u0632\
u0631 \u064A\u062D\u062A\u0627\u062C \u0627\u062A\u0635\u0627\u0644 ADB \u0639\u0644\u0649 \u0627\u0644\
u0631\u0633\u064A\u0641\u0631")
            else -> return error(
                if (key.matches(Regex("DIGIT_[0-9]"))) "\u0625\u0631\u0633\u0627\u0644 \u0623\u0631\u0642\
u0627\u0645 \u0627\u0644\u0642\u0646\u0648\u0627\u062A \u064A\u062D\u062A\u0627\u062C \u0627\u062A\u0635\
u0627\u0644 ADB \u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631"
                else "\u0632\u0631 \u063A\u064A\u0631 \u0645\u062F\u0639\u0648\u0645"
            )
        }
        return JsonObject().apply { addProperty("ok", true); addProperty("message", "\u062A\u0645 \u062A\
u0646\u0641\u064A\u0630 \u0627\u0644\u0623\u0645\u0631") }
    }

    private fun listApps(): JsonObject {
        val launcher = Intent(Intent.ACTION_MAIN).addCategory(Intent.CATEGORY_LAUNCHER)
        val resolved = if (Build.VERSION.SDK_INT >= 33) {
            packageManager.queryIntentActivities(launcher, PackageManager.ResolveInfoFlags.of(0))
        } else {
            @Suppress("DEPRECATION")
            packageManager.queryIntentActivities(launcher, 0)
        }
        val apps = resolved.map { info ->
            JsonObject().apply {
                addProperty("name", info.loadLabel(packageManager).toString())
                addProperty("package", info.activityInfo.packageName)
            }
        }.distinctBy { it.get("package").asString }.sortedBy { it.get("name").asString.lowercase() }
        return JsonObject().apply {
            addProperty("ok", true)
            add("apps", gson.toJsonTree(apps))
        }
    }

    private fun launchApp(packageName: String): JsonObject {
        require(packageName.matches(PACKAGE_NAME)) { "\u0627\u0633\u0645 \u0627\u0644\u062D\u0632\u0645\
u0629 \u063A\u064A\u0631 \u0635\u0627\u0644\u062D" }
        val intent = packageManager.getLaunchIntentForPackage(packageName)?.addFlags(Intent.
FLAG_ACTIVITY_NEW_TASK)
            ?: return error("\u0644\u0627 \u064A\u0648\u062C\u062F \u0646\u0634\u0627\u0637 \u062A\u0634\
u063A\u064A\u0644 \u0644\u0647\u0630\u0627 \u0627\u0644\u062A\u0637\u0628\u064A\u0642")
        return try {
            startActivity(intent)
            JsonObject().apply { addProperty("ok", true); addProperty("message", "\u062A\u0645 \u0637\u0644\
u0628 \u0641\u062A\u062D \u0627\u0644\u062A\u0637\u0628\u064A\u0642") }
        } catch (_: Exception) {
            error("\u0645\u0646\u0639 Android \u0641\u062A\u062D \u0627\u0644\u062A\u0637\u0628\u064A\u0642
\u0641\u064A \u0627\u0644\u062E\u0644\u0641\u064A\u0629. \u0627\u0641\u062A\u062D \u062A\u0637\u0628\u064A\
u0642 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0623\u0648\u0644\u0627\u064B \u0623\u0648 \u0627\u0633\
u062A\u062E\u062F\u0645 ADB.")
        }
    }

    private fun uninstallApp(packageName: String): JsonObject {
        require(packageName.matches(PACKAGE_NAME)) { "\u0627\u0633\u0645 \u0627\u0644\u062D\u0632\u0645\
u0629 \u063A\u064A\u0631 \u0635\u0627\u0644\u062D" }
        val intent = Intent(Intent.ACTION_DELETE, Uri.parse("package:$packageName")).addFlags(Intent.
FLAG_ACTIVITY_NEW_TASK)
```

```
        return try {
            startActivity(intent)
            JsonObject().apply { addProperty("ok", true); addProperty("message", "\u062A\u0623\u0643\u064A\
u062F \u0627\u0644\u062D\u0630\u0641 \u0645\u0637\u0644\u0648\u0628 \u0639\u0644\u0649 \u0627\u0644\u0631\
u0633\u064A\u0641\u0631") }
        } catch (_: Exception) {
            error("\u062A\u0639\u0630\u0631 \u0641\u062A\u062D \u0634\u0627\u0634\u0629 \u0625\u0644\u063A\
u0627\u0621 \u0627\u0644\u062A\u062B\u0628\u064A\u062A \u0645\u0646 \u0627\u0644\u062E\u0644\u0641\u064A\
u0629\u061B \u0627\u0633\u062A\u062E\u062F\u0645 ADB \u0623\u0648 \u0627\u0641\u062A\u062D \u0627\u0644\
u062A\u0637\u0628\u064A\u0642 \u0639\u0644\u0649 \u0627\u0644\u0631\u0633\u064A\u0641\u0631")
        }
    }

    private fun openUrl(value: String): JsonObject {
        val uri = Uri.parse(value)
        require(uri.scheme in setOf("http", "https") && !uri.host.isNullOrBlank()) { "\u0627\u0644\u0631\
u0627\u0628\u0637 \u064A\u062C\u0628 \u0623\u0646 \u064A\u0628\u062F\u0623 \u0628\u0640 HTTP \u0623\u0648
HTTPS" }
        return try {
            startActivity(Intent(Intent.ACTION_VIEW, uri).addFlags(Intent.FLAG_ACTIVITY_NEW_TASK))
            JsonObject().apply { addProperty("ok", true); addProperty("message", "\u062A\u0645 \u0637\u0644\
u0628 \u0641\u062A\u062D \u0627\u0644\u0631\u0627\u0628\u0637") }
        } catch (_: Exception) {
            error("\u0645\u0646\u0639 Android \u0641\u062A\u062D \u0627\u0644\u0631\u0627\u0628\u0637 \
u0641\u064A \u0627\u0644\u062E\u0644\u0641\u064A\u0629. \u0627\u0641\u062A\u062D \u062A\u0637\u0628\u064A\
u0642 \u0627\u0644\u0631\u0633\u064A\u0641\u0631 \u0623\u0648\u0644\u0627\u064B \u0623\u0648 \u0627\u0633\
u062A\u062E\u062F\u0645 ADB.")
        }
    }

    private fun reply(output: DataOutputStream, json: JsonObject) {
        val bytes = json.toString().toByteArray(Charsets.UTF_8)
        output.writeInt(bytes.size)
        output.write(bytes)
        output.flush()
    }

    private fun error(message: String) = JsonObject().apply { addProperty("ok", false); addProperty(
"message", message) }
    private fun safeFileName(name: String): String {
        val cleaned = name.substringAfterLast('/').substringAfterLast('\\').replace(Regex("[^\\p{L}\\p{N}._
-]"), "_").trim().trim('.')
        return cleaned.take(150).ifBlank { "received_file" }
    }

    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= 26) {
            val channel = NotificationChannel(CHANNEL_ID, "\u062E\u0627\u062F\u0645 Android TV
responsiveness", NotificationManager.IMPORTANCE_LOW)
            (getSystemService(NOTIFICATION_SERVICE) as NotificationManager).createNotificationChannel(
channel)
        }
    }

    private fun buildNotification(): Notification {
        val pending = PendingIntent.getActivity(
            this, 0, Intent(this, MainActivity::class.java),
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )
        val builder = if (Build.VERSION.SDK_INT >= 26) Notification.Builder(this, CHANNEL_ID) else
Notification.Builder(this)
        return builder.setContentTitle("Android TV responsiveness \u064A\u0639\u0645\u0644")
            .setContentText("\u062C\u0627\u0647\u0632 \u0644\u0644\u0627\u062A\u0635\u0627\u0644 \u0639\
u0644\u0649 \u0627\u0644\u0634\u0628\u0643\u0629 \u0627\u0644\u0645\u062D\u0644\u064A\u0629")
            .setSmallIcon(android.R.drawable.stat_sys_data_bluetooth)
            .setContentIntent(pending)
            .setOngoing(true)
            .build()
    }

    companion object {
        const val DISCOVERY_PORT = 39000
        const val COMMAND_PORT = 39001
        const val ACTION_START = "com.monib.receivercontrol.receiver.START_SERVER"
        const val ACTION_STOP = "com.monib.receivercontrol.receiver.STOP_SERVER"
        const val PREFS = "receiver_control"
        const val CODE_KEY = "pairing_code"
        private const val CHANNEL_ID = "receiver_server"
        private const val NOTIFICATION_ID = 77
        private const val MAX_HEADER_SIZE = 128 * 1024
        private const val MAX_FILE_SIZE = 512L * 1024 * 1024
        const val SERVER_RUNNING_KEY = "server_running"
        @Volatile var running: Boolean = false
            private set
        private val PACKAGE_NAME = Regex("[A-Za-z0-9_]+(\\.[A-Za-z0-9_]+)+")

        fun start(context: Context) {
            val intent = Intent(context, LocalReceiverService::class.java).setAction(ACTION_START)
            if (Build.VERSION.SDK_INT >= 26) context.startForegroundService(intent) else context.
startService(intent)
        }

        fun stop(context: Context) = context.stopService(Intent(context, LocalReceiverService::class.java))
    }
}
```

receiver/src/main/java/com/monib/receivercontrol/receiver/MainActivity.kt

```
package com.monib.receivercontrol.receiver

import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.net.Uri
import android.os.Build
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.result.ActivityResultLauncher
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.core.content.FileProvider
import java.io.File

class MainActivity : ComponentActivity() {
    private lateinit var notificationPermission: ActivityResultLauncher<String>
    private var serverOn by androidx.compose.runtime.mutableStateOf(false)
    private var language by androidx.compose.runtime.mutableStateOf("ar")
    private var theme by androidx.compose.runtime.mutableStateOf("purple")
    private var files by androidx.compose.runtime.mutableStateOf<List<File>>(emptyList())
    private var pairingCode by androidx.compose.runtime.mutableStateOf("")
    private var receiverIp by androidx.compose.runtime.mutableStateOf<String?>(null)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val preferences = getSharedPreferences(LocalReceiverService.PREFS, MODE_PRIVATE)
        language = preferences.getString("language", "ar") ?: "ar"
        theme = preferences.getString("theme", "purple") ?: "purple"
        pairingCode = ReceiverPairing.getOrCreateCode(this)
        receiverIp = ReceiverPairing.lanIpv4Address()
        refreshFiles()
        notificationPermission = registerForActivityResult(ActivityResultContracts.RequestPermission()) {
            startReceiverServer()
        }
        setContent {
            ReceiverDashboard(
                language = language,
                theme = theme,
                serverOn = serverOn,
                pairingCode = pairingCode,
                receiverIp = receiverIp,
                files = files,
                onToggleServer = ::toggleServer,
                onAccessibility = { ReceiverAccessibilityService.openSettings(this) },
                onRefreshFiles = ::refreshFiles,
                onOpenFile = ::openFile,
                onShareFile = ::shareFile,
                onLanguage = {
                    language = it
                    preferences.edit().putString("language", it).apply()
                },
                onTheme = {
                    theme = it
                    preferences.edit().putString("theme", it).apply()
                },
                onRefreshAddress = { receiverIp = ReceiverPairing.lanIpv4Address() }
            )
        }
    }

    override fun onResume() {
        super.onResume()
        serverOn = LocalReceiverService.running
        pairingCode = ReceiverPairing.getOrCreateCode(this)
        receiverIp = ReceiverPairing.lanIpv4Address()
        refreshFiles()
    }

    private fun toggleServer() {
        if (serverOn) {
            LocalReceiverService.stop(this)
            serverOn = false
            return
        }
        if (Build.VERSION.SDK_INT >= 33 && checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS) !=
PackageManager.PERMISSION_GRANTED) {
            notificationPermission.launch(Manifest.permission.POST_NOTIFICATIONS)
        } else startReceiverServer()
    }

    private fun startReceiverServer() {
        LocalReceiverService.start(this)
        serverOn = true
        pairingCode = ReceiverPairing.getOrCreateCode(this)
    }

    private fun refreshFiles() {
        val folder = File(getExternalFilesDir("Download") ?: filesDir, "ReceiverControl")
```

```
        files = folder.listFiles()?.filter(File::isFile)?.sortedByDescending(File::lastModified).orEmpty()
    }

    private fun openFile(file: File) {
        val intent = Intent(this, PlayerActivity::class.java).putExtra(PlayerActivity.EXTRA_PATH, file.
absolutePath)
        startActivity(intent)
    }

    private fun shareFile(file: File) {
        runCatching {
            val uri: Uri = FileProvider.getUriForFile(this, "$packageName.files", file)
            val mime = contentResolver.getType(uri) ?: mimeFor(file)
            val intent = Intent(Intent.ACTION_SEND).setType(mime).putExtra(Intent.EXTRA_STREAM, uri)
                .addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
            startActivity(Intent.createChooser(intent, if (language == "ar") "\u0645\u0634\u0627\u0631\
u0643\u0629 \u0627\u0644\u0645\u0644\u0641" else "Share file"))
        }
    }

    private fun mimeFor(file: File): String = when (file.extension.lowercase()) {
        "jpg", "jpeg" -> "image/jpeg"
        "png" -> "image/png"
        "webp" -> "image/webp"
        "gif" -> "image/gif"
        "mp4", "m4v" -> "video/mp4"
        "mkv" -> "video/x-matroska"
        "webm" -> "video/webm"
        "mov" -> "video/quicktime"
        else -> "application/octet-stream"
    }
}
```

receiver/src/main/java/com/monib/receivercontrol/receiver/PlayerActivity.kt

```
package com.monib.receivercontrol.receiver

import android.graphics.BitmapFactory
import android.net.Uri
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.media3.common.MediaItem
import androidx.media3.exoplayer.ExoPlayer
import androidx.media3.ui.PlayerView
import java.io.File

class PlayerActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val path = intent.getStringExtra(EXTRA_PATH)
        setContent {
            MaterialTheme {
                if (path.isNullOrBlank()) Text("File not found") else ReceiverMediaViewer(File(path))
            }
        }
    }

    companion object { const val EXTRA_PATH = "file_path" }
}

@Composable
private fun ReceiverMediaViewer(file: File) {
    val context = LocalContext.current
    val extension = file.extension.lowercase()
    val isImage = extension in setOf("jpg", "jpeg", "png", "webp", "gif", "bmp")
    val isVideo = extension in setOf("mp4", "m4v", "mkv", "webm", "mov", "3gp", "ts")
    Box(Modifier.fillMaxSize().background(Color.Black), contentAlignment = Alignment.Center) {
        when {
            isImage -> {
                val bitmap = remember(file.absolutePath) { BitmapFactory.decodeFile(file.absolutePath)?.
asImageBitmap() }
                if (bitmap != null) Image(bitmap, contentDescription = file.name, modifier = Modifier.
fillMaxSize())
                else Text("Unable to display image", color = Color.White, modifier = Modifier.padding(24.
dp))
            }
            isVideo -> {
                val player = remember(file.absolutePath) { ExoPlayer.Builder(context).build() }
                LaunchedEffect(player, file.absolutePath) {
                    player.setMediaItem(MediaItem.fromUri(Uri.fromFile(file)))
                    player.prepare()
                    player.playWhenReady = true
                }
                AndroidView(
                    modifier = Modifier.fillMaxSize(),
                    factory = { ctx -> PlayerView(ctx).apply { this.player = player; useController = true }
},
                    update = { it.player = player }
                )
                DisposableEffect(player) { onDispose { player.release() } }
            }
            else -> Text("${file.name}\nUnsupported preview format", color = Color.White, modifier =
Modifier.padding(24.dp))
        }
    }
}
```

receiver/src/main/java/com/monib/receivercontrol/receiver/ReceiverAccessibilityService.kt

```
package com.monib.receivercontrol.receiver

import android.accessibilityservice.AccessibilityService
import android.accessibilityservice.GestureDescription
import android.content.Intent
import android.graphics.Bitmap
import android.graphics.Path
import android.os.Build
import android.os.Handler
import android.os.Looper
import android.provider.Settings
import android.view.accessibility.AccessibilityEvent
import java.io.ByteArrayOutputStream
import java.util.concurrent.Executor
import java.util.concurrent.CountDownLatch
import java.util.concurrent.TimeUnit
import java.util.concurrent.atomic.AtomicBoolean
import kotlin.math.min

class ReceiverAccessibilityService : AccessibilityService() {
    override fun onAccessibilityEvent(event: AccessibilityEvent?) = Unit
    override fun onInterrupt() = Unit

    override fun onServiceConnected() {
        active = this
    }

    override fun onDestroy() {
        if (active === this) active = null
        super.onDestroy()
    }

    companion object {
        @Volatile private var active: ReceiverAccessibilityService? = null

        fun isEnabled(): Boolean = active != null

        fun perform(action: String): Boolean {
            val service = active ?: return false
            val globalAction = when (action) {
                "HOME" -> AccessibilityService.GLOBAL_ACTION_HOME
                "BACK" -> AccessibilityService.GLOBAL_ACTION_BACK
                "LOCK" -> if (android.os.Build.VERSION.SDK_INT >= 28) AccessibilityService.
GLOBAL_ACTION_LOCK_SCREEN else return false
                "RECENTS" -> AccessibilityService.GLOBAL_ACTION_RECENTS
                else -> return false
            }
            return service.performGlobalAction(globalAction)
        }

        fun dispatchTouch(action: String): Boolean {
            val service = active ?: return false
            val metrics = service.resources.displayMetrics
            val centerX = metrics.widthPixels / 2f
            val centerY = metrics.heightPixels / 2f
            val distance = (min(metrics.widthPixels, metrics.heightPixels) * 0.075f).coerceAtLeast(48f)
            val (dx, dy) = when (action) {
                "UP" -> 0f to -distance
                "DOWN" -> 0f to distance
                "LEFT" -> -distance to 0f
                "RIGHT" -> distance to 0f
                "ENTER" -> 1f to 1f
                else -> return false
            }
            val path = Path().apply {
                moveTo(centerX, centerY)
                lineTo(centerX + dx, centerY + dy)
            }
            val duration = if (action == "ENTER") 80L else 180L
            val gesture = GestureDescription.Builder()
                .addStroke(GestureDescription.StrokeDescription(path, 0L, duration))
                .build()
            val accepted = AtomicBoolean(false)
            val latch = CountDownLatch(1)
            Handler(Looper.getMainLooper()).post {
                try {
                    accepted.set(service.dispatchGesture(gesture, null, null))
                } catch (_: Exception) {
                    accepted.set(false)
                } finally {
                    latch.countDown()
                }
            }
            return runCatching { latch.await(1200L, TimeUnit.MILLISECONDS) && accepted.get() }.getOrDefault(
false)
        }

        fun captureScreenshot(onResult: (ByteArray?) -> Unit): Boolean {
            val service = active ?: return false
```

```
            if (Build.VERSION.SDK_INT < 30) return false
            Handler(Looper.getMainLooper()).post {
                try {
                    service.takeScreenshot(
                        android.view.Display.DEFAULT_DISPLAY,
                        Executor { callback -> Thread(callback, "receiver-screen-capture").start() },
                        object : AccessibilityService.TakeScreenshotCallback {
                            override fun onSuccess(result: AccessibilityService.ScreenshotResult) {
                                val buffer = result.hardwareBuffer
                                try {
                                    val source = Bitmap.wrapHardwareBuffer(buffer, result.colorSpace) ?:
run {
                                        onResult(null)
                                        return
                                    }
                                    val scale = minOf(1f, 1280f / source.width, 720f / source.height)
                                    val width = (source.width * scale).toInt().coerceAtLeast(1)
                                    val height = (source.height * scale).toInt().coerceAtLeast(1)
                                    val scaled = Bitmap.createScaledBitmap(source, width, height, true)
                                    val output = ByteArrayOutputStream()
                                    scaled.compress(Bitmap.CompressFormat.JPEG, 72, output)
                                    if (scaled !== source) scaled.recycle()
                                    source.recycle()
                                    onResult(output.toByteArray())
                                } catch (_: Exception) {
                                    onResult(null)
                                } finally {
                                    buffer.close()
                                }
                            }

                            override fun onFailure(errorCode: Int) = onResult(null)
                        }
                    )
                } catch (_: Exception) {
                    onResult(null)
                }
            }
            return true
        }

        fun openSettings(context: android.content.Context) {
            context.startActivity(Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS).addFlags(Intent.
FLAG_ACTIVITY_NEW_TASK))
        }
    }
}
```

receiver/src/main/java/com/monib/receivercontrol/receiver/ReceiverDashboard.kt

```
package com.monib.receivercontrol.receiver

import android.graphics.Bitmap
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.aspectRatio
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Divider
import androidx.compose.material3.FilterChip
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.platform.LocalLayoutDirection
import androidx.compose.ui.unit.LayoutDirection
import androidx.compose.ui.unit.dp
import java.io.File

@Composable
fun ReceiverDashboard(
    language: String,
    theme: String,
    serverOn: Boolean,
    pairingCode: String,
    receiverIp: String?,
    files: List<File>,
    onToggleServer: () -> Unit,
    onAccessibility: () -> Unit,
    onRefreshFiles: () -> Unit,
    onOpenFile: (File) -> Unit,
    onShareFile: (File) -> Unit,
    onLanguage: (String) -> Unit,
    onTheme: (String) -> Unit,
    onRefreshAddress: () -> Unit
) {
    val ar = language == "ar"
    val colors = if (theme == "white") lightColorScheme(primary = Color(0xFF8064DB), background = Color.
White, surface = Color.White, surfaceVariant = Color(0xFFF5F3FA), onSurface = Color(0xFF17151C),
onSurfaceVariant = Color(0xFF5D5966))
    else darkColorScheme(primary = Color(0xFFB7A3FF), background = Color(0xFF111117), surface = Color(
0xFF1D1C25), surfaceVariant = Color(0xFF2A2935), onSurface = Color(0xFFF0EFF7), onSurfaceVariant = Color(
0xFFC5C1D2))
    androidx.compose.runtime.CompositionLocalProvider(LocalLayoutDirection provides if (ar) LayoutDirection.
Rtl else LayoutDirection.Ltr) {
        MaterialTheme(colorScheme = colors) {
            varPage(language, theme, serverOn, pairingCode, receiverIp, files, onToggleServer,
onAccessibility, onRefreshFiles, onOpenFile, onShareFile, onLanguage, onTheme, onRefreshAddress)
        }
    }
}

@Composable
private fun varPage(
    language: String,
    theme: String,
    serverOn: Boolean,
    pairingCode: String,
    receiverIp: String?,
    files: List<File>,
    onToggleServer: () -> Unit,
    onAccessibility: () -> Unit,
```

```
    onRefreshFiles: () -> Unit,
    onOpenFile: (File) -> Unit,
    onShareFile: (File) -> Unit,
    onLanguage: (String) -> Unit,
    onTheme: (String) -> Unit,
    onRefreshAddress: () -> Unit
) {
    val ar = language == "ar"
    var tab by remember { androidx.compose.runtime.mutableStateOf("home") }
    Scaffold(containerColor = MaterialTheme.colorScheme.background) { padding ->
        Column(Modifier.fillMaxSize().padding(padding).padding(horizontal = 16.dp)) {
            Row(Modifier.fillMaxWidth().padding(vertical = 12.dp), horizontalArrangement = Arrangement.
SpaceBetween, verticalAlignment = Alignment.CenterVertically) {
                Column {
                    Text("Android TV responsiveness", style = MaterialTheme.typography.titleLarge)
                    Text(if (ar) "\u062A\u0637\u0628\u064A\u0642 \u0627\u0644\u0631\u0633\u064A\u0641\
u0631" else "Receiver companion", color = MaterialTheme.colorScheme.onSurfaceVariant)
                }
                Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                    TextButton(onClick = { tab = "home" }) { Text(if (ar) "\u0627\u0644\u062D\u0627\u0644\
u0629" else "Status") }
                    TextButton(onClick = { tab = "files"; onRefreshFiles() }) { Text(if (ar) "\u0645\u0644\
u0641\u0627\u062A\u064A" else "My files") }
                }
            }
            if (tab == "files") {
                FileManager(files, language, onRefreshFiles, onOpenFile, onShareFile)
            } else {
                Column(Modifier.fillMaxSize().verticalScroll(rememberScrollState()), verticalArrangement =
Arrangement.spacedBy(13.dp)) {
                    Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.
surface)) {
                        Column(Modifier.fillMaxWidth().padding(16.dp), verticalArrangement = Arrangement.
spacedBy(8.dp)) {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Surface(Modifier.size(10.dp), shape = CircleShape, color = if (serverOn)
Color(0xFF55C98B) else Color(0xFF9B98A5)) {}
                                Spacer(Modifier.width(9.dp))
                                Text(if (serverOn) (if (ar) "\u0627\u0644\u062E\u0627\u062F\u0645 \u064A\
u0639\u0645\u0644" else "Server running") else (if (ar) "\u0627\u0644\u062E\u0627\u062F\u0645 \u0645\u062A\
u0648\u0642\u0641" else "Server stopped"), style = MaterialTheme.typography.titleMedium)
                            }
                            Text(if (ar) "\u0634\u063A\u0651\u0644 \u0627\u0644\u062E\u0627\u062F\u0645 \
u0644\u064A\u0643\u062A\u0634\u0641 \u0627\u0644\u0647\u0627\u062A\u0641 \u0647\u0630\u0627 \u0627\u0644\
u0631\u0633\u064A\u0641\u0631 \u0648\u064A\u062A\u0635\u0644 \u0628\u0647 \u0639\u0628\u0631 Wi-Fi." else
"Start the server so your phone can discover and connect to this receiver over Wi-Fi.", color =
MaterialTheme.colorScheme.onSurfaceVariant)
                            Button(onClick = onToggleServer, modifier = Modifier.fillMaxWidth()) {
                                Text(if (serverOn) (if (ar) "\u0625\u064A\u0642\u0627\u0641 \u0627\u0644\
u062E\u0627\u062F\u0645" else "Stop server") else (if (ar) "\u062A\u0634\u063A\u064A\u0644 \u062E\u0627\
u062F\u0645 \u0627\u0644\u0627\u062A\u0635\u0627\u0644" else "Start connection server"))
                            }
                        }
                    }
                    Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.
surface)) {
                        Column(Modifier.fillMaxWidth().padding(16.dp), horizontalAlignment = Alignment.
CenterHorizontally, verticalArrangement = Arrangement.spacedBy(8.dp)) {
                            Text(if (ar) "\u0627\u0631\u0628\u0637 \u0627\u0644\u0647\u0627\u062A\u0641 \
u0628\u0647\u0630\u0627 \u0627\u0644\u0631\u0645\u0632" else "Pair your phone with this code", style =
MaterialTheme.typography.titleMedium)
                            Text(pairingCode.chunked(1).joinToString(" "), style = MaterialTheme.typography.
headlineMedium, color = MaterialTheme.colorScheme.primary)
                            Text(if (receiverIp != null) "IP: $receiverIp \u00B7 Port ${
LocalReceiverService.COMMAND_PORT}" else if (ar) "\u0627\u062A\u0635\u0644 \u0628\u0634\u0628\u0643\u0629
Wi-Fi \u0644\u0639\u0631\u0636 IP \u0648\u0631\u0645\u0632 QR" else "Connect to Wi-Fi to show the IP and QR
code", color = MaterialTheme.colorScheme.onSurfaceVariant)
                            val payload = remember(receiverIp, pairingCode) { ReceiverPairing.qrPayloadFrom(
receiverIp, pairingCode) }
                            val qr = remember(payload) { payload?.let { ReceiverPairing.qrBitmap(it) } }
                            if (qr != null) Image(qr.asImageBitmap(), contentDescription = "Receiver
pairing QR", modifier = Modifier.size(220.dp).background(Color.White, RoundedCornerShape(8.dp)).padding(5.
dp))
                            OutlinedButton(onClick = onRefreshAddress) { Text(if (ar) "\u062A\u062D\u062F\
u064A\u062B \u0639\u0646\u0648\u0627\u0646 \u0627\u0644\u0634\u0628\u0643\u0629" else "Refresh network
address") }
                        }
                    }
                    Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.
surfaceVariant)) {
                        Column(Modifier.fillMaxWidth().padding(16.dp), verticalArrangement = Arrangement.
spacedBy(8.dp)) {
                            Text(if (ar) "\u0627\u0644\u062A\u062D\u0643\u0645 \u0628\u0627\u0644\u0623\
u0632\u0631\u0627\u0631" else "System button controls", style = MaterialTheme.typography.titleMedium)
                            Text(if (ar) "\u0644\u0623\u0632\u0631\u0627\u0631 \u0627\u0644\u0644\u0631\u0626\
u064A\u0633\u064A\u0629 \u0648\u0627\u0644\u0631\u062C\u0648\u0639 \u0648\u0642\u0641\u0644 \u0627\u0644\
u0634\u0627\u0634\u0629\u060C \u0641\u0639\u0651\u0644 \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\u062D\
u0643\u0645 \u0627\u0644\u0645\u0633\u0627\u0639\u062F. \u0627\u0644\u062A\u062D\u0643\u0645 \u0628\u0627\
u0644\u0635\u0648\u062A \u064A\u0639\u0645\u0644 \u062F\u0648\u0646 \u0627\u0644\u062A\u0627\u0634.", color =
MaterialTheme.colorScheme.onSurfaceVariant)
                            OutlinedButton(onClick = onAccessibility, modifier = Modifier.fillMaxWidth()) {
Text(if (ar) "\u0625\u0639\u062F\u0627\u062F \u062E\u062F\u0645\u0629 \u0627\u0644\u062A\u062D\u0643\u0645 \
u0627\u0644\u0645\u0633\u0627\u0639\u062F" else "Configure accessibility control") }
                        }
```

```
                    }
                    Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.
surface)) {
                        Column(Modifier.fillMaxWidth().padding(16.dp), verticalArrangement = Arrangement.
spacedBy(8.dp)) {
                            Text(if (ar) "\u0627\u0644\u0645\u0638\u0647\u0631 \u0648\u0627\u0644\u0644\
u063A\u0629" else "Appearance and language", style = MaterialTheme.typography.titleMedium)
                            Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                                FilterChip(selected = theme == "purple", onClick = { onTheme("purple") },
label = { Text(if (ar) "\u0628\u0646\u0641\u0633\u062C\u064A \u0631\u0645\u0627\u062F\u064A" else "Purple
gray") })
                                FilterChip(selected = theme == "white", onClick = { onTheme("white") },
label = { Text(if (ar) "\u0623\u0628\u064A\u0636" else "White") })
                            }
                            Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                                FilterChip(selected = ar, onClick = { onLanguage("ar") }, label = { Text("\
u0627\u0644\u0639\u0631\u0628\u064A\u0629") })
                                FilterChip(selected = !ar, onClick = { onLanguage("en") }, label = { Text(
"English") })
                            }
                        }
                    }
                }
            }
        }
    }
}

@Composable
private fun FileManager(files: List<File>, language: String, onRefresh: () -> Unit, onOpen: (File) -> Unit,
onShare: (File) -> Unit) {
    val ar = language == "ar"
    Column(Modifier.fillMaxSize()) {
        Row(Modifier.fillMaxWidth().padding(vertical = 6.dp), horizontalArrangement = Arrangement.
SpaceBetween, verticalAlignment = Alignment.CenterVertically) {
            Column {
                Text(if (ar) "\u0645\u062F\u064A\u0631 \u0627\u0644\u0645\u0644\u0641\u0627\u062A" else
"File manager", style = MaterialTheme.typography.titleLarge)
                Text(if (ar) "\u0627\u0644\u0645\u0644\u0641\u0627\u062A \u0627\u0644\u0645\u0633\u062A\
u0644\u0645\u0629 \u0645\u0646 \u0647\u0627\u062A\u0641\u0643" else "Files received from your phone", color
= MaterialTheme.colorScheme.onSurfaceVariant)
            }
            OutlinedButton(onClick = onRefresh) { Text(if (ar) "\u062A\u062D\u062F\u064A\u062B" else
"Refresh") }
        }
        Divider()
        Column(Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(top = 10.dp),
verticalArrangement = Arrangement.spacedBy(8.dp)) {
            files.forEach { file ->
                Card(colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)) {
                    Column(Modifier.fillMaxWidth().padding(12.dp), verticalArrangement = Arrangement.
spacedBy(6.dp)) {
                        Text(file.name, style = MaterialTheme.typography.titleSmall)
                        Text("${formatFileSize(file.length())} \u00B7 ${file.extension.uppercase()}", color
= MaterialTheme.colorScheme.onSurfaceVariant, style = MaterialTheme.typography.bodySmall)
                        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                            OutlinedButton(onClick = { onOpen(file) }, modifier = Modifier.weight(1f)) {
Text(if (ar) "\u0641\u062A\u062D" else "Open") }
                            TextButton(onClick = { onShare(file) }, modifier = Modifier.weight(1f)) { Text(
if (ar) "\u0645\u0634\u0627\u0631\u0643\u0629" else "Share") }
                        }
                    }
                }
            }
            if (files.isEmpty()) Text(if (ar) "\u0644\u0627 \u062A\u0648\u062C\u062F \u0645\u0644\u0641\
u0627\u062A \u0645\u0633\u062A\u0644\u0645\u0629 \u0628\u0639\u062F." else "No received files yet.",
modifier = Modifier.padding(top = 18.dp), color = MaterialTheme.colorScheme.onSurfaceVariant)
        }
    }
}

private fun formatFileSize(bytes: Long): String = when {
    bytes >= 1024L * 1024 * 1024 -> "%.1f GB".format(bytes / (1024.0 * 1024 * 1024))
    bytes >= 1024L * 1024 -> "%.1f MB".format(bytes / (1024.0 * 1024))
    else -> "%.0f KB".format(bytes / 1024.0)
}
```

receiver/src/main/java/com/monib/receivercontrol/receiver/ReceiverPairing.kt

```
package com.monib.receivercontrol.receiver

import android.content.Context
import android.graphics.Bitmap
import com.google.gson.JsonObject
import com.google.zxing.BarcodeFormat
import com.google.zxing.EncodeHintType
import com.google.zxing.qrcode.QRCodeWriter
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel
import java.net.Inet4Address
import java.net.NetworkInterface
import java.security.SecureRandom

object ReceiverPairing {
    fun getOrCreateCode(context: Context): String {
        val prefs = context.getSharedPreferences(LocalReceiverService.PREFS, Context.MODE_PRIVATE)
        return prefs.getString(LocalReceiverService.CODE_KEY, null) ?:
            (1..7).joinToString("") { SecureRandom().nextInt(10).toString() }
                .also { prefs.edit().putString(LocalReceiverService.CODE_KEY, it).apply() }
    }

    fun lanIpv4Address(): String? = runCatching {
        NetworkInterface.getNetworkInterfaces().toList()
            .filter { it.isUp && !it.isLoopback }
            .sortedByDescending { if (it.name.startsWith("wlan") || it.name.startsWith("eth")) 1 else 0 }
            .flatMap { it.inetAddresses.toList() }
            .firstOrNull { it is Inet4Address && !it.isLoopbackAddress && !it.isLinkLocalAddress }
            ?.hostAddress
    }.getOrNull()

    fun qrPayload(context: Context): String? {
        val ip = lanIpv4Address() ?: return null
        return qrPayloadFrom(ip, getOrCreateCode(context))
    }

    fun qrPayloadFrom(ip: String?, pairingCode: String): String? {
        if (ip.isNullOrBlank()) return null
        val json = JsonObject().apply {
            addProperty("type", "ReceiverControlPairing")
            addProperty("ip", ip)
            addProperty("port", LocalReceiverService.COMMAND_PORT)
            addProperty("code", pairingCode)
            addProperty("name", android.os.Build.MODEL.orEmpty().ifBlank { "Android Receiver" })
        }
        return json.toString()
    }

    fun qrBitmap(payload: String, size: Int = 360): Bitmap? = runCatching {
        val hints = mapOf(EncodeHintType.ERROR_CORRECTION to ErrorCorrectionLevel.M, EncodeHintType.MARGIN
to 1)
        val matrix = QRCodeWriter().encode(payload, BarcodeFormat.QR_CODE, size, size, hints)
        Bitmap.createBitmap(size, size, Bitmap.Config.ARGB_8888).also { bitmap ->
            for (x in 0 until size) for (y in 0 until size) {
                bitmap.setPixel(x, y, if (matrix[x, y]) android.graphics.Color.BLACK else android.graphics.
Color.WHITE)
            }
        }
    }.getOrNull()
}
```